

Title Page

UniManage - University Course Management System

Application Development Coursework 2 Report

Module: CS6004ES Application Development

Assessment: Coursework 2

Technology Used: ASP.NET Core MVC, C#, Visual Studio, Entity Framework Core, MySQL, ASP.NET Core Identity, Razor Views, Bootstrap

Centre: ESOF Metro Campus / London Metropolitan University

Student Name: Yasas Pasindu Fernando

Student ID: 25026764

Submission Type: Approved individual submission

Lecturer / Module Leader: [LECTURER NAME TO BE CONFIRMED]

Submission Date: 5 May 2026

Declaration / Academic Integrity Note

The author confirms that this report and the accompanying UniManage application have been prepared and submitted as the author's own work for CS6004ES Application Development Coursework 2. The coursework was originally set as group work. Following approval from the lecturer and module team, the author has completed the implementation, testing, documentation, and final preparation as an individual submission.

External resources, official documentation, and tutorials are acknowledged where they have informed the work. No part of the report has been copied from another student. Where claims cannot be supported by source code, screenshots, or build evidence, those claims are clearly marked as [NEEDS CONFIRMATION] or [SCREENSHOT REQUIRED] in the body of the report.

Acknowledgement

The author would like to thank the lecturer and module team for the guidance provided throughout the CS6004ES Application Development module and for the approval to complete this coursework as an individual submission. The author also acknowledges the

value of the official Microsoft Learn documentation for ASP.NET Core, the Entity Framework Core documentation, the Pomelo MySQL provider documentation, the QuestPDF documentation, and the Bootstrap framework documentation, which informed the technical decisions throughout the project.

The author also recognises ESOF Metro Campus and London Metropolitan University for the structured academic environment that has supported this work.

Abstract

UniManage is a University Course Management System developed for CS6004ES Application Development Coursework 2 as an ASP.NET Core MVC web application targeting .NET 8. The system supports the academic administration workflow of a university by providing user registration and login, role-based access control for Student, Lecturer, and Administrator users, role-specific dashboards, course management, an enrolment system, assignment creation, submission and grading, reporting and analytics with CSV and PDF exports, internal messaging, meeting scheduling, audit logging, and a security pipeline that includes anti-forgery, rate limiting, secure cookies, and security headers.

The implementation has been confirmed by source-code review of twelve controllers, twelve domain models, sixteen view models, twelve infrastructure components, forty-six Razor views, three Entity Framework Core migrations, an ApplicationDbContext that inherits from IdentityDbContext, and a Program.cs configuration that registers Identity, EF Core with the Pomelo MySQL provider, rate limiting, anti-forgery, exception handling, and HTTPS redirection. The build has been verified locally with dotnet build --no-restore, producing zero warnings and zero errors.

The report documents the academic problem, the requirement analysis, the system architecture, the database design, the implementation, the user interface, the validation and exception handling, the security controls, the testing approach, the evaluation against the coursework brief, the challenges faced, the limitations, and the author's reflection. The report is written and submitted as an approved individual submission.

Table of Contents

Right-click and choose Update Field to populate the Table of Contents.

List of Figures

Right-click and choose Update Field to populate the List of Figures.

List of Tables

Right-click and choose Update Field to populate the List of Tables.

Abbreviations and Glossary

AD: Application Development

API: Application Programming Interface

ASP.NET: Active Server Pages .NET

CDN: Content Delivery Network

CRUD: Create, Read, Update, Delete

CSRF: Cross-Site Request Forgery

CSS: Cascading Style Sheets

CSV: Comma-Separated Values

DI: Dependency Injection

DB: Database

DTO: Data Transfer Object

EF Core: Entity Framework Core

ERD: Entity Relationship Diagram

HSTS: HTTP Strict Transport Security

HTML: HyperText Markup Language

HTTPS: HyperText Transfer Protocol Secure

ICS: iCalendar file format used to export meetings

ID: Identifier

JS: JavaScript

JSON: JavaScript Object Notation

MVC: Model View Controller

ORM: Object Relational Mapping

OWASP: Open Web Application Security Project

PDF: Portable Document Format

RBAC: Role-Based Access Control

SDK: Software Development Kit

SQL: Structured Query Language

UCMS: University Course Management System

UI: User Interface

URL: Uniform Resource Locator

UTC: Coordinated Universal Time

UX: User Experience

Glossary terms:

- **ApplicationUser**: the project's user entity that extends **IdentityUser** and stores additional profile fields such as full name and phone number.
- **ApplicationDbContext**: the EF Core context used by **UniManage**. It inherits from **IdentityDbContext<ApplicationUser>** and exposes **DbSets** for the academic entities.
- **DbInitializer**: the seeding component that applies migrations and creates demonstration roles, users, and sample data.
- **AppRoles**: a static class that holds the role name constants used by the project (Administrator, Lecturer, Student).
- **ViewModel**: a class designed for transferring data between controllers and views, used to keep entity classes separate from presentation concerns.
- **Razor view**: a .cshtml server-side template that combines HTML markup with C# logic to render the user interface.

1. Introduction

1.1 Background and Coursework Context

CS6004ES Application Development at ESOF Metro Campus and London Metropolitan University focuses on building a working web application that demonstrates practical

software engineering. Coursework 2 requires the design, implementation, testing, and documentation of a university-style management system that supports realistic academic workflows. The brief sets a series of marking items, including the application user interface, user registration and login, role dashboards, course management, an enrolment system, assignment and grading, reporting and analytics, a communication module, security and data protection, an installation guide and user manual, a concise logical solution, architecture diagrams, a detailed description of classes and methods, a personal reflection, programming style, validation and exception handling, and interface design and usability.

UniManage is the University Course Management System produced by the author to satisfy this brief. The application has been built using ASP.NET Core MVC, C#, and Visual Studio. Entity Framework Core has been used as the object relational mapper, and the Pomelo provider has been used to connect to a MySQL 8 database. ASP.NET Core Identity has been used for authentication and roles. Razor views and Bootstrap have been used for the user interface. The choice of technologies reflects current professional practice for .NET web development, and is suitable for a coursework that asks for a maintainable, secure, and well-structured application.

The system supports three actors. A Student can register, log in, view a personal dashboard, browse courses, enrol on eligible courses, view and submit assignments, view grades and feedback, send and receive messages with lecturers, and view scheduled meetings. A Lecturer can log in, view a teaching dashboard, manage assignments for owned courses, upload course materials, grade submissions, send and receive messages with enrolled students, and schedule meetings. An Administrator can log in, view a system-wide dashboard, manage courses, view audit logs, and produce reports. A dedicated administrator user-management screen has not been confirmed in the source code and is therefore marked [NEEDS CONFIRMATION] throughout the report.

1.2 Purpose of the Report

The purpose of this report is to provide a complete and evidence-based account of the UniManage project. It is written for the marker, an external moderator, and any reader who needs to understand how the application addresses the coursework brief. Specifically, the report has the following purposes:

- It explains the academic problem that UniManage solves and the way that problem has been broken down into actors, use cases, and software components.

- It documents the architecture, database design, and implementation of the system, with reference to actual source files in the project.
- It records the validation, exception handling, and security controls that have been implemented in the code.
- It records the test plan and the verification activities that have been carried out.
- It evaluates the implementation against the marking items in the coursework brief.
- It reflects on the author's experience of completing the work as an approved individual submission and identifies future improvements.

The report is therefore both a technical record and an academic reflection. The author has used direct source-code references to keep the technical record honest and has used a personal voice in the reflection chapters to make the academic experience visible.

1.3 Approved Individual Submission Context

The CS6004ES Application Development Coursework 2 was originally set as group work. Following approval from the lecturer and module team, the author has completed the implementation, testing, documentation, and final preparation as an individual submission. The report does not include any group contribution table, group task allocation, or comparison of work between students. It does not name other students and does not describe any conflict. The author refers to themselves as "the author" in the body chapters and uses the first person only in the reflection chapter, where personal experience is appropriate.

This wording approach has two effects. First, it keeps the report neutral, professional, and easy to mark. Second, it confirms academic integrity, because the author claims authorship only for the work that has actually been delivered and verified by source code, build, and screenshot evidence.

1.4 Scope of the UniManage Application

The scope of UniManage is defined by the actors that the system supports, by the academic workflows that the system implements, and by the boundaries that the system deliberately does not cross.

The system is in scope when:

- A user registers, logs in, logs out, or recovers a password.
- A Student browses, enrolls, views or submits an assignment, views grades, sends or receives messages, or views meetings.

- A Lecturer manages assignments for an owned course, uploads materials, grades submissions, sends or receives messages, or schedules meetings for an owned course.
- An Administrator manages the course catalogue, views reports, and reviews audit logs.

The system is out of scope when:

- An external timetable system is required to import or export class schedules.
- A finance or fees subsystem is required.
- A live video conference is required inside the application; UniManage stores meeting URLs and joining links rather than hosting video conferences.
- A mobile application is required; UniManage is a responsive web application accessed through a browser.
- A complex content management system for marketing pages is required; UniManage focuses on academic operations.

The boundary is enforced both at the user interface and at the controller layer. Each controller action is restricted to the appropriate role through [Authorize] attributes, and ownership checks are applied where a lecturer must only see or modify their own course or assignment. Where the source code does not implement a feature, the report records the absence honestly rather than describing a feature that does not exist.

1.5 Report Structure

The report is structured into front matter, twenty-three numbered chapters, and ten appendices. After the front matter, Chapter 1 introduces the project. Chapter 2 describes the academic problem and the stakeholders. Chapter 3 sets out the aim, objectives, and scope. Chapter 4 explains the technologies and software-engineering principles. Chapter 5 records the requirements. Chapter 6 explains the architecture and design diagrams. Chapter 7 records the database design. Chapter 8 explains the implementation in detail. Chapter 9 discusses the user interface and usability. Chapter 10 lists and explains the main classes, properties, and methods. Chapter 11 walks through the logical solution. Chapter 12 covers validation and exception handling. Chapter 13 covers security and data protection. Chapter 14 covers programming style and maintainability. Chapter 15 records the testing approach and results. Chapter 16 evaluates the work against the coursework brief. Chapter 17 records the challenges faced. Chapter 18 records the limitations. Chapter 19 contains the author's reflection. Chapter 20 records the individual contribution. Chapter 21 describes future

improvements. Chapter 22 is the conclusion. Chapter 23 lists the references. The appendices contain supporting evidence.

2. Problem Description and Case Study Analysis

2.1 Academic Administration Problem

A modern university produces and processes a large amount of academic information every day. Students need to know which courses they are enrolled on, what assignments are due, and what feedback they have received. Lecturers need to know which courses they teach, which students are enrolled on those courses, what assignments have been set, and what submissions are pending review. Administrators need to know which courses run in the current academic period, which lecturers are teaching, how many students are enrolled, how many assignments have been set, and whether the system is being used securely.

When this information is held in scattered spreadsheets, paper records, or unconnected tools, errors creep in. Students enrol twice on the same course. Capacity is exceeded because there is no automatic check. Submissions are lost because there is no central record of who submitted what and when. Grades are inconsistent because feedback is held in private files. Communication between lecturer and student is split across email, instant messengers, and printed notices. Reports are produced manually at the end of the academic period and are sometimes inaccurate.

UniManage addresses this academic administration problem by providing a single web application that records the academic data, enforces business rules through the controller layer, and presents role-aware dashboards to each user. The intention is not to replace specialist systems for finance, payroll, or live video conferencing. The intention is to provide a clean, secure, and reliable foundation for the academic operations of a university course management system.

2.2 Stakeholder Analysis

The stakeholders for UniManage are the people who use the system, the people who run the system, and the people who depend on the data that the system produces. Table 10 summarises the main stakeholders, their roles, their main goals, and the way that the system supports each goal.

Table 10: Stakeholder Analysis

Stakeholder	Role	Main Goals	How UniManage
-------------	------	------------	---------------

			Supports the Goals
Student	End user	Find courses, enrol on eligible courses, complete assignments, view grades, communicate with lecturers, plan meetings.	Student dashboard with enrolled courses, deadlines, grades, messages, calendar. Course browse with eligibility checks. Submission and grading screens. Inbox with allowed-recipient messaging.
Lecturer	End user	Manage owned courses, set and grade assignments, share materials, communicate with enrolled students, plan meetings.	Lecturer dashboard with teaching summary, pending submissions, recent activity. MyCourses view, assignment management, grading screen, materials upload, scheduling, inbox.
Administrator	End user	Manage the course catalogue, view system-wide reports, oversee security, review audit logs.	Administrator dashboard with system counts and popular courses, course CRUD, reporting suite, audit log review.
Module leader / lecturer team	Marker / overseer	Verify that the application meets the coursework brief and that the documentation is evidence based.	Marking-aligned report, evidence checklist, build verification, source-code references.
ESOFT / London Met administration	Internal stakeholder	Confirm coursework completion in line with academic standards.	Approved individual submission record, declaration, formal report.
Author	Developer	Deliver a complete and verifiable solution as an approved individual submission.	Source code, documentation, screenshots, evidence appendices.

The marker is treated as a primary reader of the report. The system has therefore been described in a marker-friendly way, with cross-references between requirements, source code, screenshots, and tests. Sections that cannot be supported by the source code have been clearly marked rather than embellished.

2.3 Problems Faced by Students

Students typically face several problems when academic information is scattered or inconsistent. They struggle to find a complete list of available courses with their prerequisites and capacity. They often miss deadlines because there is no single calendar of due dates. They cannot easily verify that an enrolment has been recorded. They cannot see grades and feedback in a structured format and instead rely on email threads. They have no clear way to message a lecturer in a record-keeping format that respects ownership and privacy.

UniManage addresses these problems through the Student dashboard, the course browse view, the enrolment workflow with capacity, duplicate, and prerequisite checks, the assignment and submission screens, the grading view, the inbox, and the meetings calendar. Each of these features is accessible from a single signed-in session, which removes the cognitive load of switching between unrelated tools.

2.4 Problems Faced by Lecturers

Lecturers also face common problems. They need to know which students are enrolled on each course they teach. They need to record grades and feedback in a structured way that the student can verify. They need to share teaching materials in a controlled environment that respects copyright and access rules. They need to plan meetings, including online meetings, and provide joining links to the right group. They need to communicate with students in a way that is respectful of role boundaries.

UniManage addresses these problems through the Lecturer dashboard, MyCourses, the assignment management screens, the grading screen, the course material upload, the meeting scheduler, and the messaging module restricted to allowed recipients. The combination of ownership filters and authorisation attributes keeps each lecturer focused on the data that belongs to them.

2.5 Problems Faced by Administrators

Administrators are responsible for the integrity of the academic catalogue and for visibility over how the system is used. They face problems such as duplicated course codes, courses without lecturers, courses set with impossible enrolment limits, and a lack of audit traceability for security-sensitive actions.

UniManage addresses these problems through the Administrator dashboard with system counts and popular courses, the course CRUD screens with unique-code validation, the reporting suite (course popularity, student performance, lecturer workload, enrolments, pass and fail, assignment attendance), the audit log review screen, and the export options for CSV and PDF reports. Where a dedicated administrator user-management screen would also be useful, the source-code review has not confirmed one, and this is therefore noted as [NEEDS CONFIRMATION] in the relevant chapters.

2.6 Success Criteria for the Solution

The success criteria for UniManage are derived from the coursework brief and from the stakeholder analysis. They are summarised below.

- The application can be built locally with dotnet build --no-restore without errors.
- The application starts on <https://localhost:7212> or <http://localhost:5103> and seeds demonstration roles and users automatically.
- A user can register, log in, and log out.
- A Student can browse courses, enrol on an eligible course, submit an assignment, view grades, message a lecturer linked to an enrolled course, and view meetings.
- A Lecturer can manage assignments for an owned course, grade a submission, upload a course material, schedule a meeting for an owned course, and message a student enrolled on an owned course.
- An Administrator can list, create, edit, and delete courses, view system-wide reports, and review the audit log.
- Validation, anti-forgery, rate limiting, secure cookies, and security headers are present and active.
- Build, screenshot, diagram, repository, and database evidence have been collected for the final submission.

The chapters that follow show how UniManage meets each of these success criteria and how the coursework brief has been addressed.

3. Aim, Objectives and Functional Scope

3.1 Aim

The aim of the project is to design, implement, document, and evaluate UniManage as an ASP.NET Core MVC University Course Management System that supports role-based academic workflows, enforces appropriate security and validation, and is delivered as an approved individual submission for CS6004ES Application Development Coursework 2.

3.2 Objectives

The aim is broken down into the following SMART objectives:

- O1. To implement registration and login using ASP.NET Core Identity, with strong password rules, account lockout, anti-forgery, and audit logging on success and failure.

- O2. To implement role-based access for Student, Lecturer, and Administrator users, with [Authorize] attributes on controllers and dashboard redirection by role.
- O3. To provide a Student dashboard, a Lecturer dashboard, and an Administrator dashboard that present role-relevant data using EF Core queries and view models.
- O4. To implement course management, including create, read, update, delete, search, paging, lecturer assignment, prerequisite handling, capacity, and material upload.
- O5. To implement an enrolment workflow with capacity, duplicate, and prerequisite checks.
- O6. To implement assignment management, submission upload, and grading with feedback, score range validation, and ownership checks.
- O7. To implement reporting and analytics including course popularity, student performance, lecturer workload, enrolments, pass and fail, and assignment attendance, with CSV and PDF export support.
- O8. To implement an internal communication module with inbox, thread, reply, compose, and allowed-recipient validation, plus meeting scheduling and ICS export.
- O9. To implement validation, exception handling, anti-forgery, rate limiting, secure cookies, security headers, and audit logging.
- O10. To prepare a complete documentation package including a main report, an individual contribution report, a user manual, evidence checklists, diagrams, screenshots, and final QA reports.
- O11. To submit the final coursework as an approved individual submission, with evidence-based wording and no group contribution material.

3.3 Functional Scope

The functional scope is the set of features that are inside the system boundary. Table 11 summarises these features.

Table 11: Functional Scope Inclusions

ID	Feature	Source-Code Evidence
FS1	User registration, login, logout, password reset	Controllers/AccountController.cs, ViewModels/RegisterViewModel.cs, ViewModels/LoginViewModel.cs, ViewModels/ForgotPasswordViewModel.cs, ViewModels/ResetPasswordViewModel.cs, Views/Account/
FS2	Optional Google login if configured	Program.cs (Google auth registration), Controllers/AccountController.cs
FS3	Role-based access control	Models/AppRoles.cs, [Authorize] attributes in

		controllers, Data/DbInitializer.cs
FS4	Student dashboard	Controllers/DashboardController.cs, ViewModels/DashboardViewModels.cs, Views/Dashboard/Student.cshtml
FS5	Lecturer dashboard	Controllers/DashboardController.cs, ViewModels/DashboardViewModels.cs, Views/Dashboard/Lecturer.cshtml
FS6	Administrator dashboard	Controllers/DashboardController.cs, ViewModels/DashboardViewModels.cs, Views/Dashboard/Administrator.cshtml
FS7	Course management with CRUD, search, paging	Controllers/CoursesController.cs, Models/Course.cs, ViewModels/CourseInputViewModel.cs, Views/Courses/
FS8	Course material upload	Controllers/CoursesController.cs upload action, Models/CourseMaterial.cs, ViewModels/MaterialUploadViewModel.cs, Infrastructure/UploadedFileStore.cs
FS9	Enrolment with capacity, duplicate, and prerequisite checks	Controllers/EnrollmentsController.cs, Models/Enrollment.cs, ViewModels/CourseBrowseRowViewModel.cs, Views/Enrollments/Browse.cshtml
FS10	Assignment management for lecturers	Controllers/AssignmentsController.cs, Models/Assignment.cs, ViewModels/AssignmentInputViewModel.cs, Views/Assignments/
FS11	Submission upload and grading	Controllers/SubmissionsController.cs, Models/Submission.cs, Models/SubmissionStatus.cs, ViewModels/SubmissionSubmitViewModel.cs, ViewModels/GradeSubmissionViewModel.cs, Views/Submissions/
FS12	Reporting and analytics with CSV and PDF	Controllers/ReportsController.cs, Infrastructure/CsvWriter.cs, Infrastructure/PdfReport.cs, Views/Reports/
FS13	Internal messaging module	Controllers/MessagesController.cs, Models/Message.cs, ViewModels/MessageInboxRowViewModel.cs, ViewModels/MessageThreadViewModel.cs, ViewModels/MessageComposeViewModel.cs, Views/Messages/
FS14	Meeting scheduling and ICS export	Controllers/MeetingsController.cs, Models/Meeting.cs, ViewModels/MeetingInputViewModel.cs, Infrastructure/CalendarLink.cs, Infrastructure/MeetLinkGenerator.cs, Views/Meetings/
FS15	Profile management and password change	Controllers/ProfileController.cs, ViewModels/ProfileViewModel.cs, Views/Profile/
FS16	Audit logging and audit log review	Models/AuditLog.cs, Infrastructure/AuditLogger.cs, Infrastructure/IAuditLogger.cs, Controllers/AuditLogsController.cs, Views/AuditLogs/Index.cshtml
FS17	Security pipeline	Program.cs (Identity, anti-forgery, rate limiting, HTTPS, HSTS),

		Infrastructure/SecurityHeadersMiddleware.cs
FS18	Email service for password reset	Infrastructure/IEmailService.cs, Infrastructure/SmtpEmailService.cs, Infrastructure/EmailSettings.cs, Infrastructure/EmailTemplates.cs

Features that are deliberately out of scope are explained in Section 1.4. Features that are not confirmed by source code are marked [NEEDS CONFIRMATION] where they appear, including the dedicated administrator user-management screen, automated test projects, and hosted deployment.

3.4 Non-Functional Scope

Table 12 summarises the non-functional quality attributes that the system aims to satisfy.

Table 12: Non-Functional Scope and Quality Attributes

ID	Quality Attribute	Description	Evidence in Project
NF1	Usability	Role-aware navigation, dashboards, alert partial, validation feedback.	Views/Shared/_Layout.cshtml, Views/Shared/_Alerts.cshtml, role views, Bootstrap.
NF2	Performance	EF Core queries use filtering, paging, and AsNoTracking for read-only views; rate limiting limits abusive traffic.	EF Core queries in controllers, Program.cs rate limiting setup.
NF3	Reliability	Retry on failure for MySQL connection, exception handling middleware, audit logging of failures.	Program.cs EnableRetryOnFailure, UseExceptionHandler, AuditLogger.
NF4	Security	Identity, RBAC, anti-forgery, rate limiting, security headers, secure cookies, file validation, audit logging.	Program.cs, Infrastructure/SecurityHeadersMiddleware.cs, controller checks.
NF5	Maintainability	Layered MVC structure with Controllers, Models, ViewModels, Views, Data, Infrastructure folders, dependency injection, single-responsibility services.	Folder layout, Program.cs service registration.
NF6	Data integrity	EF Core relationships, unique course code, unique enrolment per student/course, data annotation rules.	Data/ApplicationDbContext.cs, model annotations.
NF7	Accessibility	Skip link, semantic markup, label association in forms,	_Layout.cshtml, role views.

		Bootstrap responsive layout. Full audit is [NEEDS CONFIRMATION].	
NF8	Auditability	Audit logger records security and academic events with user identifier, action, detail, and success flag.	Infrastructure/AuditLogger.cs, Models/AuditLog.cs, Controllers/AuditLogsController.cs.

3.5 Assumptions and Constraints

Table 13 lists the assumptions and constraints that have shaped the project.

Table 13: Assumptions and Constraints

Type	Description
Assumption	The marker has access to a Windows machine with Visual Studio 2022, .NET 8 SDK, and a local MySQL 8 server, or accepts that the database connection string can be reconfigured during the viva.
Assumption	The marker accepts the Pomelo MySQL provider as a substitute for SQL Server / LocalDB. The setup steps cover both options where reasonable.
Assumption	The seeded demonstration users defined in Data/DbInitializer.cs are acceptable for the viva.
Assumption	The lecturer accepts the approved individual submission route documented in Section 1.3.
Constraint	The application targets .NET 8 and ASP.NET Core MVC; older runtimes are not supported.
Constraint	The author works within a coursework deadline and reports only what has been implemented; no fictional features are described.
Constraint	Sensitive configuration values such as database passwords, SMTP passwords, and Google client secrets are not included in screenshots or in the body of the report.
Constraint	The audit log must not store free-text personal data beyond what is needed for academic traceability.

4. Research and Background

4.1 ASP.NET Core MVC

ASP.NET Core MVC is a cross-platform web framework provided by Microsoft. It implements the Model View Controller pattern, where models represent the business data, controllers handle the request flow, and views render the user interface. The framework uses dependency injection as a first-class concept, supports configuration through appsettings.json, and integrates cleanly with Entity Framework Core, Identity, and authentication providers. The framework is suitable for the coursework because it allows the author to demonstrate

separation of concerns, request handling, validation, secure access control, and database integration in a single project.

UniManage uses the conventional MVC routing pattern `{controller}/{action}/{id?}` configured in `Program.cs`. Each controller is registered through `AddControllersWithViews`, and views are rendered through Razor. The author has used the same MVC structure across all features, so the marker can move from one controller to another without unfamiliar conventions.

4.2 C# and .NET

C# is a strongly typed, object-oriented programming language designed for the .NET runtime. It supports classes, interfaces, generics, lambdas, asynchronous programming with `async/await`, pattern matching, and nullable reference types. .NET 8 is the long term support release used by UniManage. It provides modern web hosting, JIT and AOT compilation, and high-performance runtime libraries. The author has chosen .NET 8 because it is the current long term support release, because it is fully supported by Visual Studio 2022, and because it gives access to mature ASP.NET Core, EF Core, and Identity packages.

C# language features are used throughout UniManage. For example, asynchronous controller actions return `Task<IActionResult>`, EF Core queries use LINQ, and validation messages use string interpolation. The use of nullable reference types in the project (where present) helps to identify potentially null values during compilation rather than at runtime.

4.3 Razor Views and Tag Helpers

Razor views are server-rendered templates that use the `.cshtml` file extension. The Razor engine combines C# expressions with HTML markup. UniManage uses Razor Tag Helpers to bind form fields to view-model properties, generate anti-forgery tokens, and render validation messages. The shared layout `_Layout.cshtml` provides the page chrome, including the navigation bar, the alert partial, and the footer. Specific role views inherit this layout and contribute their own content sections.

The author has chosen Razor for two reasons. First, Razor is the standard view engine for ASP.NET Core MVC and is therefore the natural choice for an MVC coursework. Second, Razor is fully integrated with model binding and validation, which keeps the user interface code small and consistent.

4.4 Entity Framework Core and MySQL

Entity Framework Core (EF Core) is the official Microsoft object relational mapper for .NET. It allows the developer to write LINQ queries against DbSet properties and to map C# entity classes to database tables. EF Core provides change tracking, migrations, and a fluent API for advanced configuration.

UniManage connects to a MySQL 8 database through the Pomelo Entity Framework Core MySQL provider. The decision to use MySQL rather than SQL Server reflects the local availability of MySQL on the development environment and the popularity of MySQL in industry. The EF Core configuration in Program.cs enables retry on failure with up to five retries and a thirty second maximum delay, which protects the application from transient connection issues. The Pomelo provider is registered through UseMySQL and is targeted at MySQL 8.0.36.

EF Core migrations are stored under Data/Migrations/. The current migrations are:

- 20260408041036_InitialUniManage.cs
- 20260501122117_AddAuditLogAndSecurity.cs
- 20260501125135_AddMeetings.cs

The application calls DbInitializer.SeedAsync at startup, which applies pending migrations and creates demonstration roles, users, and sample data. The marker can therefore start from a clean database and obtain a working environment automatically.

4.5 ASP.NET Core Identity and Role-Based Access Control

ASP.NET Core Identity is the authentication and user management subsystem provided by Microsoft. It manages user storage, password hashing with PBKDF2, sign-in and sign-out, logout, two-factor authentication, external login providers, and role membership. UniManage uses Identity through AddIdentity<ApplicationUser, IdentityRole> configured in Program.cs. The configuration enforces a minimum password length of 8 characters, requires at least one digit, one uppercase letter, and one lowercase letter, allows logout for new users with up to 5 failed attempts, and uses a 15 minute logout window.

Role-based access control is provided by the Identity role system. Three roles are seeded by DbInitializer: Administrator, Lecturer, and Student. Each controller action that requires a role uses [Authorize(Roles = AppRoles.X)], where AppRoles is a static class that holds the role

name constants. The author has used a constants class instead of magic strings so that role names are consistent across the codebase.

Identity also issues an authentication cookie configured through `ConfigureApplicationCookie`. The cookie is HTTP-only, uses `SameSiteMode.Lax`, and is marked secure when the request itself is secure. The session length is eight hours with sliding expiration. These settings reduce the chance of cookie theft and session hijacking while still allowing a normal academic session.

4.6 Bootstrap and Web User Interface Design

Bootstrap is a widely used front-end framework that provides a responsive grid system, components, and utility classes. UniManage uses the Bootstrap classes through the layout file and the role views. The decision to use Bootstrap follows from two reasons. First, Bootstrap is bundled with the default ASP.NET Core MVC project template and is therefore the path of least resistance for a coursework. Second, Bootstrap encourages the author to follow accepted user interface conventions, which improves usability without inventing custom design language.

The layout file `_Layout.cshtml` defines the main navigation bar, exposes the Razor sections for content, and includes the alerts partial. A custom CSS file `wwwroot/css/site.css` adds project-specific styling. A custom JavaScript file `wwwroot/js/site.js` is also present for any client-side behaviour. Bootstrap utility classes are used to maintain consistent spacing, typography, and colour usage across pages.

4.7 Software Engineering Principles Applied

UniManage applies a small set of well-known software engineering principles. The list below explains each principle in the context of the project.

- Separation of concerns. The project separates models (data and validation rules), view models (presentation data), controllers (request flow and business rules), views (HTML rendering), data access (`ApplicationDbContext`, migrations, seeding), and cross-cutting infrastructure (audit logger, email service, file store, security headers, CSV writer, PDF report). Each folder has a clearly defined responsibility.
- Single responsibility. Each controller is focused on one academic area, and each infrastructure service is focused on one concern. For example, `AuditLogger` is only

responsible for writing audit records, CsvWriter is only responsible for writing CSV output, and PdfReport is only responsible for building PDF reports through QuestPDF.

- Layered architecture. Requests flow through middleware (security headers, HTTPS redirection, rate limiting), then through Identity, then through the controller, then through EF Core, and finally back through the view to the browser.
- Defence in depth. Security is implemented at the network layer (HTTPS, HSTS), the request layer (rate limiting, anti-forgery), the application layer (Identity, role attributes, ownership checks, validation), the data layer (EF Core parameterised queries, unique constraints), and the audit layer (audit log).
- Convention over configuration. Where ASP.NET Core MVC provides sensible defaults (routing, model binding, view discovery), the author has used those defaults instead of overriding them. This keeps the codebase smaller and easier to maintain.
- Asynchronous I/O. Database, file, and email operations are asynchronous, which keeps the application responsive under load.

4.8 Security and Data Protection Background

Web applications face well-known threats. Cross-site request forgery, cross-site scripting, SQL injection, broken authentication, broken access control, sensitive data exposure, and security misconfiguration are all common in production systems. The Open Web Application Security Project (OWASP) provides a Top 10 list that summarises these threats, and ASP.NET Core ships with a set of features that address them.

UniManage relies on Identity for authentication, on the [Authorize] attribute for authorisation, on parameterised EF Core queries for SQL injection protection, on the Razor engine's automatic HTML encoding for XSS protection, on anti-forgery tokens for CSRF protection, on HTTPS and HSTS for transport security, on a custom security headers middleware for defence in depth, and on the audit logger for traceability. Sensitive configuration values are kept in appsettings.json and are excluded from the report. Chapter 13 explains each control in more detail.

5. Requirement Analysis

5.1 Requirement Derivation

Requirements have been derived from three sources. First, the coursework brief defines the marking items that the system must address. Second, a structured review of the source-code

repository has identified the features that are actually implemented. Third, the case-study analysis in Chapter 2 has identified the practical needs of Students, Lecturers, and Administrators.

This combination keeps the requirements grounded in evidence. Every functional requirement is supported by at least one source-code file, and every non-functional requirement is supported by at least one infrastructure or configuration setting. Where a requirement appears in the brief but cannot be supported by source code, the requirement is recorded with a [NEEDS CONFIRMATION] note rather than being silently dropped.

5.2 Functional Requirements

Table 1 lists the functional requirements that the system supports. Each requirement is referenced from later chapters and from the testing chapter.

Table 1: Functional Requirements

ID	Functional Requirement	Source-Code Evidence
FR1	The system shall allow a user to register an account with role selection (Student or Lecturer).	Controllers/AccountController.cs (Register), ViewModels/RegisterViewModel.cs, Views/Account/Register.cshtml
FR2	The system shall allow a registered user to log in using a strong password.	Controllers/AccountController.cs (Login), ViewModels/LoginViewModel.cs, Views/Account/Login.cshtml, Program.cs Identity setup
FR3	The system shall lock the account after 5 failed login attempts for 15 minutes.	Program.cs lockout configuration
FR4	The system shall allow a user to log out and invalidate the authentication cookie.	Controllers/AccountController.cs (Logout), Program.cs cookie configuration
FR5	The system shall allow a user to request a password reset and to reset the password using a token.	Controllers/AccountController.cs (ForgotPassword, ResetPassword), ViewModels/ForgotPasswordViewModel.cs, ViewModels/ResetPasswordViewModel.cs
FR6	The system shall allow optional Google login if Google client id and secret are configured.	Program.cs Google authentication block
FR7	The system shall route the signed-in user to the dashboard that matches the assigned role.	Controllers/DashboardController.cs (Index, Student, Lecturer, Administrator)
FR8	The system shall display a Student dashboard summarising enrolled courses, deadlines, grades, messages, calendar, and meetings.	Controllers/DashboardController.cs (Student), ViewModels/DashboardViewModels.cs, Views/Dashboard/Student.cshtml
FR9	The system shall display a Lecturer dashboard	Controllers/DashboardController.cs (Lecturer), Views/Dashboard/Lecturer.cshtml

	summarising teaching courses, pending submissions, recent activity, messages, and meetings.	
FR10	The system shall display an Administrator dashboard summarising system counts, popular courses, and audit activity.	Controllers/DashboardController.cs (Administrator), Views/Dashboard/Administrator.cshtml
FR11	The system shall allow an Administrator to create, read, update, delete, search, and page through courses.	Controllers/CoursesController.cs, Models/Course.cs, ViewModels/CourseInputViewModel.cs, Views/Courses/
FR12	The system shall allow a Lecturer to view and access only the courses they own.	Controllers/CoursesController.cs (MyCourses), ownership filter checks
FR13	The system shall allow a Lecturer to upload course material files for an owned course.	Controllers/CoursesController.cs (UploadMaterial), Models/CourseMaterial.cs, ViewModels/MaterialUploadViewModel.cs, Infrastructure/UploadedFileStore.cs
FR14	The system shall allow a Student to browse courses, see eligibility, and enrol on an open course.	Controllers/EnrollmentsController.cs (Browse, Enroll), ViewModels/CourseBrowseRowViewModel.cs, Views/Enrollments/Browse.cshtml
FR15	The system shall prevent duplicate enrolment, exceeding capacity, and missing prerequisites.	Controllers/EnrollmentsController.cs business rule checks
FR16	The system shall allow a Lecturer to create, edit, and delete assignments for an owned course.	Controllers/AssignmentsController.cs (Create, Edit, Delete, ForCourse, Mine), Models/Assignment.cs, ViewModels/AssignmentInputViewModel.cs
FR17	The system shall allow a Student to submit text and file content for an assignment they are enrolled on.	Controllers/SubmissionsController.cs (Submit), Models/Submission.cs, ViewModels/SubmissionSubmitViewModel.cs, Views/Submissions/Submit.cshtml
FR18	The system shall allow a Lecturer to grade submissions and record feedback.	Controllers/SubmissionsController.cs (Grade), ViewModels/GradeSubmissionViewModel.cs, Views/Submissions/Grade.cshtml
FR19	The system shall allow secure download of submission and material files only by the rightful user.	Controllers/SubmissionsController.cs (CourseMaterialFile, SubmissionFile), Infrastructure/UploadedFileStore.cs
FR20	The system shall produce reports for course popularity, student performance, lecturer workload, enrolments, pass and fail, and assignment attendance.	Controllers/ReportsController.cs, Views/Reports/
FR21	The system shall export selected reports to CSV and PDF.	Infrastructure/CsvWriter.cs, Infrastructure/PdfReport.cs, Controllers/ReportsController.cs
FR22	The system shall allow internal messaging through inbox, thread, reply, and compose actions.	Controllers/MessagesController.cs, Models/Message.cs, Views/Messages/, message view models
FR23	The system shall enforce	Controllers/MessagesController.cs Compose

	allowed-recipient rules between Students and Lecturers based on enrolments.	validation
FR24	The system shall allow Lecturers to schedule meetings for owned courses, including a join URL.	Controllers/MeetingsController.cs, Models/Meeting.cs, ViewModels/MeetingInputViewModel.cs, Views/Meetings/
FR25	The system shall allow ICS export of scheduled meetings.	Controllers/MeetingsController.cs (Ics), Infrastructure/CalendarLink.cs
FR26	The system shall allow a user to view and update their profile and to change their password.	Controllers/ProfileController.cs, ViewModels/ProfileViewModel.cs, Views/Profile/
FR27	The system shall record audit log entries for security-sensitive and academic actions.	Models/AuditLog.cs, Infrastructure/AuditLogger.cs, Controllers/AuditLogsController.cs, Views/AuditLogs/Index.cshtml
FR28	The system shall display a friendly error page if an unhandled exception occurs in production.	Program.cs UseExceptionHandler, Views/Shared/Error.cshtml
FR29	The system shall apply rate limiting to authentication and upload endpoints.	Program.cs AddRateLimiter policies (auth, upload)
FR30	The system shall enforce anti-forgery tokens on POST forms.	Program.cs AddAntiforgery, [ValidateAntiForgeryToken] attributes

5.3 Non-Functional Requirements

Table 2 lists the non-functional requirements with rationale and source-code evidence.

Table 2: Non-Functional Requirements

ID	Non-Functional Requirement	Rationale / Evidence
NFR1	Usability: Users should reach key tasks in a small number of clicks.	Role-aware navigation in _Layout.cshtml, role dashboards, alerts partial, consistent forms.
NFR2	Performance: Dashboard queries should return within a reasonable time on a typical academic dataset.	EF Core filtering and paging, AsNoTracking for read-only queries, retry on failure.
NFR3	Security: The application should resist common web threats such as CSRF, brute force login, and unauthorised access.	Identity, logout, anti-forgery, rate limiting, security headers, HTTPS, HSTS.
NFR4	Maintainability: New features should fit into the existing structure without rewriting core code.	Layered MVC structure, dependency injection, single-responsibility services.
NFR5	Reliability: Transient database errors should not cause user-visible failures.	EF Core retry on failure, exception handling middleware.
NFR6	Data integrity: Domain rules such as unique course code and unique enrolment per student/course	EF Core unique indexes, fluent configuration in ApplicationDbContext.

	should be enforced at the database level.	
NFR7	Accessibility: Forms should be usable with keyboard, semantic labels, and skip links.	_Layout.cshtml, label association in form views. Full audit is [NEEDS CONFIRMATION].
NFR8	Auditability: Important actions should be recorded with user, action, and detail.	AuditLogger and AuditLog entity.
NFR9	Configurability: Database connection, email settings, and Google credentials should be configurable through appsettings.json.	Program.cs configuration binding.
NFR10	Deployability: The project should build with a single dotnet build --no-restore command.	Build verified locally with zero warnings and zero errors.

5.4 Use Case Summary

Table 14 summarises the main use cases. The diagram in Section 6.2 shows the visual layout.

Table 14: Use Case Summary

Use Case	Primary Actor	Goal
Register Account	Visitor	Create a Student or Lecturer account.
Log In	Registered User	Access the role-appropriate dashboard.
Reset Password	Registered User	Recover access using an emailed token.
View Dashboard	Student / Lecturer / Administrator	See role-relevant academic information.
Browse Courses	Student	Find a course to enrol on.
Enrol on Course	Student	Add a course to the personal record subject to capacity, prerequisite, and duplicate checks.
Submit Assignment	Student	Upload submission text and file.
View Grades and Feedback	Student	Read marker feedback.
Send Message	Student / Lecturer	Communicate within an allowed relationship.
Manage Course	Administrator	Maintain the course catalogue.
Manage Assignment	Lecturer	Set or update assignments for an owned course.
Grade Submission	Lecturer	Record marks and feedback.
Upload Course Material	Lecturer	Provide teaching resources.
Schedule Meeting	Lecturer	Organise a class meeting for an owned course.
View Reports	Administrator	Review system-wide academic data.
Review Audit Log	Administrator	Inspect security-sensitive and academic events.
Update Profile	Any User	Maintain personal information.
Change Password	Any User	Update the password under authentication.

5.5 Requirement Traceability Summary

Table 15 maps the marking items in the brief to the functional requirements that support them and to the implementation evidence.

Table 15: Requirement Traceability Summary

Marking Item	Supporting Requirements	Source-Code Evidence	Test Cases
Application user interface	NFR1, NFR7	_Layout.cshtml, _Alerts.cshtml, role views, site.css	T1, T8, T9, T10, T11
User Registration and Login	FR1, FR2, FR3, FR4, FR5, FR6, FR30	AccountController, Program.cs	T1, T2, T3, T4, T5
Student Dashboard	FR7, FR8	DashboardController.Student, dashboard view models	T8
Lecturer Dashboard	FR7, FR9	DashboardController.Lecturer	T9
Administrator Dashboard	FR7, FR10	DashboardController.Administrator	T10
Course Management	FR11, FR12, FR13	CoursesController, Course, course views	T12, T13, T14, T15
Enrollment System	FR14, FR15	EnrollmentsController, Enrollment	T16, T17
Assignment and Grading	FR16, FR17, FR18, FR19	AssignmentsController, SubmissionsController	T18, T19, T20, T21
Reporting and Analytics	FR20, FR21	ReportsController, CsvWriter, PdfReport	T22, T23
Communication Module	FR22, FR23, FR24, FR25	MessagesController, MeetingsController	T24, T25
Security and Data Protection	NFR3, FR27, FR29, FR30	Program.cs, SecurityHeadersMiddleware, AuditLogger	T26, T27, T28
Validation and Exception Handling	FR28, NFR6	Data annotations, ModelState, Error.cshtml	T29, T30
Installation Guide and User Manual	NFR4, NFR9	User_Manual_Application_Development.md	Manual review
Concise Logical Solution	NFR4, NFR5	Chapter 11 of this report	Manual review
Architecture Diagrams	NFR4	documentation_application_development/diagrams/	Manual review
Detailed Description of Classes	NFR4	Tables 4, 5, 6 in Chapter 10	Manual review
Own Reflection	n/a	Chapter 19	Manual review
Programming Style	NFR4	Folder structure, naming, DI	Manual review
Interface Design and Usability	NFR1, NFR7	Layout, partials, CSS	T11, T31

6. System Architecture and Design

6.1 High-Level Architecture

The high-level architecture describes how the browser, the ASP.NET Core MVC application, the infrastructure services, and the database interact. UniManage follows a standard layered architecture for ASP.NET Core MVC applications. A request enters through Kestrel and passes through HTTPS redirection, static file serving, security headers, routing, rate limiting, authentication, and authorisation. After this middleware pipeline, the request reaches the controller, which uses EF Core through ApplicationDbContext to query or update the MySQL database, and which composes a view model that is rendered through Razor.

Figure 1: [INSERT EXPORTED DIAGRAM IMAGE HERE]

This diagram shows the high-level system architecture of UniManage. It traces a request from the browser through Kestrel, the HTTPS redirection middleware, the static file middleware, the security headers middleware, the routing middleware, the rate limiter, authentication, and authorisation, into the controllers under Controllers/, then through Entity Framework Core in Data/ApplicationDbContext.cs into the MySQL database accessed through the Pomelo provider, and back through Razor views to the browser. It also shows the supporting infrastructure services in Infrastructure/ and the static assets served from wwwroot/. The exportable source is held at `documentation_application_development/diagrams/architecture_diagram.mmd`.

The architecture consists of the following components:

- Browser. The user agent that renders Razor-generated HTML, executes the small amount of JavaScript in `wwwroot/js/site.js`, and submits forms using anti-forgery tokens.
- Kestrel and middleware. The ASP.NET Core hosting layer accepts the request and passes it through the middleware pipeline configured in `Program.cs`.
- Identity. Authenticates the user using a cookie. Ten failed sign-in attempts within a minute are throttled by the rate limiter under the auth policy.
- Controllers. The twelve controllers under `Controllers/` handle request flow, validation, and business rules.
- EF Core. Translates LINQ queries into MySQL SQL through `ApplicationDbContext`.
- Infrastructure services. A small set of services in `Infrastructure/` provide cross-cutting capability such as audit logging, email, file storage, CSV writing, PDF rendering, calendar links, and security headers.
- MySQL database. Stores all application data through Pomelo EF Core.
- Static assets. Bootstrap, custom CSS, JavaScript, and images served from `wwwroot/`.

The diagram description explains how each component depends on the next. The author has used this layered architecture to keep request handling, business rules, persistence, and

presentation in separate folders, which makes the project easier to maintain and easier to mark.

6.2 Use Case Diagram

Figure 2: [INSERT EXPORTED DIAGRAM IMAGE HERE]

This diagram shows the use case model for UniManage. It identifies the three actors (Student, Lecturer, Administrator) and the use cases listed in Table 14, including Register Account, Log In, Reset Password, View Dashboard, Browse Courses, Enrol on Course, Submit Assignment, View Grades and Feedback, Send Message, Manage Course, Manage Assignment, Grade Submission, Upload Course Material, Schedule Meeting, View Reports, Review Audit Log, Update Profile, and Change Password. The exportable source is held at `documentation_application_development/diagrams/use_case_diagram.mmd`.

The use case diagram identifies three actors and the use cases listed in Table 14. The Student actor is associated with the Register, Log In, View Student Dashboard, Browse Courses, Enrol, Submit Assignment, View Grades, Send Message, View Meetings, Update Profile, and Change Password use cases. The Lecturer actor is associated with the Log In, View Lecturer Dashboard, Manage Assignment, Grade Submission, Upload Course Material, Schedule Meeting, Send Message, Update Profile, and Change Password use cases. The Administrator actor is associated with the Log In, View Administrator Dashboard, Manage Course, View Report, Review Audit Log, Update Profile, and Change Password use cases. Common use cases such as Register, Log In, Update Profile, and Change Password are extended by all relevant actors.

6.3 Entity Relationship Diagram

Figure 3: [INSERT EXPORTED DIAGRAM IMAGE HERE]

This diagram shows the logical data model used by UniManage. It places ApplicationUser at the centre and connects it through one-to-many relationships to Course (lecturer side), Enrollment (student side), Submission (student and grader sides), CourseMaterial (uploader side), Message (sender and receiver sides), Meeting (lecturer side), and AuditLog (optional user side). The diagram also shows the optional self-reference from Course to Course for prerequisites, the unique-pair index on Enrollment(StudentId, CourseId), and the relationship from Assignment to Course and from Submission to Assignment. The exportable source is held at `documentation_application_development/diagrams/er_diagram.mmd`.

The ER diagram shows the relationships between the application's main entities. ApplicationUser is the central entity. A Course is taught by exactly one ApplicationUser (the lecturer) and may have a Course as a prerequisite. An Enrollment links one ApplicationUser (the student) to one Course and is unique on the combination of student and course. An Assignment belongs to one Course. A Submission belongs to one Assignment, one ApplicationUser (the student), and optionally one ApplicationUser (the grader). A CourseMaterial belongs to one Course and is uploaded by one ApplicationUser. A Message is sent from one ApplicationUser to another. A Meeting belongs to one Course and is owned by one ApplicationUser (the lecturer). An AuditLog records the user identifier where available.

The relationships are configured in `Data/ApplicationDbContext.cs` through the EF Core fluent API, including foreign keys, delete behaviour, and unique constraints such as the unique course code and the unique student-course enrolment.

6.4 UML Class Diagram

Figure 4: [INSERT EXPORTED DIAGRAM IMAGE HERE]

This diagram shows the main classes used by UniManage and their dependencies. It includes the twelve controllers (AccountController, DashboardController, CoursesController, EnrollmentsController, AssignmentsController, SubmissionsController, ReportsController, MessagesController, MeetingsController, ProfileController, AuditLogsController, HomeController), the domain models, the view models, the ApplicationDbContext, and the infrastructure services such as IAuditLogger, IEmailService, UploadedFileStore, CsvWriter, PdfReport, and SecurityHeadersMiddleware. The exportable source is held at `documentation_application_development/diagrams/class_diagram.mmd`.

The class diagram summarises the controllers, models, view models, services, and the ApplicationDbContext. The diagram shows that controllers depend on ApplicationDbContext and on infrastructure interfaces such as IAuditLogger and IEmailService. Models are plain C# classes with data annotation attributes. View models are also plain C# classes that are used only between controllers and views, and are not persisted by EF Core. The diagram does not include role-specific controller classes because the project uses a single DashboardController with separate actions for each role.

6.5 Application Flowchart

Figure 5: [INSERT EXPORTED DIAGRAM IMAGE HERE]

This diagram shows the application flow from the home page through registration or login, role-aware redirection to the appropriate dashboard, the controller validation step, the EF Core data access step, the response back to the user, and the logout path. It also shows the friendly error page that is reached when an unhandled exception is raised in production. The exportable source is held at `documentation_application_development/diagrams/application_flowchart.mmd`.

The flowchart starts at the home page, where an unauthenticated user is offered Register or Log In. After successful login, the user is redirected to the role-appropriate dashboard. From the dashboard, the user navigates to a feature, performs an action, and the controller validates the input, calls EF Core, and responds with either a success view or a validation message. The flowchart also covers the logout action and the friendly error page.

6.6 Authentication and Role-Based Access Flow

Figure 6: [INSERT EXPORTED DIAGRAM IMAGE HERE]

This diagram shows the authentication and role-based access flow. It traces the path from the login form to AccountController.Login, through the Identity sign-in API, the lockout policy and the auth rate-limiting policy, to the issued authentication cookie, and on to Dashboard/Index, which inspects the role claim and forwards the request to the Student, Lecturer, or Administrator action. It also shows the access denied page that is returned when an unauthorised role attempts to open a restricted action. The exportable source is held at `documentation_application_development/diagrams/security_flow_diagram.mmd`.

The authentication flow shows the path from the login form to the authentication cookie. The user posts the form to `AccountController.Login`, which validates the model state, calls `Identity` to verify the credentials, and either signs the user in or returns a validation error. After sign-in, the controller redirects the user to `Dashboard/Index`, which inspects the role claim and forwards the request to `Student`, `Lecturer`, or `Administrator`. If the user opens an action that requires a different role, the `[Authorize]` attribute returns a 403 response and the access denied page is shown.

6.7 Design Decisions and Rationale

The author has taken several design decisions during the project. The most important decisions are summarised below.

- MVC over Razor Pages. The brief refers to ASP.NET MVC. ASP.NET Core MVC is the modern equivalent and is therefore the natural choice. The MVC structure also matches the marking criteria for separation of concerns.
- EF Core with Pomelo MySQL over SQL Server / LocalDB. The local environment includes MySQL 8 and the Pomelo provider is mature. The EF Core layer abstracts the underlying engine, so the design remains portable.
- Single `DashboardController` over three role-specific controllers. A single controller with three role-restricted actions reduces duplication, makes routing simpler, and keeps role-specific logic in one place. The pattern matches the way the brief talks about "dashboards" rather than "dashboard controllers".
- Submission stores grade fields directly. The brief mentions assignments and grading. A separate `Grade` entity would add tables and complexity without changing the workflow. The `Submission` entity therefore stores the grade fields directly, which keeps the data model small and keeps the grading view simple.
- Custom security headers middleware. Although ASP.NET Core ships with several security features, security headers such as Content Security Policy, X-Content-Type-Options, and Referrer Policy are added through a small middleware in `Infrastructure/SecurityHeadersMiddleware.cs`. This makes the controls visible and reviewable.
- QuestPDF over a separate reporting tool. QuestPDF is open source under the Community licence, integrates cleanly with C#, and produces professional PDF output. The licence type is set to Community in `Program.cs`.

7. Database Design

7.1 Database Technology

UniManage uses Entity Framework Core 8 with the Pomelo MySQL provider against a MySQL 8 server. The connection string is read from the DefaultConnection entry of appsettings.json (or appsettings.Development.json during development). The configuration in Program.cs registers the ApplicationDbContext and enables retry on failure.

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseMySQL(connectionString, new MySqlServerVersion(new Version(8, 0,
36)), mysql =>
    mysql.EnableRetryOnFailure(maxRetryCount: 5, maxRetryDelay:
    TimeSpan.FromSeconds(30), errorNumbersToAdd: null)));
```

EF Core retry on failure is important because MySQL connections can fail transiently when the server is starting or when a network blip occurs. The retry policy hides these transient errors from the user and reduces the chance of false-negative test results.

The brief refers to SQL Server / LocalDB as a possible technology. UniManage uses MySQL through Pomelo, which is recorded honestly in the documentation. If the marker requires a SQL Server / LocalDB build, the approach would be to replace UseMySQL with UseSqlServer and update the connection string. The migrations would need to be regenerated to reflect SQL Server data types. This option remains [NEEDS CONFIRMATION] because no SQL Server scripts are present in the source.

7.2 ApplicationDbContext

ApplicationDbContext inherits from IdentityDbContext<ApplicationUser>. This means the context exposes the standard Identity tables (Users, Roles, UserRoles, UserClaims, UserLogins, UserTokens, RoleClaims) and, in addition, the academic tables described in the next section. The constructor accepts DbContextOptions<ApplicationDbContext> so that the options can be supplied through dependency injection.

The fluent API in OnModelCreating is used to configure relationships, delete behaviour, and unique indexes. The most important configurations include the unique course code, the unique student-course enrolment, the lecturer-course foreign key with restrict-on-delete behaviour, the prerequisite course self-reference, the assignment-course relationship, the submission-assignment-student relationship, the message sender and receiver self-references, the meeting-course-lecturer relationships, and the audit log user reference.

7.3 Main Entities

Table 3 lists the main academic entities. The Identity entities (AspNetUsers, AspNetRoles, etc.) are not duplicated in the table because they are managed by the framework.

Table 3: Core Database Entities

Entity	Primary Key	Main Foreign Keys	Purpose
ApplicationUser	Id (string)	Identity relationships	Stores user profile data and Identity user fields.
Course	CourseId	LecturerId (ApplicationUser), PrerequisiteId (Course, optional)	Stores course code, name, description, credits, enrolment limit, lecturer assignment, optional prerequisite.
Enrollment	EnrollmentId	StudentId (ApplicationUser), CourseId (Course)	Links a student to a course; unique on the pair.
Assignment	AssignmentId	CourseId (Course)	Records coursework tasks set by a lecturer for a course.
Submission	SubmissionId	AssignmentId, StudentId, GradedById (optional)	Records student submissions, grade, feedback, status, file metadata.
CourseMaterial	CourseMaterialId	CourseId, UploadedById	Records uploaded teaching material metadata.
Message	MessageId	SenderId, ReceiverId	Records direct messages between users.
Meeting	MeetingId	CourseId, LecturerId	Records scheduled course meetings, including a meeting URL.
AuditLog	AuditLogId	UserId (optional)	Records security-sensitive and academic events.

7.4 Entity Relationships

The relationships between the entities are summarised below.

- ApplicationUser to Course (lecturer side). One-to-many. A lecturer may teach many courses; a course has one lecturer.
- Course to Course (prerequisite). Optional self-reference. A course may require another course as a prerequisite, or none at all.
- ApplicationUser to Enrollment (student side). One-to-many. A student may have many enrolments.
- Course to Enrollment. One-to-many. A course may have many enrolments.

- Enrollment uniqueness. Unique index on the pair (StudentId, CourseId) prevents duplicate enrolments.
- Course to Assignment. One-to-many. A course may have many assignments.
- Assignment to Submission. One-to-many. An assignment may have many submissions.
- ApplicationUser to Submission (student side). One-to-many.
- ApplicationUser to Submission (grader side). Optional one-to-many.
- Course to CourseMaterial. One-to-many.
- ApplicationUser to CourseMaterial (uploader side). One-to-many.
- ApplicationUser to Message (sender and receiver sides). Two one-to-many self-references.
- Course to Meeting. One-to-many.
- ApplicationUser to Meeting (lecturer side). One-to-many.
- ApplicationUser to AuditLog (optional). One-to-many; the audit log may record events even when the user is anonymous, in which case the UserId is null.

7.5 Keys, Constraints and Validation Rules

The database enforces the following constraints, in addition to the field-level data annotation rules:

- Unique course code on Course.Code.
- Unique enrolment per student and course on Enrollment(StudentId, CourseId).
- Maximum lengths on string fields, set through data annotations such as [StringLength].
- Required fields enforced through [Required] annotations.
- Range constraints on numeric fields such as MaxPoints on Assignment and the grade on Submission.
- Optional foreign keys for PrerequisiteId and GradedById.
- Default values set in the model constructor or through migrations, for example `EnrolledAtUtc = DateTime.UtcNow`.

The combination of database constraints and application-level validation provides defence in depth. A bad request that bypasses the controller would still be caught by the database; a bad input that satisfies the database would still be caught by ModelState before reaching the database.

7.6 Migrations and Seed Data

EF Core migrations describe the schema changes required to bring the database up to date. UniManage has three migrations:

- 20260408041036_InitialUniManage.cs: creates the Identity tables and the initial academic tables (Courses, Enrollments, Assignments, Submissions, CourseMaterials, Messages).
- 20260501122117_AddAuditLogAndSecurity.cs: adds the AuditLogs table and supporting columns for security tracking.
- 20260501125135_AddMeetings.cs: adds the Meetings table and the meeting URL column.

DbInitializer.SeedAsync is called from Program.cs at startup. It applies pending migrations, creates the three roles (Administrator, Lecturer, Student), creates the demonstration users admin@unimanage.local, lecturer@unimanage.local, and student@unimanage.local, and may create sample courses and enrolments. The seeded credentials are listed in the user manual and are intended for the viva and for marking, not for production use.

7.7 Database Evidence Required

Figure 17: [INSERT SCREENSHOT HERE]

This screenshot shows a MySQL client connected to the seeded UniManage database, with the Courses, Enrollments, or Submissions table populated after a few actions in the application. Personal data is redacted before insertion in the report.

Figure 17b: [INSERT SCREENSHOT HERE]

This screenshot shows the console output of dotnet ef migrations list from the project root, listing the three EF Core migrations.

The following items remain [NEEDS CONFIRMATION]: a raw SQL backup script, a SQL Server / LocalDB equivalent script, and a database backup file (.bak or .sql dump).

8. Implementation

This chapter explains the implementation in detail, feature by feature. Each section follows the same structure: the source files involved, the request flow, the validation and business rules, the role of the view model, and the screenshot or evidence that should be captured.

8.1 Project Structure

The project follows the standard ASP.NET Core MVC layout with the following top-level folders:

- Controllers/ (twelve files): AccountController.cs, AssignmentsController.cs, AuditLogsController.cs, CoursesController.cs, DashboardController.cs, EnrollmentsController.cs, HomeController.cs, MeetingsController.cs, MessagesController.cs, ProfileController.cs, ReportsController.cs, SubmissionsController.cs.
- Models/ (twelve files): AppRoles.cs, ApplicationUser.cs, Assignment.cs, AuditLog.cs, Course.cs, CourseMaterial.cs, Enrollment.cs, ErrorViewModel.cs, Meeting.cs, Message.cs, Submission.cs, SubmissionStatus.cs.
- ViewModels/ (sixteen files): AssignmentInputViewModel.cs, CourseBrowseRowViewModel.cs, CourseInputViewModel.cs, DashboardViewModels.cs, ForgotPasswordViewModel.cs, GradeSubmissionViewModel.cs, LoginViewModel.cs, MaterialUploadViewModel.cs, MeetingInputViewModel.cs, MessageComposeViewModel.cs, MessageInboxRowViewModel.cs, MessageThreadViewModel.cs, ProfileViewModel.cs, RegisterViewModel.cs, ResetPasswordViewModel.cs, SubmissionSubmitViewModel.cs.
- Data/: ApplicationDbContext.cs, ApplicationDbContextFactory.cs, DbInitializer.cs, Data/Migrations/ (three migrations and the model snapshot).
- Infrastructure/ (twelve files): AuditLogger.cs, CalendarLink.cs, CsvWriter.cs, EmailSettings.cs, EmailTemplates.cs, IAuditLogger.cs, IEmailService.cs, MeetLinkGenerator.cs, PdfReport.cs, SecurityHeadersMiddleware.cs, SmtplibEmailService.cs, UploadedFileStore.cs.
- Views/ (forty-six Razor views) including Account/, AuditLogs/, Assignments/, Courses/, Dashboard/, Enrollments/, Home/, Meetings/, Messages/, Profile/, Reports/, Submissions/, Shared/, plus _ViewStart.cshtml and _ViewImports.cshtml.
- wwwroot/: static assets including css/site.css, js/site.js, lib/, and assets/.
- Configuration: appsettings.json, appsettings.Development.json, Properties/launchSettings.json.

This layout matches the conventional ASP.NET Core MVC structure, which makes the codebase easy to navigate for both the author and the marker.

8.2 Program.cs and Startup Configuration

Program.cs is the composition root. It is responsible for service registration and middleware configuration. The file does the following work in order:

1. Sets the QuestPDF licence to Community to comply with the QuestPDF licensing terms.
2. Reads the DefaultConnection connection string from configuration. If the connection string is missing, the application throws an InvalidOperationException at startup so that the misconfiguration is visible immediately.
3. Registers ApplicationDbContext with the Pomelo MySQL provider and enables retry on failure.
4. Registers ASP.NET Core Identity with ApplicationUser and IdentityRole. The Identity options enforce strong password rules, lockout, and unique email.
5. Registers the cookie authentication scheme as the default and conditionally registers Google authentication if a client id and client secret are configured.
6. Configures the application cookie with login, logout, and access denied paths, sliding expiration of eight hours, HTTP-only flag, SameSiteMode.Lax, and CookieSecurePolicy.SameAsRequest.
7. Binds the email settings, registers SmtpEmailService for IEmailService, registers IHttpContextAccessor, and registers AuditLogger for IAuditLogger.
8. Configures the rate limiter with auth and upload policies.
9. Configures anti-forgery cookies as HTTP-only with SameSiteMode.Strict.
10. Registers MVC controllers with views.
11. Builds the application.
12. Runs DbInitializer.SeedAsync inside a service scope.
13. Applies the production exception handler and HSTS in production, or the developer exception page in development.
14. Applies HTTPS redirection, static files, the security headers middleware, routing, the rate limiter, authentication, and authorisation, then maps the default route and runs the application.

Listing 8.1 shows the security-related part of Program.cs.

```
builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    options.AddPolicy("auth", httpContext =>
        RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: httpContext.Connection.RemoteIpAddress?.ToString() ??
"anon",
            factory: _ => new FixedWindowRateLimiterOptions
            {
                PermitLimit = 10,
                Window = TimeSpan.FromMinutes(1),
                QueueLimit = 0,
            }
        )
    )
})
```

```

        AutoReplenishment = true
    }));
    options.AddPolicy("upload", ...);
});
builder.Services.AddAntiforgery(options =>
{
    options.Cookie.HttpOnly = true;
    options.Cookie.SameSite = SameSiteMode.Strict;
    options.Cookie.SecurePolicy = CookieSecurePolicy.SameAsRequest;
});

```

8.3 User Registration and Login

Registration and login are implemented in `Controllers/AccountController.cs`. The `Register` GET action returns the registration form. The `Register` POST action validates `RegisterViewModel`, creates a new `ApplicationUser`, calls `UserManager.CreateAsync`, assigns the chosen role (Student or Lecturer), and signs the user in. Audit log entries are written for both successful and failed registrations.

The `Login` GET action returns the login form. The `Login` POST action validates `LoginViewModel`, calls `SignInManager.PasswordSignInAsync` with `lockoutOnFailure: true`, and either signs the user in or returns a validation error. Failed sign-in attempts are recorded in the audit log. After successful login, the user is redirected to `Dashboard/Index`, which routes the user to the role dashboard.

The `Logout` action calls `SignInManager.SignOutAsync` and redirects to the home page. The `ForgotPassword` and `ResetPassword` actions provide the standard Identity password recovery flow with email tokens.

Figure 7: [INSERT SCREENSHOT HERE]

This screenshot shows the login page after running the application.

Figure 7b: [INSERT SCREENSHOT HERE]

This screenshot shows the registration page with the role selector visible.

Figure 7c: [INSERT SCREENSHOT HERE]

This screenshot shows the login form after submitting invalid credentials, with the validation message displayed under the input fields.

Figure 7d: [INSERT SCREENSHOT HERE]

This screenshot shows the browser URL bar after a successful Student login, with the address pointing at the role-appropriate dashboard route (for example `/Dashboard/Student`).

8.4 Role-Based Access Control

Role-based access control is implemented through three layers. First, `AppRoles` defines the role name constants. Second, `DbInitializer` seeds the three roles and the three demonstration users. Third, each controller action uses `[Authorize(Roles = AppRoles.X)]` to restrict access.

Where a Lecturer must only see their own courses or assignments, an additional ownership filter is applied inside the action.

The author has used a constants class because magic strings are error prone. If the constant is renamed, the rename is visible everywhere through Visual Studio's refactoring tools. If a magic string had been used, a misspelt role name would silently allow or deny access without any compile-time warning.

8.5 Student Dashboard

The Student dashboard is implemented by `DashboardController.Student` and `Views/Dashboard/Student.cshtml`. The action queries the `Enrollments`, `Assignments`, `Submissions`, `Messages`, and `Meetings` tables to build a `StudentDashboardViewModel` (defined inside `ViewModels/DashboardViewModels.cs`). The view model holds enrolled courses, upcoming deadlines, recent grades, recent messages, calendar events, and meetings. EF Core read queries use `AsNoTracking` where possible to reduce the change tracker overhead.

Figure 8: [INSERT SCREENSHOT HERE]

This screenshot shows the student dashboard.

Figure 8b: [INSERT SCREENSHOT HERE]

This screenshot shows the student calendar and deadlines panel.

8.6 Lecturer Dashboard

The Lecturer dashboard is implemented by `DashboardController.Lecturer` and `Views/Dashboard/Lecturer.cshtml`. The action filters courses, assignments, and submissions by the signed-in lecturer's user identifier. The view model holds teaching courses, enrolment counts, assignment counts, pending submissions, recently graded submissions, recent messages, recent meetings, and recent activity.

Figure 9: [INSERT SCREENSHOT HERE]

This screenshot shows the lecturer dashboard.

Figure 9b: [INSERT SCREENSHOT HERE]

This screenshot shows the lecturer pending submissions view.

8.7 Administrator Dashboard

The Administrator dashboard is implemented by `DashboardController.Administrator` and `Views/Dashboard/Administrator.cshtml`. The action queries the database to count users, courses, enrolments, assignments, submissions, and messages, identify popular courses by enrolment count, and load recent audit log entries. Where a dedicated user-management

screen would be expected, the source-code review has not confirmed one. This is recorded as [NEEDS CONFIRMATION].

Figure 10: [INSERT SCREENSHOT HERE]

This screenshot shows the administrator dashboard.

Figure 10b: [INSERT SCREENSHOT HERE]

This screenshot shows the administrator audit log review.

8.8 Course Management

Course management is implemented in `Controllers/CoursesController.cs` and the `Views/Courses/` folder. The actions are:

- Index: lists courses with search and paging support.
- Details: shows a single course.
- Create (GET and POST): allows an Administrator to create a new course using `CourseInputViewModel` for validation. The unique course code is enforced through a database unique index.
- Edit (GET and POST): allows an Administrator to update an existing course.
- Delete (GET and POST): allows an Administrator to delete a course, with appropriate cascade or restrict behaviour.
- MyCourses: allows a Lecturer to see only their own courses.
- UploadMaterial (POST): allows a Lecturer to upload a course material file for an owned course. The action validates the file extension, the file size, the owner, and applies the upload rate-limiting policy.

Figure 11: [INSERT SCREENSHOT HERE]

This screenshot shows the course list (administrator).

Figure 11b: [INSERT SCREENSHOT HERE]

This screenshot shows the course create form.

Figure 11c: [INSERT SCREENSHOT HERE]

This screenshot shows the course edit form.

Figure 11d: [INSERT SCREENSHOT HERE]

This screenshot shows the course material upload.

8.9 Enrollment System

The enrolment system is implemented in `Controllers/EnrollmentsController.cs`. The Browse action lists eligible courses for the signed-in Student, joining the courses table with the student's enrolments to compute eligibility. The Enroll POST action checks that the course exists, that the course is open (i.e. capacity has not been reached), that the student has not

already enrolled (uniqueness check), and that any prerequisite has been satisfied. If all checks pass, the action creates an Enrollment record and writes an audit log entry.

The eligibility check uses a single composed LINQ query rather than separate queries to keep the database round-trip count low. The capacity check uses the count of existing enrolments for the course compared with the EnrollmentLimit field.

Figure 12: [INSERT SCREENSHOT HERE]

This screenshot shows the course browse (student).

Figure 12b: [INSERT SCREENSHOT HERE]

This screenshot shows the successful enrolment confirmation.

Figure 12c: [INSERT SCREENSHOT HERE]

This screenshot shows the duplicate or full capacity rejection message.

8.10 Assignment Management

Assignment management is implemented in Controllers/AssignmentsController.cs and the Views/Assignments/ folder. The actions are:

- ForCourse: lists assignments for a given course.
- Mine: lists assignments owned by the signed-in lecturer.
- Create (GET and POST): allows a Lecturer to create an assignment for an owned course using AssignmentInputViewModel.
- Edit (GET and POST): allows a Lecturer to update their own assignment.
- Delete (GET and POST): allows a Lecturer to delete their own assignment.

The validation enforced through AssignmentInputViewModel includes a required title, a maximum length on the description, a future or near-future due date, and a sensible MaxPoints range.

Figure 13: [INSERT SCREENSHOT HERE]

This screenshot shows the assignment list (lecturer).

Figure 13b: [INSERT SCREENSHOT HERE]

This screenshot shows the assignment create form.

8.11 Submission and Grading

Submission and grading are implemented in Controllers/SubmissionsController.cs and the Views/Submissions/ folder. The Student-facing action Submit accepts text content and a file, validates that the student is enrolled on the course, validates the file using UploadedFileStore, and creates or updates a Submission record. If the submission has already been graded, the submission is locked and further changes are not allowed.

The Lecturer-facing action Grade checks that the lecturer owns the assignment's course, displays the submission, and records a grade and feedback through GradeSubmissionViewModel. The grade is constrained to a sensible range and the feedback is constrained to a maximum length.

The action ForAssignment allows a Lecturer to see all submissions for an assignment. The actions CourseMaterialFile and SubmissionFile provide secure file download paths that check role and ownership before streaming the file.

Figure 13c: [INSERT SCREENSHOT HERE]

This screenshot shows the submission form (student).

Figure 13d: [INSERT SCREENSHOT HERE]

This screenshot shows the grading form (lecturer).

Figure 13e: [INSERT SCREENSHOT HERE]

This screenshot shows the graded submission view (student).

8.12 Reporting and Analytics

The reporting suite is implemented in Controllers/ReportsController.cs and the Views/Reports/ folder. The reports include:

- CoursePopularity: groups enrolments by course and orders by enrolment count.
- StudentPerformance: aggregates grades per student.
- LecturerWorkload: aggregates assignments and pending submissions per lecturer.
- Enrollments: lists enrolments with filters such as course or date.
- PassFail: groups submissions into pass and fail buckets using a configurable threshold.
- AssignmentAttendance: shows the proportion of enrolled students who submitted each assignment.

CSV export is provided through Infrastructure/CsvWriter.cs, which writes properly escaped CSV content. PDF export is provided through Infrastructure/PdfReport.cs, which uses QuestPDF to render the report layout. The QuestPDF licence is set to Community in Program.cs.

Figure 14: [INSERT SCREENSHOT HERE]

This screenshot shows the reports landing page.

Figure 14b: [INSERT SCREENSHOT HERE]

This screenshot shows the sample report output.

Figure 14c: [INSERT SCREENSHOT HERE]

This screenshot shows the csv export evidence.

Figure 14d: [INSERT SCREENSHOT HERE]

This screenshot shows the pdf export evidence.

8.13 Communication Module

The communication module is implemented in `Controllers/MessagesController.cs` and the `Views/Messages/` folder. The actions are:

- **Inbox:** lists messages for the signed-in user using `MessageInboxRowViewModel`.
- **Thread:** shows a conversation between two users using `MessageThreadViewModel`.
- **Reply (POST):** adds a reply message to a thread.
- **Compose (GET and POST):** allows the user to compose a message to an allowed recipient using `MessageComposeViewModel`. The validation rule is that a Student may message a Lecturer who teaches a course on which the student is enrolled, and a Lecturer may message a Student who is enrolled on a course owned by the lecturer. Administrators can message any user where required by the role.
- **MarkAllRead:** marks all messages in the inbox as read.

Figure 15: [INSERT SCREENSHOT HERE]

This screenshot shows the inbox view.

Figure 15b: [INSERT SCREENSHOT HERE]

This screenshot shows the message thread view.

Figure 15c: [INSERT SCREENSHOT HERE]

This screenshot shows the compose message form.

8.14 Meeting Scheduling

Meeting scheduling is implemented in `Controllers/MeetingsController.cs` and the `Views/Meetings/` folder. The actions are:

- **Index:** lists meetings for the signed-in user.
- **Create (GET and POST):** allows a Lecturer to schedule a meeting for an owned course using `MeetingInputViewModel`. The form captures the title, the description, the scheduled date and time, the duration, and an optional meeting URL.
- **Edit (GET and POST):** allows a Lecturer to update an owned meeting.
- **Delete (GET and POST):** allows a Lecturer to delete an owned meeting.
- **Join:** redirects the user to the meeting URL after authorisation checks.
- **Ics:** produces an ICS calendar file using `Infrastructure/CalendarLink.cs`, which can be imported into a calendar application.

Infrastructure/MeetLinkGenerator.cs provides a placeholder for generating meeting links where required. The current implementation stores a URL provided by the lecturer rather than provisioning a video conference, which keeps the system within the documented scope.

Figure 15d: [INSERT SCREENSHOT HERE]

This screenshot shows the meetings list.

8.15 Course Materials and File Uploads

Course materials are implemented through the CourseMaterial entity, the MaterialUploadViewModel, and the UploadMaterial action of CoursesController. Uploaded files are stored on disk through Infrastructure/UploadedFileStore.cs, which validates the file extension and the file size, generates a stored file name to avoid collisions, and writes the metadata into the database.

Downloads are served through controlled actions that confirm role and ownership before streaming the file. The action does not expose direct URLs into the file system, which prevents path traversal and unauthorised access to stored files.

8.16 Profile and Password Management

The profile and password actions are implemented in Controllers/ProfileController.cs and the Views/Profile/ folder. The Index action allows the signed-in user to view and update their profile fields through ProfileViewModel. The ChangePassword action allows the user to change their password under authentication, using the standard Identity password change API.

8.17 Audit Logging

Audit logging is implemented through Models/AuditLog.cs, Infrastructure/IAuditLogger.cs, Infrastructure/AuditLogger.cs, and the AuditLogsController. The audit logger writes an AuditLog record with the following fields: category (for example Account, Course, Enrollment, Submission, Security), action (for example Login, Register, Create, Update, Delete, Grade, Upload, Download), detail (a short, human-readable description), user identifier (where available), and a success flag. The log is reviewed through Views/AuditLogs/Index.cshtml, which displays paged log entries for an Administrator.

The audit logger is wrapped in a try/catch block inside the implementation so that a failure to write an audit entry does not crash the calling action. This is appropriate because the audit log is a supplementary record rather than the primary data store. If a log entry cannot be written,

the action continues, and the failure is reported through the standard exception handler to the application logs.

8.18 Infrastructure Services

The `Infrastructure/` folder contains services that support the controllers without holding business state. The full list is:

- `AuditLogger.cs` and `IAuditLogger.cs`: audit logger described in Section 8.17.
- `CalendarLink.cs`: builds ICS calendar content for meetings.
- `CsvWriter.cs`: writes CSV content with escaping.
- `EmailSettings.cs`: strongly typed settings for SMTP.
- `EmailTemplates.cs`: holds the email templates used by the password reset flow.
- `IEmailService.cs` and `SmtplibEmailService.cs`: email service contract and SMTP implementation.
- `MeetLinkGenerator.cs`: helper used during meeting creation.
- `PdfReport.cs`: PDF rendering using QuestPDF.
- `SecurityHeadersMiddleware.cs`: custom middleware that sets security headers on every response.
- `UploadedFileStore.cs`: manages uploaded file validation and storage.

Each service is registered through dependency injection in `Program.cs` where appropriate. Controllers depend on the interface (for example `IAuditLogger`, `IEmailService`) rather than on the concrete implementation, which keeps the controllers easy to test and replace.

8.19 UI Layout and Static Assets

The UI layout is centralised in `Views/Shared/_Layout.cshtml`. The layout defines the document head, the navigation bar, the alert partial, the main content section, and the footer. The alerts partial `Views/Shared/_Alerts.cshtml` displays success, information, warning, and error messages using `TempData`. The `Views/Shared/_DashboardCalendar.cshtml` partial is reused by the dashboards to render a small calendar block.

Static assets live under `wwwroot/`. The `lib/` folder holds Bootstrap and any other client libraries; the `css/site.css` file holds project-specific styles; the `js/site.js` file holds custom client-side behaviour; the `assets/` folder holds images and icons used by the UI. The author has kept the custom CSS and JavaScript small to avoid duplicating the framework features and to keep the code reviewable.

8.20 Hosting and Docker Deployment Evidence

In addition to the local Visual Studio build path, the author has prepared supporting hosting evidence that shows the application running outside the local development environment. The application has been packaged into a Docker container and deployed to a Google Cloud virtual machine. The UniManage application has also been hosted for demonstration using a custom HTTPS domain: <https://lms.yasaboy.com/>. This hosted version provides supporting evidence that the application can run outside the local Visual Studio environment. The deployment remains supporting evidence only, because the coursework brief still requires the application to be assessable through Visual Studio 2017 or higher.

Docker has been used because it provides a consistent runtime environment across the developer's machine and the cloud host. A Dockerfile is used to describe the .NET 8 base image, the build steps for the ASP.NET Core MVC application, and the entry point for the published binary. Once the image is built locally, it can be pushed to a container registry or transferred directly to the Google Cloud virtual machine and started with a single docker run command. This approach removes most of the differences between the developer's machine and the host machine, which makes the demonstration easier to reproduce. The author has not embedded the Dockerfile content in this report because the contents may include configuration details that should remain in the source repository.

The hosted demonstration is reached at <https://lms.yasaboy.com/> over HTTPS using a custom domain. An earlier IP-based endpoint, <http://34.171.6.234:8080>, was used during the initial deployment phase and is retained in this report as backup evidence in case the custom-domain endpoint is temporarily unreachable. After the application loads, the seeded demonstration users (admin@unimanage.local, lecturer@unimanage.local, and student@unimanage.local) can be used to sign in and walk through the role-based dashboards. The hosted version mirrors the local build because it runs the same Docker image. If the deployment uses a reverse proxy such as Nginx, or a DNS or content-delivery provider such as Cloudflare, the configuration of those components has not been embedded in this report and is recorded as [VERIFY] until supporting configuration or screenshot evidence is included.

The hosted demonstration is supporting evidence rather than the primary assessment artefact. The CS6004ES Application Development brief specifies that the application should be assessed using Visual Studio 2017 or higher, which is the local development environment

used by the marker. The local build path remains the main assessment route. The author confirms that dotnet build --no-restore returns zero warnings and zero errors, that Visual Studio 2022 opens the solution file AD COURSEWORK 2.sln correctly, and that the application starts on https://localhost:7212 and http://localhost:5103 from the local machine.

Several practical considerations apply to the hosted version. The custom-domain endpoint provides HTTPS and a clean address, but the wider deployment is still a coursework demonstration rather than a production service. The connection string, the SMTP password, the Google client secret, the SSH private keys, the DNS or provider credentials, and the cloud service account key are kept outside the report and outside any screenshot used as evidence. Where a screenshot of the configuration is required, the credentials are redacted before insertion. The marker is asked to verify the hosted URL shortly before the submission because cloud instances can be stopped, restarted, or reconfigured, which can temporarily make the endpoint unreachable. If the custom-domain URL is unavailable, the earlier IP-based endpoint may be used as a fallback, and the local Visual Studio run instructions in the user manual remain the supported demonstration path.

The full evidence checklist for the hosted deployment is summarised in Table 23 below and is repeated as a tracking list in Appendix E.4.

Table 23: Hosting and Deployment Evidence

Evidence Item	Details	Status	Notes
Hosted application domain	https://lms.yasaboy.com/	Completed	Main hosted demonstration URL
Previous IP-based endpoint	http://34.171.6.234:8080	Completed / Backup evidence	Earlier Google Cloud hosted endpoint, retained as fallback evidence
Hosting platform	Google Cloud	Completed	VM-hosted deployment evidence
Containerisation	Docker	Completed	Application packaged for deployment
HTTPS access	Configured	Completed	HTTPS is available through the custom domain
Custom domain	lms.yasaboy.com	Completed	Domain-based access configured
Nginx reverse proxy	[VERIFY]	Needs confirmation	Include only if configuration or screenshot evidence is available
Cloudflare or DNS provider	[VERIFY]	Needs confirmation	Include only if a DNS or provider screenshot is available
Visual Studio local assessment	Required	Completed for build, To verify by marker	Coursework assessment route as per the brief

Secrets and credentials	Not included in report	Safe	Kept outside appsettings.json screenshots and outside the body of the report
-------------------------	------------------------	------	--

9. User Interface and Usability

9.1 Design Language

The design language of UniManage is built on Bootstrap. The framework provides a consistent grid, typography, and component set, which the author has used without heavy customisation. The design language emphasises clarity over decoration. Headings use the framework's heading classes, primary actions use solid buttons, secondary actions use outline buttons, and destructive actions use the btn-danger style.

The author has avoided introducing decorative effects that distract from the academic content. Tables are compact but not stripped of borders; cards are used for dashboard summaries; alerts are used for transient feedback after a form submission. The result is a clean, professional appearance suitable for a university course management application.

9.2 Layout Structure

The layout structure is defined in `_Layout.cshtml`. The page is composed of the following parts:

- A top navigation bar with the brand link, role-aware items, profile menu, and a sign-in or sign-out action.
- A main content area where each Razor view contributes its content.
- A footer with the project name and the academic context.
- A skip-to-main-content link near the top of the page to support keyboard navigation.

The `_Alerts.cshtml` partial is included in the layout so that any TempData-based feedback message is shown automatically after a form submission. The `_DashboardCalendar.cshtml` partial renders a calendar block when included by a dashboard view.

9.3 Role-Aware Navigation

The navigation bar adapts to the signed-in user's role. A Student sees links for the Student dashboard, course browsing, assignments, submissions, messages, and meetings. A Lecturer sees links for the Lecturer dashboard, owned courses, assignments, submissions, materials,

messages, and meetings. An Administrator sees links for the Administrator dashboard, the course catalogue, reports, audit logs, and meetings.

The role-aware navigation is implemented using Razor conditions inside the layout. The view checks `User.IsInRole(AppRoles.X)` before rendering each role-specific link. Anonymous users see only the home, login, and register links. This pattern reduces navigation noise and ensures that no link points to a forbidden action.

9.4 Forms and Validation Feedback

Forms in UniManage use the standard ASP.NET Core MVC form pattern. Each form binds to a view model rather than to an entity. The form fields use Tag Helpers such as `asp-for`, `asp-validation-for`, and `asp-validation-summary`. The view models declare validation rules through data annotations such as `[Required]`, `[StringLength]`, `[Range]`, and `[EmailAddress]`. When the form is posted with invalid data, the controller returns the same view with the populated model and the validation errors are displayed next to the offending field.

The `_ValidationScriptsPartial.cshtml` partial loads the client-side jQuery validation scripts for client-side validation. Server-side validation always runs in addition to client-side validation, so a malicious client cannot bypass the rules.

9.5 Responsive Design

The Bootstrap grid provides responsive layout support. Forms, tables, and dashboards adjust their column widths according to the viewport. The author has avoided fixed pixel widths in custom CSS so that the layout remains usable on smaller screens. The navigation bar uses the framework's collapse component to convert into a hamburger menu on narrow viewports.

Figure 9c (optional): [INSERT SCREENSHOT HERE]

This screenshot shows a screenshot of the dashboard at a narrower viewport could be included as evidence of responsive design.

9.6 Accessibility Considerations

The author has applied several accessibility considerations:

- A skip link near the top of the layout allows keyboard users to bypass the navigation and jump to the main content.
- Form labels are associated with their input fields through Tag Helpers, which improves screen-reader behaviour.
- Headings follow a logical structure (h1 for the page title, h2 for major sections).

- Colour is not used as the only way to communicate state; alerts use both colour and icon or text.
- Buttons have meaningful text rather than relying on icons alone.

A full accessibility audit (for example WCAG 2.1 AA compliance) has not been carried out and is therefore marked [NEEDS CONFIRMATION] in the limitations chapter.

9.7 Error and Confirmation Messages

Error and confirmation messages are shown through three mechanisms:

- Inline validation messages next to form fields, generated through Razor Tag Helpers.
- A page-level validation summary at the top of forms, generated through asp-validation-summary.
- TempData-based alerts shown by the _Alerts.cshtml partial after a redirect.

For deeper errors that are not caused by user input, the application displays the friendly error page Views/Shared/Error.cshtml in production. The friendly page does not reveal stack traces or internal server detail. In development, the developer exception page is shown instead.

Figure 16: [INSERT SCREENSHOT HERE]

This screenshot shows the validation error example.

Figure 16b: [INSERT SCREENSHOT HERE]

This screenshot shows the friendly error page.

10. Detailed Description of Classes, Properties and Methods

10.1 Main Controllers and Responsibilities

Table 4 summarises the controllers and their responsibilities, key actions, related views, and the coursework area that they serve.

Table 4: Main Controllers and Responsibilities

Controller	Main Responsibility	Key Actions / Methods	Related Views	Coursework Area
AccountController	Authentication and account flows	Register, Login, Logout, ForgotPassword, ResetPassword, GoogleCallback	Views/Account/	Registration / Login
DashboardController	Role dashboard routing and dashboard data	Index, Student, Lecturer, Administrator	Views/Dashboard/	Dashboards

CoursesController	Course CRUD, lecturer courses, materials	Index, MyCourses, Details, Create, Edit, Delete, UploadMaterial	Views/Courses/	Course Management
EnrollmentsController	Student course browsing and enrolment	Browse, Enroll	Views/Enrollments/Browse.cshtml	Enrollment
AssignmentsController	Assignment CRUD and listing	ForCourse, Mine, Create, Edit, Delete	Views/Assignments/	Assignment Management
SubmissionsController	Submission, grading, file access	Submit, Grade, ForAssignment, CourseMaterialFile, SubmissionFile	Views/Submissions/	Submission and Grading
ReportsController	Reporting and exports	CoursePopularity, StudentPerformance, LecturerWorkload, Enrollments, PassFail, AssignmentAttendance	Views/Reports/	Reporting
MessagesController	Messaging workflow	Inbox, Thread, Reply, Compose, MarkAllRead	Views/Messages/	Communication
MeetingsController	Meeting scheduling and ICS export	Index, Create, Edit, Delete, Join, Ics	Views/Meetings/	Communication
ProfileController	Profile and password change	Index, ChangePassword	Views/Profile/	Account Management
AuditLogsController	Audit log review	Index	Views/AuditLogs/Index.cshtml	Security Evidence
HomeController	Public landing and error page	Index, Privacy (if present), Error	Views/Home/, Views/Shared/Error.cshtml	Application Foundation

10.2 Main Models and Properties

Table 5 summarises the main domain models and their properties.

Table 5: Main Models and Properties

Model	Key Properties	Purpose	Database Relationship
ApplicationUser	Id, FullName, Email, PhoneNumber (plus inherited Identity fields)	Extends Identity user with profile fields	Related to Course (lecturer), Enrollment (student), Submission (student/grader), Message (sender/receiver), CourseMaterial (uploader), Meeting (lecturer), AuditLog (user)
Course	CourseId, Code (unique),	Course record	One lecturer, optional

	Name, Description, Credits, EnrollmentLimit, LecturerId, PrerequisiteId		prerequisite, many enrolments, assignments, materials, meetings
Enrollment	EnrollmentId, StudentId, CourseId, EnrolledAtUtc	Student-course link	Many-to-one with student and course; unique on the pair
Assignment	AssignmentId, CourseId, Title, Description, DueDateUtc, MaxPoints	Coursework task	Many assignments per course
Submission	SubmissionId, AssignmentId, StudentId, Status, Grade, Feedback, SubmittedAtUtc, GradedById, GradedAtUtc, file metadata	Student submission and grading record	Many submissions per assignment
SubmissionStatus	Submitted, Graded (enum)	Submission status	Used by Submission
Message	MessageId, SenderId, ReceiverId, Subject, Content, IsRead, SentAtUtc	Communication record	Two foreign keys to ApplicationUser
CourseMaterial	CourseMaterialId, CourseId, Title, StoredFileName, OriginalFileName, UploadedById, UploadedAtUtc	Uploaded teaching material metadata	Many materials per course
Meeting	MeetingId, CourseId, LecturerId, Title, Description, ScheduledAtUtc, DurationMinutes, MeetingUrl	Meeting record	Many meetings per course
AuditLog	AuditLogId, Category, Action, Detail, UserId, Success, CreatedAtUtc	Audit trail	Optional foreign key to ApplicationUser
AppRoles	Administrator, Lecturer, Student (constants)	Role name constants	Used by Identity roles and [Authorize]
ErrorViewModel	RequestId, ShowRequestId	Used by error view	Not persisted

10.3 Main Methods and Purpose

Table 6 lists the main methods that drive the application.

Table 6: Main Methods and Purpose

Class / File	Method	Purpose	Input / Output	Error Handling / Validation
Program.cs	service configuration	Registers DbContext, Identity, email, audit, rate	App configuration to service container	Throws if connection string missing

		limiting, anti-forgery and MVC		
Program.cs	middleware configuration	Configures HTTPS, security headers, routing, rate limiter, authentication, authorisation	Request pipeline	Production exception handler vs developer exception page
DbInitializer	SeedAsync	Applies migrations and creates demo roles, users, and sample data	Service provider to seeded database	Throws on failed user creation
AccountController	Register	Creates Student or Lecturer user	Register form to signed-in user	Role check, password validation, audit logging
AccountController	Login	Authenticates user with lockout	Login form to dashboard redirect	ModelState, lockout, audit failure
AccountController	ForgotPassword	Sends password reset email	Email address to email send	Token generation, generic response to avoid user enumeration
AccountController	ResetPassword	Resets password using token	Token and new password to updated user	Token validation, password rules
DashboardController	Index	Routes by role	Role claim to role action	Returns 403 if unauthorised
DashboardController	Student / Lecturer / Administrator	Builds role-specific dashboard view models	EF Core queries to Razor view	Role attribute
CoursesController	Create	Creates course	Course form to database record	Lecturer and prerequisite validation, unique code
CoursesController	Edit	Updates course	Course form to database record	Concurrency and validation
CoursesController	UploadMaterial	Uploads course material	File and title to stored metadata	File type, size, owner, rate limit
EnrollmentsController	Browse	Lists eligible courses	Student claim to course list	Role attribute
EnrollmentsController	Enroll	Enrols student on course	Course ID to enrolment record	Duplicate, capacity, prerequisite checks, audit
AssignmentsController	Create	Creates assignment	Assignment form to database record	Course ownership and model validation
AssignmentsController	Edit / Delete	Updates or removes assignment	Assignment ID and form	Course ownership
SubmissionsController	Submit	Saves student submission	Text/file to submission	Enrolment, file validation, graded

			record	lock
SubmissionsController	Grade	Records grade and feedback	Grade form to updated submission	Lecturer ownership and score range
SubmissionsController	CourseMaterialFile / SubmissionFile	Streams a stored file	File ID to file content	Role and ownership
ReportsController	report actions	Builds analytics rows and exports	Query output to view, CSV, or PDF	Role attribute
MessagesController	Compose	Sends message	Recipient, subject, content	Allowed-recipient validation
MessagesController	Inbox / Thread / Reply	Lists or extends a conversation	View models to Razor views	Role attribute
MeetingsController	Create / Edit / Delete	Manages meetings	Meeting form to meeting record	Course ownership, URL and time validation
MeetingsController	Join / Ics	Joins meeting or downloads ICS	Meeting ID to redirect or file	Role and ownership
ProfileController	Index / ChangePassword	Profile and password update	Profile form, password form	Authentication and validation
AuditLogsController	Index	Lists audit log entries	View model to Razor view	Administrator role
AuditLogger	LogAsync	Saves audit record	Category, action, detail, user, success	Internal try/catch logging
UploadedFileStore	SaveAsync	Validates and stores uploaded file	File stream and metadata	File type, size, naming
CsvWriter	Write	Writes CSV content	Rows to stream	Escapes commas, quotes, newlines
PdfReport	Build	Builds PDF using QuestPDF	Data to byte array	Layout exceptions handled by middleware
SmtpEmailService	SendAsync	Sends email using SMTP	Subject, body, recipient	Connection and auth errors
SecurityHeadersMiddleware	InvokeAsync	Adds security headers to every response	HttpContext to response headers	Headers added before next middleware

10.4 Controller Layer Explanation

Each controller in UniManage follows a consistent structure. The class is decorated with the `[Authorize]` attribute (or `[Authorize(Roles = ...)]` where the controller is role-specific). The controller depends on `ApplicationDbContext`, on `UserManager<ApplicationUser>` (where user lookups are needed), on `SignInManager<ApplicationUser>` (in account controllers), and on infrastructure interfaces such as `IAuditLogger`, `IEmailService`, and `UploadedFileStore` where required.

Action methods are asynchronous and return `Task<IActionResult>`. Each action begins with `ModelState.IsValid` checks where appropriate, performs ownership and role checks, executes the EF Core query or update, writes an audit log entry where appropriate, and returns a redirect or a view. The action does not contain duplicated business logic; shared helpers are placed in services or in private methods.

10.5 Model and ViewModel Layer Explanation

Models are plain C# classes with primary key, foreign key, and field properties. They use data annotations for length, range, and required validation. They are mapped to database tables through `ApplicationDbContext` and the EF Core conventions.

View models are also plain C# classes, but they are not persisted. They carry data between controllers and views. The author has used view models for every form so that user input cannot bind directly to entity properties that should not be exposed (a defence against over-posting). For example, `CourseInputViewModel` exposes only the course fields that are supposed to be set during create or edit.

10.6 Infrastructure and Service Layer Explanation

The Infrastructure layer encapsulates services that are not strictly part of the domain model but are needed by multiple controllers. The layer contains:

- The audit logger (`AuditLogger`, `IAuditLogger`), which saves audit records and is used by every controller that performs a security-sensitive action.
- The email service (`SmtpEmailService`, `IEmailService`, `EmailSettings`, `EmailTemplates`), which is used by the password reset flow.
- The file store (`UploadedFileStore`), which is used by submissions and course materials.
- The CSV writer (`CsvWriter`), which is used by the reports controller.
- The PDF report builder (`PdfReport`), which is used by the reports controller.
- The calendar link helper (`CalendarLink`) and the meeting link generator (`MeetLinkGenerator`), which are used by the meetings controller.
- The custom security headers middleware (`SecurityHeadersMiddleware`), which is registered in `Program.cs` and runs for every response.

Each service uses dependency injection in the constructor, which makes the service replaceable in tests or in different deployments.

11. Logical Solution Explanation

11.1 Overall Request-to-Response Flow

The end-to-end request flow can be summarised as follows:

15. The browser sends a request to the application.
16. Kestrel accepts the connection and passes it to the middleware pipeline.
17. HTTPS redirection ensures the request is on the secure scheme.
18. The static file middleware serves any request that targets a file in wwwroot.
19. The custom security headers middleware adds standard headers to the response.
20. Routing matches the request to a controller and action.
21. Rate limiting checks the policy attached to the action and either allows the request or returns 429.
22. Authentication identifies the user from the cookie.
23. Authorisation checks the [Authorize] attribute and the role.
24. The controller action runs. It validates the model, performs business rules, queries or updates EF Core, and writes audit log entries.
25. EF Core translates the LINQ queries into MySQL SQL and executes them through the Pomelo provider.
26. The action returns either a redirect or a view.
27. The Razor engine renders the view using the view model.
28. The response flows back through the pipeline to the browser.
29. The browser renders the HTML and any small amount of JavaScript runs.

This flow is the same for every feature. The differences between features are the validation rules, the EF Core queries, and the resulting view. Keeping the flow consistent is one of the strengths of MVC and makes the project easier to maintain and to mark.

11.2 Registration and Login Logic

The registration logic accepts a RegisterViewModel, validates it through ModelState, builds an ApplicationUser with the provided full name, email, and phone number, and calls UserManager.CreateAsync. If the operation succeeds, the user is added to the chosen role (Student or Lecturer) through UserManager.AddToRoleAsync, signed in through SignInManager.SignInAsync, and recorded in the audit log. If the operation fails, the

validation errors from IdentityResult are added to ModelState and the form is returned with the errors.

The login logic accepts a LoginViewModel, validates it, and calls SignInManager.PasswordSignInAsync with lockoutOnFailure: true. The result indicates success, lockout, requires-two-factor, or failure. The action handles each case explicitly, writes an audit log entry, and either redirects to Dashboard/Index or returns the form with an error.

11.3 Dashboard Logic

DashboardController.Index inspects the role claim of the signed-in user and forwards to the role-appropriate action. Each role action runs a series of EF Core queries to fill the role-specific view model and returns the matching view. The author has consciously kept role-specific business rules out of Index so that the role mapping remains a single, easy-to-read switch.

11.4 Course Management Logic

The course management logic for an Administrator follows the standard CRUD pattern. Index lists courses, optionally filtered by a search term and paged. Create shows the form (GET) and saves the course (POST), enforcing the unique course code constraint. Edit and Delete follow the same pattern.

For a Lecturer, MyCourses filters the courses by LecturerId == userId, and UploadMaterial enforces ownership before saving the uploaded file.

11.5 Enrollment Validation Logic

The enrolment workflow contains several rules that demonstrate validation working together:

- The course must exist.
- The student must not already be enrolled on the course.
- The course must have spare capacity, computed as EnrollmentLimit - count(existing enrolments).
- The course's prerequisite, if any, must be satisfied by the student.
- The student must not have a soft business reason to be blocked, for example a temporarily disabled account.

If all rules pass, the action creates the Enrollment record, writes an audit log entry, and redirects to a confirmation page. If any rule fails, the action returns the browse view with a friendly error message. The combination of unique index (database) and rule check (application) provides defence in depth.

11.6 Assignment, Submission and Grading Logic

The assignment, submission, and grading workflow connects three controllers. `AssignmentsController.Create` lets a Lecturer set an assignment for an owned course. `SubmissionsController.Submit` lets a Student upload text and a file for an assignment they are enrolled on. `SubmissionsController.Grade` lets a Lecturer record a grade and feedback for a submission on an owned course.

The grading workflow is the most rule-heavy. The action verifies that the submission belongs to an assignment whose course is owned by the lecturer, displays the submission, accepts a grade and feedback through `GradeSubmissionViewModel`, validates the grade range, updates the Submission entity, sets the grader and grading time, and writes an audit log entry. After grading, the submission is locked so that the student can no longer modify it.

11.7 Reporting Logic

The reports controller runs an EF Core query for each report. Most queries use `GroupBy` and aggregate functions, and they project into a report view model. The view model is rendered through Razor for HTML output, written through `CsvWriter` for CSV output, or rendered through `PdfReport` for PDF output. The reports are gated by the Administrator role.

11.8 Communication Logic

Messages and meetings represent the communication features. The messaging logic enforces an allowed-recipient rule: a student can message a lecturer who teaches a course on which the student is enrolled, and vice versa. This rule is implemented as a database join during the recipient look-up, so the front end never offers an unauthorised recipient.

The meeting logic restricts scheduling and editing to the lecturer who owns the course. Students can see meetings that belong to courses on which they are enrolled. The ICS export is provided for users who want to import the meeting into a personal calendar.

11.9 Security and Validation Logic

Security and validation logic runs across every feature. Each form posts an anti-forgery token; each input is validated through data annotations and ModelState; each action is gated by [Authorize] and ownership checks; each controlled file download checks the requester's role and ownership before streaming the file; each security-sensitive action writes an audit log entry. The combination of these controls is described in Chapter 13.

12. Validation, Exception Handling and Error Management

12.1 Data Annotation Validation

Data annotation validation is the first line of defence. Each view model declares attributes such as [Required], [StringLength], [Range], [EmailAddress], [Compare], and [DataType]. The MVC framework runs these attributes during model binding and populates ModelState. Table 16 shows representative validation rules for selected forms.

Table 16: Validation Rules by Form

Form / View Model	Selected Validation Rules
RegisterViewModel	[Required] on full name, email, password; [EmailAddress] on email; [StringLength] on full name; password requirements enforced by Identity (8 chars, digit, upper, lower)
LoginViewModel	[Required] on email and password; [EmailAddress] on email
ForgotPasswordViewModel	[Required], [EmailAddress] on email
ResetPasswordViewModel	[Required] on token, password, confirm password; [Compare] on confirm password
CourseInputViewModel	[Required] on code and name; [StringLength] on code, name, description; [Range] on credits and enrolment limit
AssignmentInputViewModel	[Required] on title, due date; [StringLength] on title and description; [Range] on max points
SubmissionSubmitViewModel	[StringLength] on text content; file validation in controller
GradeSubmissionViewModel	[Range] on grade; [StringLength] on feedback
MessageComposeViewModel	[Required] on subject, content, recipient; allowed-recipient check in controller
MeetingInputViewModel	[Required] on title, scheduled time; [Url] on meeting URL where present; [Range] on duration
MaterialUploadViewModel	[Required] on title; file validation in controller
ProfileViewModel	[Required] on full name and email; [EmailAddress] on email; [Phone] on phone number where present

12.2 ModelState Handling

Every POST action that accepts user input begins with a ModelState.IsValid check. If the model state is invalid, the action returns the same view with the populated view model so that

the user can see and correct the validation errors. The author has avoided silent failures and has avoided redirecting to a generic error page when the user can recover by correcting the form.

12.3 Server-Side Ownership Checks

Some validation rules cannot be expressed as data annotations. For example, "a Lecturer can only edit assignments for courses they own" requires a database lookup. These checks are implemented inside the action after the ModelState check. If the check fails, the action returns either a 403 Forbidden response or a redirect to the appropriate index view with a TempData warning. The combination of role attributes and ownership checks prevents both unauthorised role access and unauthorised data access.

12.4 File Upload Validation

UploadedFileStore validates the uploaded file before storing it. The validation includes:

- File size: rejects files larger than the configured maximum.
- File extension: rejects files whose extension is not in the allowed list.
- File name: stores the file under a generated stored file name to avoid collisions and to prevent path traversal attacks.
- Content type: optional check against the allowed MIME types.

If the file fails validation, the action adds a model error and returns the form with an explanation. The rate-limiting policy upload further restricts the number of uploads per minute per user to prevent abuse.

12.5 Anti-Forgery Protection

Anti-forgery is configured globally in Program.cs through AddAntiforgery. Every Razor form generated through Tag Helpers includes a hidden anti-forgery token. POST actions that accept form data are decorated with [ValidateAntiForgeryToken]. The combination prevents cross-site request forgery attacks.

The anti-forgery cookie is configured with HttpOnly = true, SameSite = Strict, and SecurePolicy = SameAsRequest, which provides additional defence against cookie theft.

12.6 Exception Handling Middleware

In production, Program.cs configures `app.UseExceptionHandler("/Home/Error")` so that any unhandled exception is caught, logged, and replaced with the friendly error view. The view does not include any stack trace, source file, or server detail. In development, `app.UseDeveloperExceptionPage()` provides a detailed exception page that helps the author find the source of the problem.

The friendly error view uses `ErrorViewModel` to optionally show a request identifier, which can be cross-referenced with the application logs.

12.7 Friendly Error Pages

Apart from the global exception handler, individual actions use the standard MVC return types `BadRequest`, `NotFound`, and `Forbidden` to indicate specific error conditions. The framework renders these as the appropriate status code page, which can be customised through `app.UseStatusCodePagesWithReExecute` if required. The author has retained the standard behaviour and instead emphasised friendly TempData messages on success and on most foreseeable failures.

12.8 Audit and Error Evidence

Each security-sensitive failure is recorded in the audit log with `success = false`. This means that a marker reviewing the audit log can see not only successful actions but also blocked actions, such as failed sign-ins, rejected enrolments, and unauthorised access attempts. The audit log therefore acts as a forensic record as well as a behavioural record.

Figure 16: [INSERT SCREENSHOT HERE]

This screenshot shows the validation error example.

Figure 16b: [INSERT SCREENSHOT HERE]

This screenshot shows the friendly error page.

13. Security and Data Protection

Security is a cross-cutting concern in UniManage. The system is designed with defence in depth: each request passes through a chain of controls so that a failure of one control does not lead to a compromise.

13.1 Authentication

Authentication is provided by ASP.NET Core Identity. The `ApplicationUser` class extends `IdentityUser` and adds profile fields such as `FullName` and `PhoneNumber`. The `Identity`

options enforce a minimum password length of eight characters, require a digit, require an uppercase letter, and require a lowercase letter. A non-alphanumeric character is not strictly required, which keeps the password rule consistent with the seeded credentials.

Account lockout is enabled. After five failed attempts, the account is locked for fifteen minutes. The lockout policy reduces the effectiveness of brute-force attacks. The login form is also protected by the auth rate-limiting policy, which allows ten requests per IP address per minute. Combined, the lockout and rate-limiting policies make brute-force attacks impractical against a single account or against the system as a whole.

Optional Google login is supported. Where the `Authentication:Google:ClientId` and `Authentication:Google:ClientSecret` configuration values are present, the Google authentication handler is registered in `Program.cs`, including a `OnRemoteFailure` handler that redirects to the login page with the failure reason. Google login is not required for marking, and the demonstration users use plain Identity credentials.

13.2 Authorisation

Authorisation is enforced through `[Authorize]` attributes on controllers and actions, with `Roles` set to `AppRoles.Administrator`, `AppRoles.Lecturer`, or `AppRoles.Student` as appropriate. Where multiple roles are allowed, the attribute lists them separated by commas. The base controller class is decorated with `[Authorize]` so that anonymous users cannot reach any action by default; only the home page and the account flows are explicitly anonymous.

Ownership checks are added inside actions where role membership is not enough. For example, a Lecturer must only see their own courses, materials, assignments, submissions, and meetings. The ownership check is implemented as a database filter (`LecturerId == userId`) in the EF Core query, so the resulting view never contains rows that the user is not authorised to see.

13.3 Password Handling

Passwords are never stored in plain text. ASP.NET Core Identity uses PBKDF2 with HMAC-SHA-512 by default, with a per-user salt and configurable iteration count. Password reset uses Identity's tokenised flow. The password reset token is generated by `UserManager.GeneratePasswordResetTokenAsync`, sent through the email service, and validated by `UserManager.ResetPasswordAsync`. The reset endpoint is rate limited under the auth policy.

The author has not implemented "have I been pwned" checks or pre-generated weak password lists; this could be added in a future version. The current rules are sufficient for an academic deployment.

13.4 Role-Based Access Control

Role-based access control is the third major control. Three roles are seeded by DbInitializer: Administrator, Lecturer, Student. Each role is assigned to the corresponding demonstration user. New registrants choose either Student or Lecturer. The Administrator role is not available through self-registration; only the seeded administrator can use it. This is consistent with the principle that high-privilege roles should not be granted by self-service.

13.5 Protection Against Common Web Risks

Table 17 maps the security controls in UniManage to common web risks.

Table 17: Security Controls Mapped to Common Web Risks

Risk	Control in UniManage	Source-Code Evidence
Injection (SQL, command)	EF Core parameterised queries	All controller queries through DbSet and LINQ
Broken authentication	Identity with strong password rules, lockout, secure cookies	Program.cs Identity options, ConfigureApplicationCookie
Sensitive data exposure	HTTPS redirection, HSTS, secure cookie policy, redacted screenshots	Program.cs, User_Manual_Application_Development.md
Broken access control	[Authorize] attributes, role checks, ownership filters	All controllers
Security misconfiguration	Security headers, HSTS, secure cookie policy	Infrastructure/SecurityHeadersMiddleware.cs, Program.cs
Cross-site scripting (XSS)	Razor automatic HTML encoding, content-type checks	All views, model binding
Cross-site request forgery (CSRF)	Anti-forgery cookie and token	Program.cs, [ValidateAntiForgeryToken] attributes
Insecure deserialization	Strongly typed view models, no untrusted serialiser	View models throughout
Components with known vulnerabilities	Pinned package versions, latest patch level when developed	AD COURSEWORK 2.csproj
Insufficient logging and monitoring	Audit logger with IAuditLogger and AuditLog entity	Infrastructure/AuditLogger.cs, Models/AuditLog.cs
Brute-force attacks	Account lockout, rate limiter on auth policy	Program.cs lockout, AddRateLimiter
Path traversal in file storage	UploadedFileStore generates stored file names	Infrastructure/UploadedFileStore.cs
Excess upload load	Rate limiter on upload policy	Program.cs upload policy

13.6 File Security

File security combines validation, controlled storage, and controlled retrieval. Files are validated by `UploadedFileStore` and stored under a generated file name in a controlled directory. Files are retrieved through controller actions (`CourseMaterialFile`, `SubmissionFile`) that check role and ownership before streaming the file. Direct URL access into the upload directory is not exposed to the browser, which prevents direct file enumeration.

13.7 Audit Logging and Traceability

Audit logging is provided through `IAuditLogger` and `AuditLogger`. The audit log records the category of the event, the action, a short human-readable detail, the user identifier (where available), and a success flag. Categories include Account, Course, Enrollment, Assignment, Submission, Material, Message, Meeting, Report, and Security. Table 18 shows representative categories and actions.

Table 18: Audit Logger Categories

Category	Actions Recorded
Account	Register, Login, LoginFailed, Logout, PasswordReset
Course	Create, Update, Delete
Enrollment	Enroll, EnrollFailed
Assignment	Create, Update, Delete
Submission	Submit, Grade
Material	Upload, Download
Message	Send, Reply
Meeting	Create, Update, Delete, Join
Report	Generate, Export
Security	AccessDenied, RateLimited, AntiForgeryFailure (where reported)

Audit log entries are reviewed through `AuditLogsController.Index`, which is restricted to Administrators.

13.8 Data Protection Considerations

The author has applied several data protection considerations:

- Sensitive configuration (database password, SMTP password, Google client secret) is held in `appsettings.json` and is excluded from screenshots and the report.
- Personal data shown in screenshots is redacted before insertion in the report.
- The audit log records identifiers and actions but does not record passwords or sensitive payloads.
- HTTPS and HSTS protect data in transit.

- Cookies are HTTP-only and SameSite protected.
- The retention of audit log data is not formally documented; this is recorded as [NEEDS CONFIRMATION] in the limitations chapter.

14. Programming Style and Maintainability

Table 19 summarises the main programming style practices applied in the project.

Table 19: Programming Style Practices

Practice	Description	Evidence
Naming conventions	PascalCase for classes, methods, and properties; camelCase for parameters and local variables.	All controllers, models, services
MVC folder structure	Controllers, Models, ViewModels, Views, Data, Infrastructure	Project root
Single responsibility	Each service and controller addresses one area	Infrastructure folder
Dependency injection	Services are injected through constructors	All controllers
Asynchronous programming	EF Core, Identity, file, and email operations use async/await	All controllers and services
Strongly typed view models	Forms bind to view models, not entities	ViewModels folder
Routing conventions	Default {controller}/{action}/{id?} pattern	Program.cs
Configuration through appsettings.json	Connection strings, Google credentials, email settings	Program.cs
Migrations under source control	Data/Migrations is committed	Git history
Centralised security headers	Custom middleware sets headers consistently	Infrastructure/SecurityHeadersMiddleware.cs
Centralised audit logging	Single IAuditLogger service	Infrastructure/AuditLogger.cs
Build verification	dotnet build --no-restore returns 0 errors and 0 warnings	Build evidence

14.1 Naming and Readability

Names in the project are descriptive. Controller names end in Controller. Model names match the academic concept (Course, Enrollment, Submission). View models have a ViewModel suffix. Methods are named after the action they perform (Submit, Grade, Enroll). Local variables are named after their purpose. Acronyms such as Url, Id, and Pdf follow the .NET naming style.

14.2 MVC Separation of Concerns

Separation of concerns is enforced through the folder structure and through the use of dependency injection. Models contain data and validation rules. Controllers contain request flow and business rules. Views contain presentation. Services contain cross-cutting concerns. The data access layer is encapsulated behind `ApplicationDbContext`. This makes it possible to update one layer without touching the others.

14.3 View Models and Entities

View models and entities are kept separate. View models hold only the fields that the form expects. Entities hold the fields that are persisted. This protects against over-posting attacks where a malicious user adds extra fields to a form to update properties they should not be able to set.

14.4 Dependency Injection

All services are registered through `Program.cs` and consumed through constructor injection. The author has not used a service locator pattern. Lifetimes are chosen carefully: scoped for `ApplicationDbContext` and most services, singleton for stateless utility classes where used, and transient for short-lived services where necessary.

14.5 Asynchronous Programming

EF Core operations, Identity calls, file operations, and email calls are asynchronous. The asynchronous pattern keeps the application responsive and reduces thread pool starvation. Controller actions are also asynchronous and use `await` rather than `.Result` or `.Wait()`, which prevents deadlocks.

14.6 Comments and Code Clarity

The author has used comments sparingly. Self-documenting names, small methods, and conventional structure reduce the need for comments. Where a comment is added, it explains intent or trade-offs rather than restating the code. The author has avoided narrating obvious operations (such as "increment the counter") and has not added emojis or decorative comments.

14.7 Maintainability Improvements

The current codebase is maintainable, but there is always room for improvement. The reflection chapter (Chapter 19) and the future improvements chapter (Chapter 21) include

items such as automated tests, continuous integration, structured logging, and caching of read-heavy queries. These items would further improve maintainability without requiring a full rewrite.

15. Testing

15.1 Testing Strategy

The testing strategy combines manual functional testing, security testing, and build verification. Automated unit and integration tests are not included in the project; this is honestly noted as a limitation. Manual testing has been used because the marking criteria emphasise feature evidence and screenshots, and because the author has prioritised feature implementation and documentation given the approved individual submission scope.

The strategy follows three principles. First, every feature is tested at least once with a positive test case (the feature works) and one negative test case (the feature rejects bad input). Second, every test result is captured by a screenshot or by the audit log. Third, every test is traceable to a requirement and to a marking item.

15.2 Test Environment

The test environment is the author's local machine running Windows, Visual Studio 2022, .NET 8 SDK, MySQL 8, and a modern browser. The application is started with dotnet run or through Visual Studio. The seeded demonstration users are used for sign-in. The browser developer tools are used to verify HTTPS, cookie attributes, and request/response details where relevant.

15.3 Functional Testing

Functional testing covers the path through every major feature. Each functional test starts from a known state (for example a clean database after seeding) and ends with a verified outcome. The test plan in Section 15.9 lists the individual cases.

15.4 Validation Testing

Validation testing covers data annotations, ModelState behaviour, ownership checks, and file validation. Each form is tested with valid input and with input that violates one or more rules. The expected result is a friendly validation message and no database change.

15.5 Role-Based Access Testing

Role-based access testing confirms that a user in one role cannot access a controller action restricted to a different role. The test sequence is: sign in as a Student, attempt to open /Courses/Create, observe access denied; repeat for the Lecturer and Administrator. The audit log entries should also reflect the access denial.

15.6 Database Testing

Database testing confirms that records are created, updated, and deleted correctly. The verification is done by signing in to MySQL with a database tool and inspecting the table contents after each action. The unique constraints and foreign keys are also tested by attempting to violate them through the UI.

15.7 UI and Usability Testing

UI and usability testing covers navigation, layout, alerts, and responsive design. The expectation is that each page is reachable through the role-aware menu, that each form provides appropriate feedback, and that the layout adapts to a narrow viewport.

15.8 Build Verification

Build verification is performed with `dotnet build --no-restore` from the project root. The build has been verified to succeed with zero warnings and zero errors. The build evidence is available as a console capture.

15.9 Test Plan and Results

Table 8 lists the test plan with at least twenty-five test cases.

Table 8: Test Plan and Results

Test ID	Test Area	Test Scenario	Input / Setup	Expected Result	Actual Result	Status
T1	Registration	Register a new Student	Valid form	User created, signed in, audit log entry written	[SCREENSHOT REQUIRED]	Pending
T2	Registration	Register a new Lecturer	Valid form	User created, signed in	[SCREENSHOT REQUIRED]	Pending
T3	Registration validation	Submit register form with weak password	Password "abc"	Validation message, no user created	[SCREENSHOT REQUIRED]	Pending
T4	Login	Log in as seeded Student	Demo credentials	Redirect to Student dashboard	[SCREENSHOT REQUIRED]	Pending
T5	Login	Submit five	Wrong	Account	[SCREENSHOT	Pending

	logout	wrong passwords	password	locked, audit log entry	REQUIRED]	
T6	Logout	Sign out	Authenticated session	Redirect to home, cookie cleared	[SCREENSHOT REQUIRED]	Pending
T7	Forgot password	Request reset for known email	Valid email	Email sent (or token logged), generic confirmation	[SCREENSHOT REQUIRED]	Pending
T8	Student dashboard	Student opens dashboard	Authenticated student	Dashboard shows enrolled courses and deadlines	[SCREENSHOT REQUIRED]	Pending
T9	Lecturer dashboard	Lecturer opens dashboard	Authenticated lecturer	Dashboard shows teaching summary and pending submissions	[SCREENSHOT REQUIRED]	Pending
T10	Administrator dashboard	Administrator opens dashboard	Authenticated administrator	Dashboard shows counts and popular courses	[SCREENSHOT REQUIRED]	Pending
T11	UI navigation	Navigate using role-aware menu	Each role	Each role sees only relevant items	[SCREENSHOT REQUIRED]	Pending
T12	Course list	Administrator views course list	Administrator	List displayed with search and paging	[SCREENSHOT REQUIRED]	Pending
T13	Course create	Administrator creates a course	Valid form	Course saved, redirect, audit log entry	[SCREENSHOT REQUIRED]	Pending
T14	Course code uniqueness	Administrator creates two courses with the same code	Duplicate code	Second create rejected	[SCREENSHOT REQUIRED]	Pending
T15	Course delete	Administrator deletes a course	Course id	Course removed, related records handled	[SCREENSHOT REQUIRED]	Pending
T16	Enrolment success	Student enrolls on an open course	Course with capacity	Enrolment saved, confirmation displayed	[SCREENSHOT REQUIRED]	Pending
T17	Enrolment rejection	Student attempts duplicate enrolment	Already enrolled	Rejection message, no new record	[SCREENSHOT REQUIRED]	Pending

T18	Assignment create	Lecturer creates an assignment for an owned course	Valid form	Assignment saved	[SCREENSHOT REQUIRED]	Pending
T19	Assignment ownership	Lecturer attempts to edit an assignment for another lecturer	Other lecturer's assignment	Forbidden	[SCREENSHOT REQUIRED]	Pending
T20	Submission upload	Student submits text and file	Enrolled student	Submission saved, file stored	[SCREENSHOT REQUIRED]	Pending
T21	Grading	Lecturer grades a submission with valid grade	Valid grade and feedback	Grade and feedback saved, submission locked	[SCREENSHOT REQUIRED]	Pending
T22	Reporting	Administrator opens course popularity report	Administrator	Report rows displayed	[SCREENSHOT REQUIRED]	Pending
T23	CSV export	Administrator exports a report as CSV	Administrator	CSV file downloaded	[SCREENSHOT REQUIRED]	Pending
T24	Messaging compose	Student messages an allowed lecturer	Valid recipient	Message saved, thread updated	[SCREENSHOT REQUIRED]	Pending
T25	Messaging restriction	Student attempts to message an unrelated lecturer	Disallowed recipient	Message rejected	[SCREENSHOT REQUIRED]	Pending
T26	Anti-forgery	Submit POST without token using developer tools	No token	Request rejected	[SCREENSHOT REQUIRED]	Pending
T27	Rate limit	Submit eleven login requests within a minute	Eleven requests	Eleventh receives 429	[SCREENSHOT REQUIRED]	Pending
T28	Access denied	Student opens /Courses/Create	Authenticated student	Forbidden, audit log entry	[SCREENSHOT REQUIRED]	Pending
T29	Validation message	Submit invalid course form	Missing required fields	Inline validation messages	[SCREENSHOT REQUIRED]	Pending
T30	Friendly error	Trigger an error in production	Production environment	Error.cshtml page displayed	[SCREENSHOT REQUIRED]	Pending
T31	Responsive layout	Open dashboard on a narrow viewport	Browser resize	Layout adapts, navigation collapses	[SCREENSHOT REQUIRED]	Pending
T32	Build verification	Run dotnet build --no-restore	Project root	Zero warnings and zero	Confirmed (build succeeded)	Confirmed

				errors		
--	--	--	--	--------	--	--

16. Evaluation Against Coursework Brief

16.1 Marking Scheme Alignment

Table 7 evaluates the implementation against each marking item in the coursework brief.

Table 7: Coursework Marking Alignment

Marking Item	Evidence in Project	Strength	Risk
Application user interface	Razor views, _Layout.cshtml, _Alerts.cshtml, Bootstrap, site.css, role views	Strong source evidence	Screenshots required
User Registration and Login	AccountController, Identity options, anti-forgery, logout, audit	Strong source evidence	Screenshots required
Student Dashboard	DashboardController.Student, Views/Dashboard/Student.cshtml, view model	Strong source evidence	Screenshot required
Lecturer Dashboard	DashboardController.Lecturer, lecturer view, view model	Strong source evidence	Screenshot required
Administrator Dashboard	DashboardController.Administrator, administrator view, view model	Strong source evidence	Screenshot required; user-management screen [NEEDS CONFIRMATION]
Course Management	CoursesController, Course, CourseInputViewModel, course views	Strong source evidence	Screenshots required
Enrollment System	EnrollmentsController, Enrollment, CourseBrowseRowViewModel	Strong source evidence	Screenshots required
Assignment and Grading	AssignmentsController, SubmissionsController, view models	Strong source evidence	Screenshots required
Reporting and Analytics	ReportsController, CsvWriter, PdfReport, QuestPDF	Strong source evidence	Screenshots and export evidence required
Communication Module	MessagesController, MeetingsController, view models	Strong source evidence	Screenshots required
Security and Data Protection	Identity, RBAC, anti-forgery, rate limiting, security headers, audit	Strong source evidence	Audit log screenshot

		e	required
Installation Guide and User Manual	User_Manual_Application_Development.md	Draft prepared	Final Word/PDF export pending
Concise Logical Solution	Chapter 11 of this report	Draft prepared	Keep concise after Word formatting
Architecture Diagrams	documentation_application_development/diagrams/*.mmd	Sources prepared	Export required
Class/property/method description	Chapter 10 tables	Strong	Cross-check after final code freeze
Own Reflection	Chapter 19	Draft prepared	Strengthen with concrete examples
Programming Style	Chapter 14	Strong	Keep code samples short in appendix
Validation and Exception Handling	Chapter 12	Strong	Validation screenshot required
Interface Design and Usability	Chapter 9	Draft prepared	Screenshots required

16.2 Strengths of the Implementation

Table 20 summarises the strengths of the implementation.

Table 20: Strengths of the Implementation

Strength	Description
Complete MVC structure	Twelve controllers, twelve models, sixteen view models, forty-six views, and a clear infrastructure layer.
Strong security pipeline	Identity, lockout, anti-forgery, rate limiting, security headers, HTTPS, HSTS, audit logging.
Clean separation of concerns	Models, view models, controllers, views, and infrastructure are kept in dedicated folders.
Consistent role-based access	All controllers use [Authorize] with role attributes and ownership checks.
Reliable data layer	EF Core with retry on failure, three migrations, and a clear DbContext.
Reporting suite	Six reports with CSV and PDF export through QuestPDF.
Audit traceability	Every security-sensitive action is logged through IAuditLogger.
Build verified	dotnet build --no-restore returns 0 warnings and 0 errors.
Documentation package	Main report, individual contribution, user manual, evidence checklist, diagrams, and QA reports.
Approved individual submission	Wording is consistent with the approved individual completion route.

16.3 Remaining Risks

Table 21 lists the remaining risks before submission.

Table 21: Remaining Risks

Risk	Severity	Mitigation
Final screenshots not yet captured	High	Capture screenshots listed in Chapter 5 and Appendix B before Word export.
Diagrams not yet exported	High	Export Mermaid sources to PNG and PDF.
Repository link placeholder	High	Paste the final repository link into Source_And_Build_Links.md.
Build / deployment link placeholder	High	Paste the packaged build or hosted deployment link.
Database evidence not captured	Medium	Capture migration list output and a redacted table view.
Automated tests absent	Medium	Document as a future improvement; add manual test evidence in Chapter 15.
Administrator user-management screen [NEEDS CONFIRMATION]	Medium	Confirm with the brief; if required, document as future improvement.
Accessibility audit not carried out	Low	Document as future improvement; current measures listed in Chapter 9.

16.4 Evidence Still Required

The complete list of evidence still required is held in Evidence_Checklist.md and is summarised below:

- Screenshots for Figures 7 to 20b.
- Exported diagrams for Figures 1 to 6.
- Repository link, packaged build link, and database evidence in Source_And_Build_Links.md.
- Final Word and PDF version of this report.
- Optional individual approval evidence file in evidence/.

17. Challenges Faced

17.1 Technical Challenges

The main technical challenges encountered during the project were the integration of Identity with the Pomelo MySQL provider, the configuration of EF Core retry on failure to handle transient connection issues, the setup of the rate limiter with two distinct policies for authentication and uploads, the configuration of QuestPDF to operate under the Community licence, and the design of the audit logger to write log entries reliably without disrupting the calling action. Each challenge was addressed by reading the official documentation, by

checking the resulting behaviour through running the application, and by reading the audit log to confirm that events were captured as expected.

17.2 Documentation Challenges

The main documentation challenge was producing a long, evidence-based report without inventing features. The author has resolved this by reviewing the source code chapter by chapter and by marking unverifiable claims as [NEEDS CONFIRMATION] or [SCREENSHOT REQUIRED]. The author has also avoided copying content from unrelated coursework reports, which would damage academic integrity even if the surface text were rewritten.

17.3 Evidence Preparation Challenges

Capturing evidence in a marker-friendly way is harder than it looks. Screenshots need to be taken with the right resolution, the right window size, and with personal data redacted. Diagrams need to be exported in a vector or high-resolution format so that they print cleanly. Database evidence must avoid revealing connection strings or production-like credentials. The author has prepared the evidence checklist and the screenshots README so that each piece of evidence is captured with consistent quality.

17.4 Individual Completion Challenges

The approved individual completion route adds two challenges. First, it requires the author to take responsibility for every part of the work, including planning, time management, and submission. Second, it requires the documentation to remain neutral, factual, and respectful of other students. The author has addressed both challenges by structuring the work week by week, by maintaining the evidence checklist, and by following the wording rules in the documentation expansion plan.

18. Limitations

18.1 Functional Limitations

The current build does not include a dedicated administrator user-management screen, automated unit and integration tests, advanced reporting filters, or live video conferencing. Each of these is recorded as [NEEDS CONFIRMATION] or as a future improvement.

18.2 Testing Limitations

Automated tests are not present. Manual testing has been used to validate functional, validation, and security behaviour. The test plan in Chapter 15 lists thirty-two test cases, of which the build verification (T32) is confirmed. The remaining cases require screenshots and manual verification before submission.

18.3 Deployment Limitations

The application has been verified to run locally on `https://localhost:7212` and `http://localhost:5103`. A supporting hosted demonstration has been deployed to Google Cloud using a Docker container, with the main public endpoint at `https://lms.yasaboy.com/` and an earlier IP-based endpoint `http://34.171.6.234:8080` retained as backup evidence. The hosted endpoint and the Docker packaging are described in Section 8.20 and in Appendix E.4.

Although the hosted demonstration is now accessible through the custom HTTPS domain `https://lms.yasaboy.com/`, it is still presented as coursework demonstration evidence rather than a fully production-ready deployment. For production use, the author would need to document and harden the complete deployment pipeline, including reverse proxy configuration if used, DNS configuration, secret management, firewall rules, monitoring, database backup procedures, and CI/CD automation. Any Nginx or Cloudflare configuration should only be claimed if supporting configuration or screenshot evidence is included; at the time of writing, both items are recorded as [VERIFY] in Section 8.20 and Appendix E.4.

The marker is asked to verify the hosted URL shortly before submission. Cloud instances can be stopped, restarted, or reconfigured, and the public domain or IP may temporarily become unreachable. If the custom-domain URL is unreachable, the earlier IP-based endpoint may be used as a fallback. If neither hosted endpoint is reachable, the local Visual Studio assessment route documented in the user manual remains supported.

18.4 Accessibility Limitations

The author has applied basic accessibility considerations such as a skip link, label association, and semantic markup. A formal WCAG 2.1 AA audit has not been carried out and is therefore noted as a limitation.

18.5 Evidence Limitations

The screenshot, diagram, repository, build, and database evidence is to be captured before final submission. Until that capture is completed, the report records the requirement explicitly using the placeholders [SCREENSHOT REQUIRED], [DIAGRAM EXPORT REQUIRED], and [NEEDS CONFIRMATION].

19. Own Reflection on Experience

19.1 Technical Learning

The author has gained practical experience in several technical areas. Building UniManage required combining ASP.NET Core MVC routing, Razor views, EF Core, the Pomelo MySQL provider, Identity, anti-forgery, rate limiting, custom middleware, and dependency injection in a single project. The most useful technical learning has been the realisation that defence in depth is achieved by combining several controls rather than by relying on any one. For example, the unique constraint on Enrollment(StudentId, CourseId) and the application-level duplicate check work together to prevent duplicate enrolments; either control on its own would leave a gap.

The author has also learned to read and trust official documentation. Microsoft Learn, the Entity Framework Core documentation, the Pomelo MySQL provider documentation, the QuestPDF documentation, and the Bootstrap documentation have all influenced the design. Where a feature exists in the framework (for example anti-forgery), the author has used the framework feature rather than rebuilding it.

A further technical learning point is the importance of asynchronous programming. Database, file, and email operations all benefit from async/await, which keeps the application responsive and avoids blocking the thread pool. Working through the project has reinforced the discipline of writing asynchronous code from the outset rather than retrofitting it later.

19.2 Professional Learning

The author has gained professional learning around documentation, evidence handling, and academic integrity. Producing a substantial report requires planning, structure, and discipline. The author has used the documentation expansion plan to structure the work, has used the evidence checklist to track screenshots and diagrams, and has used the lecturer marking alignment report to keep the writing oriented towards the brief.

The author has also learned how to balance breadth and depth. A long report can become repetitive if every section says the same thing in different words. The author has therefore tried to give each chapter a clear focus and to avoid unnecessary overlap. Where overlap is unavoidable, the author has used cross-references rather than restating the same content.

19.3 Problem-Solving Experience

Several problems required deliberate problem-solving during the project. Three examples are given.

First, the integration of EF Core with the Pomelo MySQL provider initially produced transient connection errors during startup because the local MySQL service was slow to start. The author resolved this by enabling retry on failure with up to five retries and a thirty second maximum delay, which hid the transient failures from the user.

Second, the rate limiter was first configured with a single policy that applied to every endpoint, which caused unnecessary 429 responses on dashboard pages. The author resolved this by splitting the rate limiter into two named policies (auth and upload) and applying them only to the relevant actions.

Third, the audit logger initially failed silently when the audit table did not exist (during the first migration). The author resolved this by ordering the migrations carefully, ensuring that the audit table is created before any audit calls run, and by wrapping the audit write in a try/catch block so that an audit failure cannot crash the calling action.

19.4 Impact of Approved Individual Submission

The approved individual completion route had a significant impact on the planning, the workload, and the documentation. The author had to plan the project as a single-person delivery from the start, which required a clear scope and a clear weekly schedule. The workload was higher than for a group submission because every chapter, every screenshot, and every diagram is the author's own. However, the author found that the individual route also provides clarity. There is no risk of conflicting decisions, no need for coordination meetings, and no ambiguity around ownership.

Crucially, the individual route did not change the academic standards. The author still produced a complete code base, a complete documentation package, and a complete evidence set. The author's response to the individual completion approval has been to invest in evidence-based writing rather than to claim shortcuts.

19.5 Ethical and Academic Integrity Awareness

Academic integrity has been a central concern. The author has not copied text from unrelated coursework reports. Where official documentation has informed a decision, the documentation is acknowledged in the references chapter. Where a feature could not be confirmed by source code, the author has marked the claim as [NEEDS CONFIRMATION] rather than embellishing the report. The author has also kept sensitive configuration values out of the report and has redacted personal data in screenshots.

Data protection has informed several decisions. The audit log records identifiers and actions but does not record passwords or sensitive payloads. Personal data shown in screenshots is redacted. The retention period for audit log data is not formally documented, which is recorded as a limitation.

19.6 Future Learning Plan

The author plans to extend the technical and professional learning gained from this coursework. Three priorities are noted.

First, the author intends to learn automated testing in depth, including unit testing with xUnit, integration testing with the ASP.NET Core test host, and UI testing with Playwright. This will allow the author to ship code with regression tests rather than relying solely on manual testing.

Second, the author intends to learn continuous integration and deployment using GitHub Actions, which will support a hosted demonstration environment and consistent build verification.

Third, the author intends to expand the security and data protection awareness gained during this project by studying OWASP guidance, the .NET security documentation, and accessibility standards in more depth. These topics combine technical and professional learning, which is a useful foundation for further academic work and for industry practice.

20. Individual Contribution

The full individual contribution report is attached as a supporting document and is also held at `documentation_application_development/Expanded_Individual_Contribution.md`. The supporting document explains the author's approved individual completion context and summarises the implementation, testing, documentation, and submission preparation work.

For convenience, the contribution is summarised below in a single paragraph: the author selected and completed UniManage as an approved individual submission, has produced the source code, the documentation package, the diagrams, the user manual, the evidence checklist, and this main report. The author has reviewed the source code chapter by chapter, has prepared a marker-friendly mapping between requirements and evidence, and has marked unverifiable claims as [NEEDS CONFIRMATION] or [SCREENSHOT REQUIRED]. The contribution is therefore the entire delivery, prepared and submitted in accordance with the academic integrity rules of CS6004ES Application Development.

21. Future Improvements

Table 22 lists the future improvements that the author has identified.

Table 22: Future Improvements and Justification

Improvement	Justification
Automated unit and integration tests	Provide regression coverage and reduce reliance on manual testing.
Continuous integration and deployment	Provide consistent build verification and a hosted demonstration environment.
Accessibility audit (WCAG 2.1 AA)	Confirm that the user interface is accessible to all users.
Dedicated administrator user-management screen	Allow administrators to create, disable, or remove users without direct database access.
Advanced report filters and chart visualisations	Improve the academic insight provided by the reporting suite.
Email and in-app notifications	Notify students of new assignments and grades, and lecturers of new submissions.
Two-factor authentication	Strengthen authentication for high-privilege accounts.
Structured logging with Serilog or similar	Improve observability and integrate with log management tools.
Caching of read-heavy queries	Improve dashboard performance for large datasets.
Internationalisation and localisation	Support multiple languages where required.
Mobile-friendly enhancements beyond Bootstrap defaults	Improve the experience on small viewports.
Database backup automation	Provide a routine backup script and a restore process.
Maintain and renew HTTPS / TLS configuration and document the certificate setup	The hosted demonstration now uses the custom domain https://lms.yasaboy.com/ . Document the certificate authority used, the renewal cadence, and any automation in place so that the HTTPS configuration remains valid over time.
Document and harden the reverse proxy configuration if Nginx is used	If the deployment uses Nginx in front of the Docker container, capture the relevant configuration (with secrets redacted) and harden the proxy with appropriate timeouts, headers, and rate limits.
Document DNS and Cloudflare configuration if used	If the deployment uses Cloudflare or another DNS / content-delivery provider, capture redacted screenshots of the relevant DNS and provider settings to support the deployment evidence.

Move sensitive settings to environment variables or a managed secret store	Avoid keeping connection strings, SMTP credentials, Google client secrets, and DNS or provider credentials in committed configuration files; use environment variables, Google Secret Manager, or a similar managed store.
Add CI/CD deployment automation for Docker images	Automate the build, image push, and deployment of the Docker container so that updates to the source repository can be pushed to the hosted demonstration without manual steps.
Add uptime monitoring and database backup procedures	Add a basic uptime monitor for the hosted endpoint and a routine MySQL backup so that the demonstration link remains reliable across the assessment window.

22. Conclusion

UniManage has been delivered as an approved individual submission for CS6004ES Application Development Coursework 2. The project consists of a complete ASP.NET Core MVC application with twelve controllers, twelve domain models, sixteen view models, twelve infrastructure components, forty-six Razor views, three EF Core migrations, an Identity configuration with strong password rules and lockout, a rate limiter with two policies, anti-forgery, security headers, HTTPS, HSTS, and a custom audit logger.

The system addresses the academic management problem set by the brief, supports Student, Lecturer, and Administrator roles through role-based dashboards, implements course management, enrolment, assignment management, submission upload, grading, reporting and analytics with CSV and PDF export, internal messaging, meeting scheduling, profile and password management, and an audit log review screen. The system applies validation through data annotations, ModelState handling, ownership checks, file validation, anti-forgery, and exception handling.

The documentation package supports the submission with a main report, an individual contribution report, a user manual, an evidence checklist, diagram sources, screenshot guides, evidence guides, conversion guides, and final QA reports. The build has been verified locally with zero warnings and zero errors. Items that are not confirmed by source code or by screenshots are marked [NEEDS CONFIRMATION] or [SCREENSHOT REQUIRED]; the author will replace these markers with evidence before final submission.

The author has reflected on the technical, professional, and academic learning gained during the project and has identified a clear set of future improvements. The submission is therefore complete, evidence based, and ready to support the marker's assessment.

23. References

Microsoft, 2026. ASP.NET Core MVC documentation. Microsoft Learn. Available at: <https://learn.microsoft.com/aspnet/core/mvc> [Accessed 5 May 2026].

Microsoft, 2026. C# documentation. Microsoft Learn. Available at: <https://learn.microsoft.com/dotnet/csharp> [Accessed 5 May 2026].

Microsoft, 2026. ASP.NET Core Identity documentation. Microsoft Learn. Available at: <https://learn.microsoft.com/aspnet/core/security/authentication/identity> [Accessed 5 May 2026].

Microsoft, 2026. Entity Framework Core documentation. Microsoft Learn. Available at: <https://learn.microsoft.com/ef/core> [Accessed 5 May 2026].

Microsoft, 2026. ASP.NET Core security documentation. Microsoft Learn. Available at: <https://learn.microsoft.com/aspnet/core/security> [Accessed 5 May 2026].

Microsoft, 2026. ASP.NET Core rate limiting middleware. Microsoft Learn. Available at: <https://learn.microsoft.com/aspnet/core/performance/rate-limit> [Accessed 5 May 2026].

Pomelo Foundation, 2026. Pomelo Entity Framework Core MySQL provider. GitHub. Available at: <https://github.com/PomeloFoundation/Pomelo.EntityFrameworkCore.MySql> [Accessed 5 May 2026].

QuestPDF, 2026. QuestPDF documentation. QuestPDF.com. Available at: <https://www.questpdf.com/documentation/getting-started.html> [Accessed 5 May 2026].

Bootstrap, 2026. Bootstrap 5 documentation. getbootstrap.com. Available at: <https://getbootstrap.com/docs/5.3/> [Accessed 5 May 2026].

Open Web Application Security Project, 2026. OWASP Top 10. owasp.org. Available at: <https://owasp.org/www-project-top-ten/> [Accessed 5 May 2026].

Pressman, R. and Maxim, B., 2014. Software Engineering: A Practitioner's Approach. 8th ed. New York: McGraw-Hill.

Sommerville, I., 2015. Software Engineering. 10th ed. Harlow: Pearson Education.

24. Appendices

Appendix A: Selected Source Code Evidence

This appendix collects short, illustrative code snippets that support the body of the report. Each snippet identifies the file path so the marker can locate the full source.

A1. Identity configuration in Program.cs (excerpt):

```
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.Password.RequiredLength = 8;
    options.Password.RequireDigit = true;
    options.Password.RequireUppercase = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireNonAlphanumeric = false;
    options.User.RequireUniqueEmail = true;
    options.Lockout.AllowedForNewUsers = true;
    options.Lockout.MaxFailedAccessAttempts = 5;
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
}))
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();
```

A2. EF Core registration in Program.cs:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseMySQL(connectionString, new MySqlServerVersion(new Version(8, 0,
36)), mysql =>
        mysql.EnableRetryOnFailure(maxRetryCount: 5, maxRetryDelay:
TimeSpan.FromSeconds(30), errorNumbersToAdd: null)));
```

A3. Rate limiter registration in Program.cs (excerpt) is shown in Listing 8.1.

A4. Cookie configuration in Program.cs:

```
builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Account/Login";
    options.LogoutPath = "/Account/Logout";
    options.AccessDeniedPath = "/Account/AccessDenied";
    options.SlidingExpiration = true;
    options.ExpireTimeSpan = TimeSpan.FromHours(8);
    options.Cookie.HttpOnly = true;
    options.Cookie.SameSite = SameSiteMode.Lax;
    options.Cookie.SecurePolicy = CookieSecurePolicy.SameAsRequest;
});
```

A5. Other illustrative listings should be added during final Word formatting from the following files: Controllers/AccountController.cs, Controllers/EnrollmentsController.cs, Controllers/SubmissionsController.cs, Infrastructure/AuditLogger.cs, Infrastructure/UploadedFileStore.cs, Data/ApplicationDbContext.cs, Data/DbInitializer.cs. Keep each excerpt to ten to thirty lines.

Appendix B: Screenshots

This appendix lists the screenshot evidence that must be captured and inserted into the final Word document. The figure numbers match the body of the report.

Figure	Subject	Source View / Action	Status
Figure 7	Login Screen	Views/Account/Login.cshtml	[SCREENSHOT REQUIRED]
Figure 7b	Registration Screen	Views/Account/Register.cshtml	[SCREENSHOT REQUIRED]
Figure 7c	Invalid Login Validation Message	Login form with bad input	[SCREENSHOT REQUIRED]
Figure 7d	Successful Role-Based Redirect	URL bar after login	[SCREENSHOT REQUIRED]
Figure 8	Student Dashboard	Views/Dashboard/Student.cshtml	[SCREENSHOT REQUIRED]
Figure 8b	Student Calendar Panel	_DashboardCalendar.cshtml	[SCREENSHOT REQUIRED]
Figure 9	Lecturer Dashboard	Views/Dashboard/Lecturer.cshtml	[SCREENSHOT REQUIRED]
Figure 9b	Pending Submissions	Lecturer dashboard panel	[SCREENSHOT REQUIRED]
Figure 10	Administrator Dashboard	Views/Dashboard/Administrator.cshtml	[SCREENSHOT REQUIRED]
Figure 10b	Audit Log Review	Views/AuditLogs/Index.cshtml	[SCREENSHOT REQUIRED]
Figure 11	Course List	Views/Courses/Index.cshtml	[SCREENSHOT REQUIRED]
Figure 11b	Course Create Form	Views/Courses/Create.cshtml	[SCREENSHOT REQUIRED]
Figure 11c	Course Edit Form	Views/Courses/Edit.cshtml	[SCREENSHOT REQUIRED]
Figure 11d	Course Material Upload	Material upload modal/page	[SCREENSHOT REQUIRED]
Figure 12	Course Browse	Views/Enrollments/Browse.cshtml	[SCREENSHOT REQUIRED]
Figure 12b	Successful Enrolment	Confirmation alert	[SCREENSHOT REQUIRED]
Figure 12c	Duplicate / Capacity Rejection	Browse view error message	[SCREENSHOT REQUIRED]
Figure 13	Assignment List	Views/Assignments/Mine.cshtml	[SCREENSHOT REQUIRED]
Figure 13b	Assignment Create Form	Views/Assignments/Create.cshtml	[SCREENSHOT REQUIRED]
Figure 13c	Submission Form	Views/Submissions/Submit.cshtml	[SCREENSHOT REQUIRED]
Figure 13d	Grading Form	Views/Submissions/Grade.cshtml	[SCREENSHOT REQUIRED]
Figure 13e	Graded Submission View	Student-side view	[SCREENSHOT REQUIRED]
Figure 14	Reports Landing	Views/Reports/Index.cshtml	[SCREENSHOT REQUIRED]
Figure 14b	Sample Report	One report view	[SCREENSHOT

	Output		REQUIRED]
Figure 14c	CSV Export Evidence	Downloaded CSV in browser	[SCREENSHOT REQUIRED]
Figure 14d	PDF Export Evidence	Downloaded PDF	[SCREENSHOT REQUIRED]
Figure 15	Inbox	Views/Messages/Inbox.cshtml	[SCREENSHOT REQUIRED]
Figure 15b	Message Thread	Views/Messages/Thread.cshtml	[SCREENSHOT REQUIRED]
Figure 15c	Compose Message	Views/Messages/Compose.cshtml	[SCREENSHOT REQUIRED]
Figure 15d	Meetings List	Views/Meetings/Index.cshtml	[SCREENSHOT REQUIRED]
Figure 16	Validation Error	Any form with bad input	[SCREENSHOT REQUIRED]
Figure 16b	Friendly Error Page	Views/Shared/Error.cshtml	[SCREENSHOT REQUIRED]
Figure 17	Database Table View	MySQL client	[SCREENSHOT REQUIRED]
Figure 17b	Migration List Output	dotnet ef migrations list	[SCREENSHOT REQUIRED]
Figure 18	Source Repository	GitHub or other host	[SCREENSHOT REQUIRED]
Figure 19	Packaged Build / Deployment	ZIP location or hosted URL	[SCREENSHOT REQUIRED]
Figure 20	Visual Studio Solution Open	IDE screenshot	[SCREENSHOT REQUIRED]
Figure 20b	Successful Build Output	dotnet build console	[SCREENSHOT REQUIRED]

Appendix C: Diagram Sources

Diagram	Source File	Action
System Architecture	documentation_application_development/diagrams/architecture_diagram.mmd	Export to PNG and PDF, insert as Figure 1.
Use Case	documentation_application_development/diagrams/use_case_diagram.mmd	Export to PNG and PDF, insert as Figure 2.
Entity Relationship	documentation_application_development/diagrams/er_diagram.mmd	Export to PNG and PDF, insert as Figure 3.
UML Class	documentation_application_development/diagrams/class_diagram.mmd	Export to PNG and PDF, insert as Figure 4.
Application Flowchart	documentation_application_development/diagrams/application_flowchart.mmd	Export to PNG and PDF,

		insert as Figure 5.
Authentication and RBAC Flow	documentation_application_development/diagrams/security_flow_diagram.mmd	Export to PNG and PDF, insert as Figure 6.

Appendix D: Individual Approval and Planning Evidence

The author has prepared the documentation package as an approved individual submission. The supporting individual contribution document at documentation_application_development/Expanded_Individual_Contribution.md records the contribution, the timeline, and the workflow. Approval evidence (for example a screenshot of the lecturer's email or a note from the module leader) should be attached at documentation_application_development/evidence/approval/ if available. This item is currently [NEEDS CONFIRMATION].

Appendix E: Source Repository, Database and Build Evidence

The repository link, the packaged build link, and the database evidence are held in documentation_application_development/Source_And_Build_Links.md. The current detected repository remote is <https://github.com/YasasPasinduFernando/AD-COURSEWORK-2.git>. The current detected migrations are listed in Section 7.6. The build was verified locally with dotnet build --no-restore, returning zero warnings and zero errors.

Figure 18: [INSERT SCREENSHOT HERE]

This screenshot shows the public landing page of the project's source repository on GitHub.

Figure 19: [INSERT SCREENSHOT HERE]

This screenshot shows the packaged build (ZIP) or hosted deployment evidence available for the marker to download or visit.

Figure 17: [INSERT SCREENSHOT HERE]

This screenshot shows a database client connected to the seeded UniManage database, displaying the schema and a representative table such as Courses, Enrollments, or Submissions.

Figure 17b: [INSERT SCREENSHOT HERE]

This screenshot shows the console output of dotnet ef migrations list, confirming the three EF Core migrations.

Figure 20: [INSERT SCREENSHOT HERE]

This screenshot shows the Visual Studio 2022 solution open with the AD COURSEWORK 2 project loaded in Solution Explorer.

Figure 20b: [INSERT SCREENSHOT HERE]

This screenshot shows the console output of dotnet build --no-restore reporting zero warnings and zero errors.

Appendix E.4 Hosted Deployment Evidence

In addition to the local Visual Studio build, the application has been deployed to a Google Cloud virtual machine using a Docker container as supporting demonstration evidence. The current main hosted endpoint and the earlier IP-based endpoint are listed below.

Item	Value	Notes
Hosted application domain	https://lms.yasaboy.com/	Main hosted demonstration URL at the time of writing.
Previous IP-based endpoint	http://34.171.6.234:8080	Earlier Google Cloud hosted endpoint, retained as backup evidence.
Hosting platform	Google Cloud	Hosted on a Compute Engine virtual machine.
Containerisation	Docker	The application is packaged as a Docker image.
HTTPS access	Configured	HTTPS is available through the custom domain.
Custom domain	lms.yasaboy.com	Domain-based access configured.
Nginx reverse proxy	[VERIFY]	Include only if configuration or screenshot evidence is available.
Cloudflare or DNS provider	[VERIFY]	Include only if DNS or provider screenshot is available.
Deployment purpose	Supporting viva and demonstration evidence	Not a replacement for the local Visual Studio assessment route.
Visual Studio local assessment	Required	The coursework brief assesses the application using Visual Studio 2017 or higher.

Figure UM19: [INSERT SCREENSHOT HERE]

This screenshot shows the UniManage application running through the custom HTTPS domain.

Figure UM20: [INSERT SCREENSHOT HERE]

This screenshot shows the Google Cloud and Docker deployment evidence used to support the hosted demonstration.

Notes for the marker:

- HTTPS is now in place through the custom domain. Maintenance and renewal of the TLS configuration, plus documentation of any reverse proxy or DNS provider in use, are listed in Chapter 21 as future improvements.
- The local Visual Studio build remains the primary assessment route. The local build has been verified with `dotnet build --no-restore` returning zero warnings and zero errors.
- The author has confirmed that no database password, SMTP password, Google client secret, SSH key, DNS or provider credential, or cloud service account key is included in the body of the report or in the hosted screenshots.

Appendix F: User Manual Reference

The complete user manual is stored at `documentation_application_development/User_Manual_Application_Development.md`. The user manual covers system requirements, project package contents, installation, login, user roles, troubleshooting, reporting, and viva demonstration notes. It should be exported to Word and PDF and either embedded or linked in the final submission package.

Appendix G: Test Logs and Audit Evidence

The test plan in Chapter 15 lists the test cases. Each test case should produce a screenshot or, where appropriate, an audit log entry. The audit log should be exported through the Administrator audit log view and included as evidence. The build console output should be saved as plain text and stored at `documentation_application_development/evidence/build/`. The audit log export and the saved build console output are listed as outstanding manual evidence items in the QA report.

Appendix H: Validation Message Evidence

The validation rules used by each form are listed in Table 16. Selected examples of validation messages should be captured as screenshots and included here, including:

- Required field missing on registration.
- Password too weak on registration.
- Wrong credentials on login.
- Course code already in use.
- Enrolment limit reached.
- File too large or wrong type on submission upload.

Each of the validation messages above must be captured as a screenshot before final submission.

Appendix I: Configuration Redaction Notes

The author has redacted sensitive configuration values from the report. The following keys must not appear in screenshots, source code listings, or appendices:

- `ConnectionStrings:DefaultConnection` (database password and host).
- `Authentication:Google:ClientSecret`.
- `Email:Password` (SMTP password).

If a screenshot accidentally captures one of these values, the screenshot must be retaken or redacted before submission. The author has avoided pasting appsettings.json or appsettings.Development.json into the report for this reason.

Appendix J: Viva Demonstration Script

The following script supports the viva demonstration. The author should be ready to perform the steps in the order listed.

30. Open Visual Studio and the AD COURSEWORK 2.sln solution. Highlight the folder structure.
31. Run dotnet build --no-restore from a terminal and show the zero warning, zero error result.
32. Start the application through Visual Studio (F5) or dotnet run. Show the seeded URL.
33. Sign in as the seeded Administrator (admin@unimanager.local / Admin123!) and demonstrate the Administrator dashboard, the course list, the reports, and the audit log.
34. Sign out and sign in as the seeded Lecturer (lecturer@unimanager.local / Lecturer123!). Demonstrate the Lecturer dashboard, MyCourses, an assignment, a grading screen, and a meeting.
35. Sign out and sign in as the seeded Student (student@unimanager.local / Student123!). Demonstrate the Student dashboard, course browse, enrolment, submission, grade view, and inbox.
36. Trigger a validation error (for example a missing required field) and show the friendly message.
37. Show the audit log entry that corresponds to a recent action (for example a login or an enrolment).
38. Briefly walk through Program.cs, focusing on Identity, EF Core, rate limiter, anti-forgery, security headers, and routing.
39. Highlight the items that remain [NEEDS CONFIRMATION] honestly.

End of report.

Supporting Document: Individual Contribution Report

Student Name: Yasas Pasindu Fernando

Student ID: 25026764

Module: CS6004ES Application Development

Assessment: Coursework 2

Project: UniManage - University Course Management System (UCMS)

Centre: ESOF Metro Campus / London Metropolitan University

Submission Type: Approved individual submission

Companion Document: Main_Application_Development_Report.md

1. Introduction

This supporting document records the author's individual contribution to the final UniManage coursework submission. It complements the main technical report by focusing specifically on the work that the author has performed, the decisions taken, the evidence gathered, and the personal learning experienced during the project. The document is written as an approved individual submission and uses neutral, evidence-based wording throughout.

The contribution report has been organised so that the marker can move quickly from one academic area to the next. Each numbered section addresses a specific area of the project. Two consolidated tables, Table IC1 and Table IC2, summarise the contribution and the supporting evidence so that the marker has both a high-level and a section-by-section view.

The report avoids any group blame, group conflict, or comparative wording. It does not include a group contribution table or a work-splitting table. It focuses on the work that the author has produced and on how that work has been verified through source code, build evidence, and prepared screenshots.

2. Project Context

UniManage is a University Course Management System implemented as an ASP.NET Core MVC web application targeting .NET 8. It uses C# for application logic, Razor views and Bootstrap for the user interface, Entity Framework Core for data access through the Pomelo MySQL provider, and ASP.NET Core Identity for authentication and roles. The system supports academic workflows for Student, Lecturer, and Administrator users, including

dashboards, course management, enrolment, assignments, submissions, grading, reporting, internal messaging, meeting scheduling, profile management, and audit logging.

The CS6004ES Application Development brief sets a structured list of marking items that includes the application user interface, registration and login, role dashboards, course management, the enrolment system, assignment and grading, reporting and analytics, the communication module, security and data protection, validation and exception handling, programming style, the installation guide and user manual, the concise logical solution, the architecture diagrams, the detailed description of classes and methods, and the personal reflection. The author has aligned the implementation and the documentation with these items in a way that is intended to be straightforward to mark.

The technical baseline is recorded in the main report. Twelve controllers, twelve domain models, sixteen view models, twelve infrastructure components, forty-six Razor views, three Entity Framework Core migrations, and a Program.cs startup file have been confirmed by source-code review. The build has been verified locally with `dotnet build --no-restore`, returning zero warnings and zero errors.

3. Approved Individual Submission Context

The CS6004ES Application Development Coursework 2 was originally set as group work. Following approval to complete the coursework individually, the author took responsibility for the final implementation, testing, documentation, and submission preparation. The author refers to themselves as "the author" throughout this contribution report. The author does not name other students, does not describe any conflict or disagreement, and does not include any work-splitting table.

The approved individual submission context has shaped the structure of this contribution report in two ways. First, every academic area is described as the author's own work because that is what has actually been delivered and verified. Second, the report makes a clear distinction between work that is provable from the source code and work that requires manual evidence such as screenshots and exported diagrams. Items that cannot be confirmed at the time of writing are honestly marked [VERIFY] or [NEEDS CONFIRMATION].

4. Personal Motivation and Project Selection

The author chose to complete UniManage as the coursework project for several reasons. First, a university course management system is a realistic academic problem that combines

authentication, role-based access, structured database relationships, validation, reporting, and user interface design in a single application. Second, the problem aligns naturally with the marking items in the brief, which makes it easier to demonstrate coverage of each marking area through source-code evidence. Third, the technologies selected for the project, namely ASP.NET Core MVC, C#, Razor, Entity Framework Core, MySQL, and ASP.NET Core Identity, are widely used in industry and are therefore valuable to the author's professional development beyond the coursework.

Personal motivation also played a part. The author has been interested in modern .NET web development since the start of the module and chose UniManage because it allowed the author to apply that interest to a focused academic problem. The approved individual completion route reinforced the author's commitment to deliver a complete, evidence-based submission rather than a partial implementation.

5. Summary of Main Contribution

The author's contribution covers the full life cycle of the project. This includes the requirement analysis, the architectural design, the database design, the implementation of every feature, the validation and security pipeline, the user interface, the testing, the evidence preparation, and the documentation. Table IC1 summarises the contribution by area, with cross-references to the relevant figures in the main report and a status flag for marker convenience.

Table IC1: Summary of Individual Contribution

Area	Contribution Summary	Evidence / Figure Reference	Status
Requirement analysis	Mapped 30 functional requirements (FR1 to FR30) and 10 non-functional requirements (NFR1 to NFR10) to source-code evidence and to marking items.	Tables 1, 2, 7, 11, 14, 15 in the main report	Confirmed
Architecture and system design	Prepared six diagrams (architecture, use case, ERD, UML class, application flowchart, authentication flow) as Mermaid sources and documented design decisions.	Figures 1 to 6, Section 6 of the main report, Appendix C	Sources prepared, exports pending
Database design	Documented EF Core configuration, nine academic entities, relationships, unique indexes, and three migrations.	Figure 3, Table 3, Section 7 of the main report	Confirmed
User registration and login	Documented AccountController with Register, Login, Logout, ForgotPassword, ResetPassword, and optional Google login support.	Figures 7, 7b, 7c, 7d, Section 8.3 of the main report	Confirmed; screenshots pending
Role-based	Documented AppRoles, role	Section 8.4 of the main report, Table	Confirmed

access control	seeding through DbInitializer, [Authorize] attributes, and ownership filters.	17	
Student dashboard	Documented DashboardController.Student, StudentDashboardViewModel, and Views/Dashboard/Student.cshtml.	Figures 8, 8b, Section 8.5 of the main report	Confirmed; screenshot pending
Lecturer dashboard	Documented DashboardController.Lecturer, LecturerDashboardViewModel, and Views/Dashboard/Lecturer.cshtml.	Figures 9, 9b, Section 8.6 of the main report	Confirmed; screenshot pending
Administrator dashboard	Documented DashboardController.Administrator, AdminDashboardViewModel, and Views/Dashboard/Administrator.cshtml.	Figures 10, 10b, Section 8.7 of the main report	Confirmed; user-management screen [NEEDS CONFIRMATION]
Course management	Documented CoursesController with Index, MyCourses, Create, Edit, Delete, UploadMaterial, search, paging, and ownership filters.	Figures 11, 11b, 11c, 11d, Section 8.8 of the main report	Confirmed; screenshots pending
Enrolment system	Documented EnrollmentsController with Browse and Enroll actions, capacity, duplicate, and prerequisite checks.	Figures 12, 12b, 12c, Section 8.9 of the main report	Confirmed; screenshots pending
Assignment, submission and grading	Documented AssignmentsController, SubmissionsController, ownership checks, file validation, grade range, feedback, and graded lock.	Figures 13, 13b, 13c, 13d, 13e, Sections 8.10 and 8.11 of the main report	Confirmed; screenshots pending
Reporting and analytics	Documented ReportsController with six reports plus CSV and PDF export through QuestPDF.	Figures 14, 14b, 14c, 14d, Section 8.12 of the main report	Confirmed; export evidence pending
Communication and meetings	Documented MessagesController, allowed-recipient validation, MeetingsController, ICS export.	Figures 15, 15b, 15c, 15d, Sections 8.13 and 8.14 of the main report	Confirmed; screenshots pending
Security, validation and error handling	Documented Identity options, logout, anti-forgery, rate limiter, security headers middleware, exception handling, validation rules.	Figures 16, 16b, Sections 12 and 13 of the main report, Tables 16, 17	Confirmed; validation screenshot pending
UI/UX and usability	Documented layout, alerts partial, role-aware navigation, validation feedback, responsive design, accessibility considerations.	Section 9 of the main report	Confirmed; layout screenshots pending
Testing and debugging	Prepared test plan with 32 cases (T1 to T32), build verification, manual functional, validation, role, and security testing.	Section 15 of the main report, Table 8	Build confirmed; remaining cases pending screenshots
Documentation and user manual	Prepared the main report (24,000+ words), this contribution report, the user manual, evidence checklists, source/build links, diagram README, screenshot README,	documentation_application_development/ folder	Confirmed

	evidence README, conversion guide, and final QA reports.		
Evidence preparation and viva readiness	Built screenshot, diagram, source, build, and database evidence checklists; prepared a viva demonstration script.	Appendix B, Appendix J of the main report	Sources prepared; manual capture pending

6. Contribution to Requirement Analysis

The author derived the requirements from three sources. The first source is the coursework brief, which sets the marking items for CS6004ES Application Development Coursework 2. The second source is a structured review of the source-code repository, which confirms what is actually implemented. The third source is the case-study analysis of an academic course management problem, which identifies the practical needs of Students, Lecturers, and Administrators.

The author produced thirty functional requirements (FR1 to FR30) covering registration, login, logout, role-based redirection, dashboards, course management with paging and search, lecturer ownership, course material upload, enrolment with capacity, duplicate, and prerequisite checks, assignment management, submission upload, grading with feedback, secure file download, six reports, CSV and PDF export, internal messaging, allowed-recipient validation, meeting scheduling, ICS export, profile and password management, audit logging, friendly error pages, rate limiting, and anti-forgery protection. The author also produced ten non-functional requirements (NFR1 to NFR10) covering usability, performance, security, maintainability, reliability, data integrity, accessibility, auditability, configurability, and deployability.

The author then mapped the requirements onto the marking items through Table 7 (Coursework Marking Alignment) and Table 15 (Requirement Traceability Summary) in the main report. This traceability is intended to make the marker's job easier, because every marking item can be traced to specific requirements, source files, and test cases.

7. Contribution to Architecture and System Design

The author prepared the system architecture, the use case model, the entity relationship diagram, the UML class diagram, the application flowchart, and the authentication and role-based access flow. Each diagram is stored as a Mermaid source file in `documentation_application_development/diagrams/`. The author produced the diagrams from the actual source code rather than from an abstract design, which keeps the diagrams faithful to the implementation.

The architectural narrative explains how the request flows through the middleware pipeline (HTTPS redirection, static files, security headers, routing, rate limiting, authentication, authorisation), through the controller, through Entity Framework Core, and finally to the MySQL database. The narrative also identifies cross-cutting concerns such as audit logging, validation, and exception handling. Design decisions are documented in Section 6.7 of the main report, including the choice of MVC over Razor Pages, the choice of Pomelo MySQL over SQL Server / LocalDB, the choice of a single DashboardController rather than three role-specific controllers, the decision to store grade fields directly in Submission rather than in a separate Grade entity, the decision to add a custom security headers middleware, and the choice of QuestPDF for PDF reports.

8. Contribution to Database Design

The author documented the database design from `Data/ApplicationDbContext.cs`, the model classes in `Models/`, and the three EF Core migrations under `Data/Migrations/`. The contribution includes a clear list of nine academic entities (Table 3), a relationship narrative covering the lecturer-course link, the optional self-reference for prerequisites, the unique student-course enrolment, the assignment-course relationship, the submission-assignment-student-grader relationship, the message sender and receiver relationships, the meeting-course-lecturer relationships, and the optional audit log user reference.

The author also documented the constraints used in the database, including the unique index on `Course.Code`, the unique index on `Enrollment(StudentId, CourseId)`, the maximum-length attributes on string fields, the required attributes on key fields, and the range constraints on numeric fields such as `MaxPoints` on assignments and the grade on submissions. The seeding logic in `DbInitializer` is documented because it provides the reproducible demonstration data needed for the viva.

The author has clearly recorded that UniManage uses MySQL through the Pomelo provider rather than SQL Server / LocalDB. Where the brief refers to SQL Server / LocalDB, the author has documented the alternative approach in honest terms instead of overclaiming.

9. Contribution to User Registration and Login

The author documented the registration, login, logout, password reset, and optional Google login flows implemented in `Controllers/AccountController.cs`. The contribution links each action to its view, view model, and Identity API. The author has also documented the security

configuration in Program.cs that supports the flow, including the strong password rules (minimum eight characters, at least one digit, at least one uppercase letter, at least one lowercase letter), the unique email constraint, the lockout policy (five failed attempts, fifteen minute lockout), and the application cookie configuration (HTTP-only, SameSiteMode.Lax, secure when the request is secure, sliding expiration of eight hours).

The author has identified the audit log entries written by the registration and login flows. Successful registrations write a positive audit entry, failed registrations record the validation failure, successful logins write a positive audit entry, and failed logins write a negative audit entry. This combination supports the marker's review by providing a concrete record of who registered or signed in, when, and whether the action succeeded.

10. Contribution to Role-Based Access Control

The author documented the three-layer approach to role-based access control. First, Models/AppRoles.cs defines the role name constants (Administrator, Lecturer, Student). Second, Data/DbInitializer.cs seeds the three roles and creates the three demonstration users. Third, controller actions use [Authorize(Roles = AppRoles.X)] to restrict access to the appropriate role. Where the role check is not enough, controller actions add an ownership filter through `LecturerId == userId` or `StudentId == userId` so that a lecturer cannot see another lecturer's data and a student cannot see another student's data.

The author has also documented the Views/Account/AccessDenied.cshtml page, which presents a friendly response when a user tries to open an action they are not authorised to use. The combination of role attributes, ownership filters, and the access denied page provides defence in depth at the application layer.

11. Contribution to Student Dashboard

The author documented the student dashboard implementation through DashboardController.Student, the StudentDashboardViewModel defined in ViewModels/DashboardViewModels.cs, and Views/Dashboard/Student.cshtml. The dashboard view model exposes enrolled courses, upcoming deadlines, recent grades, recent messages, calendar entries, and meetings. EF Core read queries use `AsNoTracking` where possible to reduce the change tracker overhead, and queries are scoped to the signed-in student's user identifier so that no cross-account data is exposed.

The author has identified the supporting partial Views/Shared/_DashboardCalendar.cshtml, which renders a small calendar block reused by the dashboards. This partial is one example of the maintainability practices documented in Section 14 of the main report.

12. Contribution to Lecturer Dashboard

The author documented the lecturer dashboard implementation through DashboardController.Lecturer, the LecturerDashboardViewModel, and Views/Dashboard/Lecturer.cshtml. The lecturer dashboard view model exposes the courses owned by the signed-in lecturer, enrolment counts, assignment counts, pending submissions, recently graded submissions, recent messages, recent meetings, and recent audit activity relevant to the lecturer's work. The author has identified that the dashboard uses ownership filters to ensure that the lecturer only sees their own data and is therefore consistent with the role-based access control approach documented in Section 10 of this contribution report.

13. Contribution to Administrator Dashboard

The author documented the administrator dashboard implementation through DashboardController.Administrator, the AdminDashboardViewModel, and Views/Dashboard/Administrator.cshtml. The administrator dashboard exposes counts for users, courses, enrolments, assignments, submissions, and messages, identifies popular courses by enrolment count, and presents recent audit log entries. Where a dedicated user-management screen is referenced in the brief, the author has not confirmed one in the source code; this is recorded as [NEEDS CONFIRMATION] in the main report and in this contribution report so that the marker has an honest record.

14. Contribution to Course Management

The author documented the course management implementation through Controllers/CoursesController.cs, the Course entity, the CourseInputViewModel, the MaterialUploadViewModel, and the Views/Courses/ folder. The actions documented include Index with search and paging, Details, Create, Edit, Delete, MyCourses, and UploadMaterial. The author has documented the unique course code constraint at the database level, the prerequisite handling, the lecturer assignment, the capacity limit, and the file upload pipeline through Infrastructure/UploadedFileStore.cs.

The author has also documented that the upload action is gated by the upload rate-limiting policy configured in Program.cs, which permits up to thirty uploads per minute per user. This protects the file store against abuse without preventing reasonable academic use.

15. Contribution to Enrolment System

The author documented the enrolment workflow in Controllers/EnrollmentsController.cs, including the Browse action that lists eligible courses for the signed-in student and the Enroll action that creates an enrolment record after the business rules have been checked. The business rules are: the course exists, the course is open (capacity has not been reached), the student has not already enrolled, and any prerequisite has been satisfied. The combination of the unique index on Enrollment(StudentId, CourseId) and the application-level duplicate check prevents duplicates by defence in depth.

The author has produced eligibility logic that uses a single composed LINQ query rather than multiple queries to keep the database round-trip count low. The author has also documented the audit log entries that record successful and unsuccessful enrolment attempts.

16. Contribution to Assignment, Submission and Grading

The author documented the assignment lifecycle in Controllers/AssignmentsController.cs, including ForCourse, Mine, Create, Edit, and Delete, with ownership checks that ensure a lecturer can only manage assignments for their own courses. The submission lifecycle is documented in Controllers/SubmissionsController.cs, including Submit, Grade, ForAssignment, CourseMaterialFile, and SubmissionFile. The author has documented the file validation through Infrastructure/UploadedFileStore.cs, including extension allow-listing, size limits, and stored file naming to prevent path traversal.

The grading workflow is documented through GradeSubmissionViewModel, the score range validation, the feedback length validation, and the graded lock that prevents a student from modifying a submission after it has been graded. Each grading action writes an audit log entry that records the lecturer, the submission, and the grade.

17. Contribution to Reporting and Analytics

The author documented the reporting suite in Controllers/ReportsController.cs, including six reports: course popularity, student performance, lecturer workload, enrolments, pass and fail, and assignment attendance. Each report uses an EF Core query with GroupBy and aggregate functions and projects into a report view model. CSV export is provided through

Infrastructure/CsvWriter.cs, which writes properly escaped CSV content. PDF export is provided through Infrastructure/PdfReport.cs, which uses QuestPDF to render the report layout. The author has identified that the QuestPDF licence type is set to Community in Program.cs, which keeps the project within the QuestPDF licensing terms.

18. Contribution to Communication and Meetings

The author documented the messaging implementation in Controllers/MessagesController.cs, including Inbox, Thread, Reply, Compose, and MarkAllRead. The allowed-recipient rule is documented: a Student may message a Lecturer who teaches a course on which the Student is enrolled, and a Lecturer may message a Student enrolled on an owned course. The author has identified that this rule is implemented at the database level through a join during the recipient look-up, so the user interface does not offer an unauthorised recipient.

The author has also documented the meeting implementation in Controllers/MeetingsController.cs, including Index, Create, Edit, Delete, Join, and Ics. The ICS export is provided through Infrastructure/CalendarLink.cs, which produces an iCalendar file for import into a personal calendar tool. The meeting URL is stored as part of the Meeting entity rather than being provisioned by the application; this is consistent with the documented scope.

19. Contribution to Security, Validation and Error Handling

The author documented the security pipeline in detail. Identity uses strong password rules and logout. Anti-forgery is enabled globally through AddAntiforgery in Program.cs, with the cookie configured as HTTP-only, SameSiteMode.Strict, and secure when the request is secure. Rate limiting is configured with two named policies, auth (ten requests per minute per IP) and upload (thirty requests per minute per user). The custom Infrastructure/SecurityHeadersMiddleware.cs adds standard headers to every response. HTTPS redirection and HSTS are enabled. The application cookie is HTTP-only with SameSiteMode.Lax and secure when the request is secure.

Validation is documented through data annotations on view models, ModelState checks in controller actions, server-side ownership checks where role membership is not enough, and file upload validation through UploadedFileStore. Error handling is documented through the production exception handler app.UseExceptionHandler("/Home/Error"), the developer

exception page in development, and the friendly Views/Shared/Error.cshtml page. The author has also documented the audit logger as a key part of the error and event-tracking story.

20. Contribution to UI/UX and Usability

The author documented the user interface design through Views/Shared/_Layout.cshtml, Views/Shared/_Alerts.cshtml, Views/Shared/_DashboardCalendar.cshtml, the role-specific dashboard views, the form views, and the static assets in wwwroot/css/site.css and wwwroot/js/site.js. The author has documented the role-aware navigation that adapts the menu items to the signed-in user's role, the alert partial that displays TempData feedback, the responsive Bootstrap grid that supports narrower viewports, the skip link near the top of the layout that helps keyboard users, and the label association used throughout the form views to support assistive technology. A formal WCAG 2.1 AA audit has not been carried out and is recorded as [VERIFY].

21. Contribution to Testing and Debugging

The author prepared a manual testing strategy with thirty-two test cases (T1 to T32) recorded in Section 15.9 of the main report. The cases cover registration, login, logout, password reset, role dashboards, role-aware navigation, course list, course create, course code uniqueness, course delete, enrolment success and rejection, assignment create, ownership rejection, submission upload, grading, reporting, CSV export, messaging compose, messaging restriction, anti-forgery, rate limit, access denied, validation message, friendly error, responsive layout, and build verification. The build verification (T32) is confirmed by dotnet build --no-restore returning zero warnings and zero errors. The remaining cases require screenshots to be captured during the final preparation week.

Debugging during development included resolving transient MySQL connection errors at startup by enabling EF Core retry on failure, splitting a single rate limiter policy into two named policies (auth and upload) to avoid spurious 429 responses on dashboard pages, ordering the migrations so that the audit table exists before any audit calls run, and wrapping the audit write in a try/catch block so that an audit failure cannot crash the calling action. Each problem and its resolution is documented in Section 17.1 of the main report.

22. Contribution to Documentation and User Manual

The author prepared the documentation package at documentation_application_development/. The package contains the main technical report at

Main_Application_Development_Report.md, this contribution report at Expanded_Individual_Contribution.md, the installation guide and user manual at User_Manual_Application_Development.md, the evidence checklist at Evidence_Checklist.md, the source and build links file at Source_And_Build_Links.md, the diagram readme at diagrams/README_DIAGRAMS.md, the screenshot readme at screenshots/README_SCREENSHOTS.md, the evidence readme at evidence/README_EVIDENCE.md, the Word formatting guide at conversion/WORD_FORMATTING_GUIDE.txt, the Word conversion script at conversion/convert_to_docx.py, and a number of final QA artefacts under final_document/, including the documentation expansion plan, the lecturer marking alignment report, the manual final checklist, the individual wording cleanup report, and the final QA report.

The author has applied a consistent style across these documents, namely UK English spelling, "the author" voice in the body of the technical report, first person voice limited to the reflection chapter, no em dashes, no strange symbols, and no group blame.

23. Contribution to Evidence Preparation and Viva Readiness

The author prepared the evidence package by listing every screenshot, every diagram export, every source-code listing, every database evidence item, every build evidence item, and every viva preparation item. The list is held in three places. First, the main report's Appendix B lists thirty-eight screenshot placeholders with the corresponding view file. Second, the main report's Appendix C lists six diagram exports. Third, the evidence checklist tracks the status of each item.

The author has also prepared the viva demonstration script in Appendix J of the main report. The script begins with the solution open in Visual Studio, includes a build verification, demonstrates each role using the seeded demonstration users, triggers a validation error to show the friendly response, presents a corresponding audit log entry, and ends with a summary of items that remain [NEEDS CONFIRMATION] honestly.

The seeded demonstration users that support the viva are admin@unimanager.local (Administrator), lecturer@unimanager.local (Lecturer), and student@unimanager.local (Student). The seeded passwords are documented in Data/DbInitializer.cs and in the user manual.

24. Challenges, Learning and Final Preparation

The author faced a small set of recurrent challenges during the project. Aligning the source-code review with the coursework wording required care because the project uses Pomelo MySQL rather than SQL Server / LocalDB. The author resolved this by recording the implementation honestly and by explaining the alternative approach in the main report. Capturing evidence in a marker-friendly way required discipline because screenshots need to be redacted and named consistently. The author resolved this by preparing a screenshot README and by listing the figures in Appendix B. Maintaining a consistent voice across the documents required attention because the report mixes a third-person body with a first-person reflection. The author resolved this by reviewing each chapter against the wording rules in the documentation expansion plan.

The learning from this project has been substantial. On the technical side, the author has gained experience with MVC routing, Razor views, EF Core with the Pomelo provider, Identity, anti-forgery, rate limiting, custom middleware, dependency injection, asynchronous programming, and PDF generation through QuestPDF. On the professional side, the author has gained experience with evidence-based academic writing, traceability between requirements and implementation, and the discipline of marking unverifiable claims rather than embellishing them. On the personal side, the author has gained confidence in completing a substantial individual delivery within an academic deadline.

The final preparation stage focuses on capturing screenshots, exporting Mermaid diagrams, completing the title page placeholders, completing the references, pasting the final repository and packaged build links, and converting the report into Word and PDF for upload.

25. Supporting Evidence

Table IC2 lists the evidence prepared for assessment, the purpose of each item, the location, and the manual action that is still required.

Table IC2: Evidence Prepared for Assessment

Evidence Type	Purpose	Location	Manual Action Needed
Source code	Confirms implemented features and	Project root, including Controllers/, Models/, ViewModels/, Views/, Data/, Infrastructure/, wwwroot/, Program.cs, AD COURSEWORK 2.sln, AD COURSEWORK 2.csproj	None for source itself; ensure final commit before submission

	support source-code traceability		
Build verification	Confirms that the project compiles cleanly	dotnet build --no-restore console output (zero warnings, zero errors)	Capture build console as plain text into documentation_application_development/evidence/build/
Architecture and design diagrams	Supports Section 6 of the main report	documentation_application_development/diagrams/*.md	Export each diagram to PNG and PDF, save into documentation_application_development/diagrams/exported/
Screenshots	Supports figures referenced from the main report and the user manual	documentation_application_development/screenshots/	Capture every screenshot listed in Appendix B of the main report and Section 7-9 of the user manual, redact personal data
Evidence checklist	Tracks the status of every required evidence item	documentation_application_development/Evidence_Checklist.md	Update each row as evidence is captured
Source and build links	Records repository, build, and database evidence URLs	documentation_application_development/Source_And_Build_Links.md	Paste final repository link, packaged build link, and database evidence link
User manual	Supports the	documentation_application_development/User_Manual_Application_Development.md	Convert to Word and PDF; insert UM screenshots

1	install ation guide and user manua l markin g item		
Evide nce checkl ist for figure s	Tracks figure- by- figure captur e	Appendix B of main report	Replace [SCREENSHOT REQUIRED] with the actual figure once captured
Datab ase eviden ce	Suppo rts the databa se design markin g item	documentation_application_development/evidence/datab ase/ (planned)	Capture migration list output and a redacted view of one or two tables
Audit log eviden ce	Suppo rts the securit y and audit tracea bility story	Audit log review screen Views/AuditLogs/Index.cshtml	Capture an audit log export screenshot
Valida tion messa ge eviden ce	Suppo rts the validat ion and except ion handli ng markin g item	Each form view	Capture screenshots of representative validation messages
Indivi dual appro val eviden ce	Suppo rts the approv ed individ ual submis sion context	documentation_application_development/evidence/appr oval/ (planned)	Attach approval email or note if available; otherwise leave [VERIFY]
Repos itory link	Suppo rts submis sion packag ing	documentation_application_development/Source_And_ Build_Links.md	Paste final repository link
Packa ged	Suppo rts	documentation_application_development/Source_And_ Build_Links.md	Paste packaged build (ZIP) or

build / hosted deploy ment	submis sion packag ing	Build_Links.md	hosted deployment link
Final Word and PDF report	Suppo rts the formal report submis sion	documentation_application_development/final_documen t/Application_Development_Final_Report.docx (and PDF)	Re-export from Markdown after evidence is inserted

26. Summary

The author has completed UniManage as an approved individual submission for CS6004ES Application Development Coursework 2. The contribution covers requirement analysis, architecture and design, database design, implementation of every feature, validation and security, user interface and usability, manual testing and debugging, documentation, and evidence preparation. The technical baseline is a working ASP.NET Core MVC 8 application with twelve controllers, twelve domain models, sixteen view models, twelve infrastructure components, forty-six Razor views, three EF Core migrations, and a complete Program.cs startup configuration that enables Identity, EF Core retry on failure, rate limiting, anti-forgery, security headers, HTTPS redirection, and HSTS.

Following approval to complete the coursework individually, the author took responsibility for the final implementation, testing, documentation, and submission preparation. Items that remain to be completed before the final upload are the screenshot capture, the diagram export, the title page placeholders, the references date placeholders, the repository and build links, and the conversion of the Markdown documents to Word and PDF. The current status of each item is recorded in Table IC2 above and in the evidence checklist.

End of individual contribution report.

Installation Guide and User Manual

Project: UniManage - University Course Management System (UCMS)

Module: CS6004ES Application Development

Assessment: Coursework 2

Student: Yasas Pasindu Fernando

Student ID: 25026764

Centre: ESOF Metro Campus / London Metropolitan University

Submission Type: Approved individual submission

Companion Documents: Main_Application_Development_Report.md,
Expanded_Individual_Contribution.md

1. Introduction

This document is the installation guide and user manual for UniManage, the University Course Management System developed for CS6004ES Application Development Coursework 2. The manual is intended for the marker, the external moderator, and any reader who needs to install, run, demonstrate, or operate the application. The instructions are based on the actual project files and have been verified locally during the development of the project.

The manual is organised into fourteen sections. Sections 1 to 5 explain the system requirements, the project package contents, the installation steps, and the first-time setup. Sections 6 to 9 explain the login process and the role-by-role user guides for Student, Lecturer, and Administrator. Section 10 lists the validation and error messages the user can expect to see. Section 11 explains the reporting and analytics screens. Section 12 contains security notes for users. Section 13 contains a troubleshooting table. Section 14 contains the viva demonstration script.

Following approval to complete the coursework individually, the author took responsibility for the final implementation, testing, documentation, and submission preparation. The manual is written in UK English, uses a practical step-by-step style, and refers to "the user" or to a specific role rather than to any other student.

2. System Requirements

The following items are required to install and run UniManage on a development workstation.

Item	Requirement	Notes
Operating system	Windows 10 or later	The development environment is Windows; macOS and Linux can run the application using the .NET 8 SDK and Pomelo MySQL provider, but Visual Studio integration is best on Windows.
Development environment	Visual Studio 2022 (Community edition is sufficient)	Older Visual Studio versions are not supported because the project targets .NET 8.
.NET runtime	.NET 8 SDK	Required for dotnet build, dotnet run, and the Visual Studio project.
Database engine	MySQL Server 8.x	The project uses the Pomelo Entity Framework Core MySQL provider.
Database tool	Optional MySQL Workbench, HeidiSQL, or DBeaver	Used to inspect tables and capture database evidence.
Browser	Microsoft Edge, Google Chrome, or Mozilla Firefox (latest stable)	Used to access the running application.
Internet connection	Required for the first run	Used to restore NuGet packages, optional Google login, optional SMTP email, and the small number of CDN-hosted assets.
Disk space	About 1 GB free	Project source, NuGet cache, and database storage.

If the marker uses SQL Server / LocalDB instead of MySQL, the connection string and the EF Core provider must be replaced. The current implementation uses MySQL through Pomelo and is therefore the supported configuration. SQL Server / LocalDB compatibility is [VERIFY].

3. Project Package Contents

The project package contains the following items.

Item	Path	Purpose
Solution file	AD COURSEWORK 2.sln	Visual Studio solution file.
Project file	AD COURSEWORK 2.csproj	Web project file with package references.
Application entry point	Program.cs	Service registration, middleware pipeline, and seeding.
Controllers	Controllers/	Twelve controllers handling the request flow.
Domain models	Models/	Twelve model classes for academic entities.
View models	ViewModels/	Sixteen view models for forms and dashboards.

Razor views	Views/	Forty-six .cshtml views including shared layouts and partials.
Data access	Data/	ApplicationDbContext, factory, seeder, and EF Core migrations.
Infrastructure	Infrastructure/	Twelve infrastructure files: audit logger, email service, file store, security headers middleware, CSV writer, PDF report builder, calendar link helper, meeting link helper, and supporting interfaces.
Static assets	wwwroot/	Bootstrap, custom CSS, custom JavaScript, library folder, and images.
Configuration	appsettings.json, appsettings.Development.json	Connection string, email settings, optional Google credentials.
Launch settings	Properties/launchSettings.json	HTTP and HTTPS launch profiles for development.
Documentation	documentation_application_development/	Main report, individual contribution, this manual, evidence checklist, source and build links, diagrams, screenshots README, and final QA artefacts.
Project README	README.md	Top-level project description.

4. Installation Guide

The installation guide assumes that the marker has Windows, Visual Studio 2022, the .NET 8 SDK, and a running MySQL 8 server. Each step has been kept short and practical so that the marker can complete the setup without unnecessary detours.

4.1 Extracting the Project

If the project has been delivered as a ZIP archive, extract the archive into a short local path such as C:\src\unimanage or C:\workspace\AD COURSEWORK 2. Avoid placing the project under a deep path with spaces and special characters because some EF Core tooling commands behave better with shorter paths.

If the project has been delivered as a Git repository, clone the repository into a similar local path:

```
git clone https://github.com/YasasPasinduFernando/AD-COURSEWORK-2.git
```

After extraction or cloning, the local folder should contain AD COURSEWORK 2.sln, AD COURSEWORK 2.csproj, Program.cs, the source folders listed in Section 3, and the documentation_application_development folder.

Figure UM1: [INSERT SCREENSHOT HERE]

This screenshot shows the solution loaded in Visual Studio with the source folders visible in Solution Explorer.

4.2 Opening the Solution in Visual Studio

Double-click AD COURSEWORK 2.sln to open the solution in Visual Studio 2022. Confirm that the project loads without missing project errors. The status bar should report that the solution is ready and the Solution Explorer should display the AD COURSEWORK 2 web project together with the documentation_application_development folder if it is included in the solution.

If Visual Studio prompts to install missing components, accept the prompts so that the .NET 8 SDK and the ASP.NET Core workload are present. If the prompt suggests SQL Server LocalDB, that component is not required because the project uses MySQL.

4.3 Restoring NuGet Packages

NuGet packages are restored automatically the first time Visual Studio opens the solution. To restore manually from the command line, open a PowerShell window in the project folder and run:

```
dotnet restore
```

The main packages used by the project include:

- Microsoft.AspNetCore.Identity.EntityFrameworkCore
- Microsoft.EntityFrameworkCore and Microsoft.EntityFrameworkCore.Design
- Pomelo.EntityFrameworkCore.MySql
- Microsoft.AspNetCore.Authentication.Google
- QuestPDF

If a restore failure occurs, confirm that NuGet.org is reachable and that the workstation has internet access. The exact pinned versions are recorded in AD COURSEWORK 2.csproj.

4.4 Checking appsettings.json

Open appsettings.json in the project root. Confirm the following sections.

- ConnectionStrings:DefaultConnection contains the MySQL server, port, database name, user, and password to use during development.
- Email contains the SMTP host, port, sender name, sender email, username, and password used by the password reset flow. If email is not required for the marker's environment, the application still runs because email is only sent during password reset.

- Authentication:Google contains the optional Google client identifier and client secret. If both values are present, Google login is enabled in Program.cs. If either value is empty, Google login is disabled and the standard Identity login is used.

Figure UM2: [INSERT SCREENSHOT HERE]

This screenshot shows the configuration file with the connection string visible. Personal credentials must be redacted before the screenshot is included in the report.

The marker should not paste sensitive credentials such as the database password, the SMTP password, or the Google client secret into the report or into shared environments. The recommended practice is to redact these values in screenshots and to keep the actual configuration on the local machine only.

4.5 Configuring the Database Connection

The default connection string targets a local MySQL server on a non-standard port. The format is:

```
Server=localhost;Port=3307;Database=unimanager;User=root;Password=YOUR_PASSWORD;Tre
atTinyAsBoolean=true;CharSet=utf8mb4;
```

To adapt the connection string to the marker's machine, edit the Server, Port, Database, User, and Password parts as appropriate. If the local MySQL server uses the default port, change 3307 to 3306. If a different database name is preferred, change unimanager to the chosen name; the application will create the schema during the first run.

The marker can also place the connection string into appsettings.Development.json so that the value is only used during development. Sensitive credentials must remain outside of source control and outside of the report.

4.6 Applying Migrations or Creating the Database

The project applies pending migrations automatically at startup through Data/DbInitializer.cs. As soon as the application runs, the schema is created and the demonstration roles, users, and sample data are seeded.

If the marker prefers to apply migrations manually before the first run, open a PowerShell window in the project folder and run:

```
dotnet ef database update
```

The current migrations are:

- 20260408041036_InitialUniManage
- 20260501122117_AddAuditLogAndSecurity

- 20260501125135_AddMeetings

To list the migrations from the command line, run:

```
dotnet ef migrations list
```

The output is the evidence captured for Figure 17b in the main report. Raw SQL backup scripts and SQL Server / LocalDB scripts are not included in the project; the EF Core migrations are the supported approach.

4.7 Building the Project

To build the project from the command line, run the following from the project folder:

```
dotnet build --no-restore
```

The expected output is:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

Figure UM3: [INSERT SCREENSHOT HERE]

This screenshot shows the console output from dotnet build --no-restore with zero warnings and zero errors.

To build inside Visual Studio, select Build, then Build Solution from the top menu. The Output window displays the same zero warning and zero error result.

4.8 Running the Application

To run the application from the command line, use:

```
dotnet run
```

To run inside Visual Studio, press F5 or click the Start button on the main toolbar. The Visual Studio launch profile uses the HTTPS profile by default.

The detected launch URLs are:

- https://localhost:7212
- http://localhost:5103

The first run applies the migrations, seeds the demonstration roles and users, and starts listening on the launch URLs. Open a browser and navigate to the HTTPS URL. The home page is displayed. From there, the user can log in or register.

4.9 Troubleshooting Build Errors

Error	Likely cause	Suggested fix
The .NET 8 SDK is missing	Visual Studio components are out of date	Open Visual Studio Installer, modify the workload, and add the

		.NET 8 SDK.
NuGet restore fails	No internet access or NuGet feed not reachable	Confirm internet access, run <code>dotnet nuget list source</code> , and add <code>https://api.nuget.org/v3/index.json</code> if missing.
EF Core tools not found	Global tool not installed	Run <code>dotnet tool install --global dotnet-ef</code> .
Database connection refused	MySQL not running or wrong port	Start the MySQL service and confirm the port matches the connection string.
Access denied for the database user	Wrong user or password	Recreate the user, grant rights to the unimanage database, and update the connection string.
Migration fails	Database user does not have CREATE privilege	Grant the user the necessary privileges or use a higher-privilege user for the first run.
dotnet build fails	Stale obj/bin folders or wrong .NET SDK	Run <code>dotnet clean</code> and <code>dotnet build --no-restore</code> again.
Port already in use	Another process is using 5103 or 7212	Stop the conflicting process or change the port in <code>Properties/launchSettings.json</code> .
Static files not served	wwwroot missing or build skipped	Confirm that the wwwroot folder is present and that the build completed.
HTTPS certificate trust prompt	Development certificate not trusted	Run <code>dotnet dev-certs https --trust</code> once on the workstation.

5. First-Time Setup

The first time the application runs, DbInitializer creates three roles (Administrator, Lecturer, Student) and three demonstration users. Sample courses, enrolments, and assignments may also be created where the seeder has been configured to do so. The marker can sign in straight away using the demonstration credentials listed in Section 6.

If the seeded users are not visible after the first run, restart the application or check the logs for an error during seeding. The application logs the seeding result through the standard logger.

Figure UM4: [INSERT SCREENSHOT HERE]

This screenshot shows the login page after the first run.

Figure UM5: [INSERT SCREENSHOT HERE]

This screenshot shows the registration page with the role selector visible.

5A. Hosted Demonstration Access

In addition to the local installation route described in Section 4, the author has prepared a supporting hosted demonstration of UniManage on Google Cloud. The hosted endpoint is now reached through a custom HTTPS domain. The hosted route can be used during the viva, during marker review, or for any quick demonstration that does not require a local

development environment. The hosted route is supporting evidence only. The CS6004ES Application Development brief still expects the marker to assess the application using Visual Studio 2017 or higher, so the local installation route remains the primary path.

5A.1 Hosted URL

The main hosted demonstration URL is:

<https://lms.yasaboy.com/>

The earlier IP-based endpoint, which was used during the initial deployment phase and is retained as a backup in case the custom-domain endpoint is temporarily unreachable, is:

<http://34.171.6.234:8080>

The custom-domain URL uses HTTPS, so a modern browser displays the standard padlock indicator. The earlier IP-based URL uses HTTP and is provided only as a fallback evidence item.

5A.2 Browser Access Steps

40. Open a modern browser such as Microsoft Edge, Google Chrome, or Mozilla Firefox.
41. Go to <https://lms.yasaboy.com/>.
42. Confirm that the UniManage homepage loads. The address bar should display the padlock indicator that comes with a valid HTTPS certificate.
43. Use Sign in or Register depending on the demonstration task. The seeded demonstration credentials listed in Section 6 of this manual are sufficient.
44. If the hosted URL is unavailable, use the local Visual Studio run method described in Section 4 of this manual. The earlier IP-based endpoint <http://34.171.6.234:8080> may also be tried as a temporary backup.

5A.3 Expected Landing Page

The expected landing page is the UniManage public home page rendered by Views/Home/Index.cshtml. The page offers two primary actions: "Create your account" linking to /Account/Register and "I already have an account" linking to /Account/Login. The footer shows the project name and the copyright line.

5A.4 Login on the Hosted Version

After the home page loads, the marker can sign in using the seeded demonstration users defined in Data/DbInitializer.cs. The credentials are listed in Section 6 of this manual. The

hosted version uses the same role-based redirection as the local version, so signing in as the seeded administrator, lecturer, or student opens the appropriate dashboard.

The author has not exposed any additional credentials beyond the seeded demonstration users. Real student data, real lecturer data, and any production-style credentials are not used on the hosted demonstration.

5A.5 Availability Warning

The hosted demonstration depends on the underlying Google Cloud virtual machine and the Docker container being in a running state. Cloud instances can be stopped, restarted, or reconfigured between the time of writing and the marking session. Before relying on the hosted URL, the marker should verify it by opening it in a browser. If the custom-domain URL does not open, the earlier IP-based endpoint may be tried as a temporary fallback. If neither endpoint is reachable, please use the local installation route in Section 4 instead.

5A.6 Security Notes for the Hosted Demonstration

- The custom-domain endpoint uses HTTPS, but the wider deployment is still a coursework demonstration rather than a production service. The marker is asked not to enter real personal data on the demonstration site.
- Database passwords, SMTP passwords, Google client secrets, SSH keys, DNS or provider credentials, and cloud service account keys are not included in this manual or in the body of the report. The marker is asked not to add these values to any future copy of the document.
- If the marker chooses to demonstrate the hosted login on screen, the seeded demonstration credentials are sufficient. There is no need to expose any other account information.

Figure UM19: [INSERT SCREENSHOT HERE]

This screenshot shows the UniManage application running through the custom HTTPS domain.

Figure UM20: [INSERT SCREENSHOT HERE]

This screenshot shows the Google Cloud and Docker deployment evidence used to support the hosted demonstration.

6. Login and User Roles

The seeded demonstration accounts are created by Data/DbInitializer.cs. The credentials are intended for the viva and for marking, not for production use.

Role	Email	Password
------	-------	----------

Administrator	admin@unimanager.local	Admin123!
Lecturer	lecturer@unimanager.local	Lecturer123!
Student	student@unimanager.local	Student123!

To log in, open the application, navigate to the login page, enter one of the credentials above, and click Sign in. The system applies the password rules, the lockout policy (five failed attempts, fifteen minute lockout), and the rate limiter on the authentication endpoint (ten requests per minute per IP).

After a successful login, the user is redirected to the dashboard appropriate to their role. The Administrator goes to the Administrator dashboard, the Lecturer goes to the Lecturer dashboard, and the Student goes to the Student dashboard.

To register a new account, click the Register link on the login page. The user is asked for a full name, an email address, a phone number, a password, and a role (Student or Lecturer). The Administrator role is not available through self-registration. After a successful registration, the new user is signed in automatically and redirected to the role dashboard.

7. Student User Guide

After logging in, the Student is redirected to the Student dashboard. The dashboard shows enrolled courses, upcoming deadlines, recent grades, recent messages, the calendar, and meetings.

The Student can perform the following actions.

45. View enrolled courses. The dashboard lists the Student's current enrolments. Each course title links to a detailed view.
46. Browse the course catalogue. Use the navigation menu to open the course browse page. The page lists eligible courses with their code, name, credits, lecturer, prerequisite, and current enrolment count.
47. Enrol on a course. Click the Enrol button next to an open course. The system checks that the course is not full, that the Student has not already enrolled, and that any prerequisite has been satisfied. If the checks pass, the enrolment is saved and a confirmation message is displayed. If a check fails, an error message explains the reason.
48. View assignments. Open the Assignments link in the navigation menu. The page lists assignments for the Student's enrolled courses with the due date and status.
49. Submit an assignment. Open an assignment, enter the submission text, optionally attach a file, and click Submit. The system validates the file extension, the file size, and the Student's enrolment, then stores the submission.

50. View grades and feedback. Open the assignment after grading. The Student can read the grade and the lecturer's feedback. After grading, the submission is locked and cannot be edited.
51. Send a message to a lecturer. Open the Messages page, click Compose, choose a Lecturer who teaches a course on which the Student is enrolled, enter a subject and content, and click Send. The system enforces the allowed-recipient rule.
52. View meetings. Open the Meetings page. Meetings linked to the Student's enrolled courses are listed with the scheduled time and the join link.
53. Update profile. Open the Profile page to view and update the full name, email, and phone number.
54. Change password. Open the Change Password page from the Profile menu, enter the current and new passwords, and click Save.

Figure UM6: [INSERT SCREENSHOT HERE]

This screenshot shows the Student dashboard after a successful login.

Figure UM10: [INSERT SCREENSHOT HERE]

This screenshot shows the course browse page with the Enrol button visible.

Figure UM12: [INSERT SCREENSHOT HERE]

This screenshot shows the assignment submission form with text and file fields.

8. Lecturer User Guide

After logging in, the Lecturer is redirected to the Lecturer dashboard. The dashboard shows the Lecturer's teaching courses, enrolment counts, assignment counts, pending submissions, recently graded submissions, recent messages, recent meetings, and recent activity.

The Lecturer can perform the following actions.

55. View taught courses. Open the My Courses page. The page lists only the courses owned by the signed-in Lecturer.
56. Open course details. Click a course to open its details, including the description, credits, enrolment limit, lecturer, prerequisite, enrolled students, assignments, materials, and meetings.
57. Upload course materials. From the course details page, click Upload Material, enter a title, select a file, and click Save. The system validates the file extension and size and stores the metadata in the database. The actual file is stored on disk through the upload service.

58. Create an assignment. Open the Assignments page for an owned course, click Create, and complete the form. The form requires a title, description, due date, and maximum points. After submission, the assignment is saved and listed.
59. Edit or delete an assignment. From the assignments list, click Edit or Delete next to an assignment. The system checks that the assignment belongs to a course owned by the Lecturer.
60. View submissions. Click an assignment to view its submissions. Each submission shows the Student, the submitted time, the file name where applicable, and the status.
61. Grade a submission. Click Grade next to a submission, enter a grade and feedback, and click Save. The system validates the grade range and the feedback length. After grading, the submission is locked and the Student can see the grade and feedback.
62. Send a message to a Student. Open the Messages page, click Compose, choose a Student enrolled on a course owned by the Lecturer, and send the message.
63. Schedule a meeting. Open the Meetings page, click Create, select a course owned by the Lecturer, enter a title, scheduled time, duration, and meeting URL, then click Save. The system validates the URL format and the schedule.
64. Edit or delete a meeting. From the meeting list, click Edit or Delete next to a meeting. The Lecturer can only manage meetings for owned courses.
65. Update profile and change password. Use the Profile menu in the same way as the Student.

Figure UM7: [INSERT SCREENSHOT HERE]

This screenshot shows the Lecturer dashboard with teaching summary and pending submissions.

Figure UM11: [INSERT SCREENSHOT HERE]

This screenshot shows the assignment create form.

Figure UM13: [INSERT SCREENSHOT HERE]

This screenshot shows the grading form with grade and feedback fields.

Figure UM15: [INSERT SCREENSHOT HERE]

This screenshot shows the inbox or compose page.

Figure UM16: [INSERT SCREENSHOT HERE]

This screenshot shows the meeting create form.

9. Administrator User Guide

After logging in, the Administrator is redirected to the Administrator dashboard. The dashboard shows system-wide counts (users, courses, enrolments, assignments, submissions, messages), popular courses by enrolment count, and recent audit log activity.

The Administrator can perform the following actions.

66. View the dashboard. The dashboard provides a quick overview of the system.
67. Manage courses. Open the Courses page to list, search, page, view, create, edit, or delete courses. The course form requires the code (unique), name, description, credits, enrolment limit, lecturer, and optional prerequisite.
68. View reports. Open the Reports page to access the six reports: course popularity, student performance, lecturer workload, enrolments, pass and fail, and assignment attendance. Each report can be exported to CSV. Selected reports can be exported to PDF through QuestPDF.
69. Review the audit log. Open the Audit Logs page to inspect recent events. The log records the category, action, detail, user identifier (where available), success flag, and timestamp.
70. Manage meetings (where authorised). Administrators can review meetings as part of the academic oversight responsibility.
71. Update profile and change password. Use the Profile menu in the same way as the other roles.

A dedicated user-management screen is referenced in the brief but has not been confirmed in the source code. This is recorded as [VERIFY] in this manual and as [NEEDS CONFIRMATION] in the main report.

Figure UM8: [INSERT SCREENSHOT HERE]

This screenshot shows the Administrator dashboard with counts and popular courses.

Figure UM9: [INSERT SCREENSHOT HERE]

This screenshot shows the course list with search and paging visible.

10. Validation and Error Messages

The application uses data annotations on view models and additional checks in controller actions. The following validation behaviour is expected during normal operation.

Scenario	Expected Validation Message
Required field missing	The field is required.
Invalid email format	The Email field is not a valid email address.
Weak password	The password must be at least 8 characters and include an uppercase letter, a lowercase letter, and a digit.
Confirm password does not match	The new password and confirmation password do not match.
Course code already exists	A course with this code already exists.
Enrolment full	This course has reached its enrolment limit.
Already enrolled	You are already enrolled on this course.
Prerequisite not satisfied	You must complete the prerequisite course before you can enrol.
Wrong file extension	The file type is not allowed.

File too large	The file is larger than the allowed size.
Disallowed message recipient	You are not allowed to message this user.
Invalid grade	The grade must be within the allowed range.
Anti-forgery token missing	The form has expired. Reload the page and try again.
Rate limit hit	Too many requests. Please wait and try again.
Access denied	You do not have permission to view this page.

Figure UM17: [INSERT SCREENSHOT HERE]

This screenshot shows a form with one or more validation messages displayed.

The friendly error page Views/Shared/Error.cshtml is shown in production for any unhandled exception. The page does not reveal stack traces or internal server information. In development, the developer exception page provides detail to help locate the cause.

11. Reporting and Analytics

The Reports area is available to the Administrator. The following reports are implemented.

Report	Description	Source-Code Evidence
Course Popularity	Lists courses ordered by enrolment count.	ReportsController.CoursePopularity, Views/Reports/CoursePopularity.cshtml
Student Performance	Aggregates grades per student.	ReportsController.StudentPerformance, Views/Reports/StudentPerformance.cshtml
Lecturer Workload	Aggregates assignments and pending submissions per lecturer.	ReportsController.LecturerWorkload, Views/Reports/LecturerWorkload.cshtml
Enrolments	Lists enrolments with optional filters.	ReportsController.Enrollments, Views/Reports/Enrollments.cshtml
Pass and Fail	Groups submissions into pass and fail buckets using a configurable threshold.	ReportsController.PassFail, Views/Reports/PassFail.cshtml
Assignment Attendance	Shows the proportion of enrolled students who submitted each assignment.	ReportsController.AssignmentAttendance, Views/Reports/AssignmentAttendance.cshtml

To export a report, open the report and click the Export CSV or Export PDF button where present. The CSV file is generated through Infrastructure/CsvWriter.cs and the PDF file is generated through Infrastructure/PdfReport.cs using QuestPDF.

Figure UM14: [INSERT SCREENSHOT HERE]

This screenshot shows the reports landing page or a representative report output.

12. Security Notes for Users

The following security notes apply to all users of UniManage.

72. The application uses HTTPS by default during local development. If the browser shows a certificate warning, run `dotnet dev-certs https --trust` once to install the development certificate.

73. The application uses anti-forgery tokens on every form. Submitted forms must include the token; otherwise the request is rejected.
74. The application uses rate limiting on authentication and upload endpoints. Submitting too many requests in a short period of time produces a 429 Too Many Requests response.
75. The application uses account lockout. After five failed sign-in attempts, the account is locked for fifteen minutes.
76. Passwords are not stored in plain text. The Identity framework hashes passwords using PBKDF2 with a per-user salt.
77. Audit log entries are recorded for security-sensitive actions. The Administrator can review these entries through the Audit Logs page.
78. Sensitive configuration values (database password, SMTP password, Google client secret) must remain outside the report and outside any shared screenshots. The screenshots in this manual must be redacted.
79. Personal data shown in screenshots must also be redacted before inclusion in the final submission.

Figure UM18: [INSERT SCREENSHOT HERE]

This screenshot shows a database table such as Courses, Enrollments, or Submissions after a few actions, with personal data redacted.

13. Troubleshooting

Problem	Suggested fix
The application does not start	Confirm that the .NET 8 SDK is installed and that the connection string is valid. Inspect the console output for an exception.
The login page rejects valid credentials	Confirm that the seeded users have been created. Check the application logs and re-run the application.
The login page shows a rate-limit error	Wait one minute and try again. The auth policy permits ten requests per minute per IP.
The account is locked	Wait fifteen minutes for the lockout window to expire, or reset the password.
A POST form is rejected	Reload the page to obtain a new anti-forgery token, then submit again.
File upload fails	Confirm that the file extension is allowed and that the file is under the maximum size.
The dashboard is empty	Confirm that the seeded data has been created. The Administrator dashboard counts use the seeded users and any data added during testing.
The friendly error page is shown	Inspect the application logs for the exception detail; the page does not reveal stack traces.
Email is not delivered	Check the Email section in appsettings.json and confirm that the SMTP server is reachable.
Google login is not available	Confirm that the Authentication:Google:ClientId and Authentication:Google:ClientSecret values are populated. If either is empty, Google login is

	disabled by design.
Migrations fail	Confirm database privileges. Run dotnet ef migrations list to see the current migrations and dotnet ef database update to apply them.
The reports page is empty	Confirm that data has been seeded or added through the application. Empty reports indicate no underlying data, not an error.

14. Notes for Viva Demonstration

The viva demonstration follows the script in Appendix J of the main report. The notes below summarise the order in which the application is presented.

80. Open the solution. Open Visual Studio and load AD COURSEWORK 2.sln. Highlight the controller, model, view model, view, data, and infrastructure folders to demonstrate the MVC structure.
81. Build verification. Run dotnet build --no-restore from a terminal and show the zero warning, zero error result.
82. Start the application. Press F5 in Visual Studio or run dotnet run. Show the seeded URL <https://localhost:7212>.
83. Administrator demonstration. Sign in as admin@unimanager.local / Admin123!. Show the Administrator dashboard, the course list, the reports landing page, one report (for example course popularity) with CSV export, and the audit log review.
84. Lecturer demonstration. Sign out and sign in as lecturer@unimanager.local / Lecturer123!. Show the Lecturer dashboard, My Courses, an assignment, the grading screen, the materials upload, and a meeting create form.
85. Student demonstration. Sign out and sign in as student@unimanager.local / Student123!. Show the Student dashboard, the course browse page, an enrolment, an assignment submission, and an inbox conversation.
86. Validation demonstration. Trigger a validation error such as an empty required field or a duplicate course code, and show the friendly inline message.
87. Security demonstration. Show the audit log entry for a recent action. Optionally show the access denied page by attempting an unauthorised action as a Student.
88. Code walkthrough. Briefly walk through Program.cs to highlight Identity, EF Core retry on failure, the rate limiter (auth and upload policies), anti-forgery configuration, the security headers middleware, and the routing pattern.
89. Honest reporting. Briefly mention the items that remain [VERIFY] or [NEEDS CONFIRMATION], such as the dedicated user-management screen, the automated test

project, the hosted deployment, and the formal accessibility audit. The aim is to keep the viva grounded in evidence.

End of installation guide and user manual.